Sonder: Sequential Hypothesis Discrimination for Misconception Diagnosis in Grade-8 Mathematics

Project Team

Syed Ashher Majid	23I-0007
Ahmed Laiq	23I-0657
Muhammad Harris Azhar	23I-2535

Session 2023-2027

Supervised by

Dr. Ali Zeeshan Ijaz



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled:

Sonder: Sequential Hypothesis Discrimination for Misconception Diagnosis in Grade-8 Mathematics

Student name	**Reg. no.**	**Signature**
Syed Ashher Majid	23I-0007	_____
Ahmed Laiq	23I-0657	_____
Muhammad Harris Azhar	23I-2535	_____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

This report distinguishes the 12 October 2026 evaluation baseline from the report revision prepared on 21 October 2026. The implementation and contribution evidence are described in Chapter 8; the revised work allocation is proposed for agreement, not represented as a completed transfer.

Claude, Claude Code and OpenAI Codex assisted with content, implementation, tests, diagrams and report preparation as recorded in Section 8.7. This revision, its added diagrams, reproduction tools and wording were prepared with Codex assistance. The diagnostic calculation itself contains no generative-model call. Source checks are limited to the references and claims explicitly identified in Chapter 2; no blanket original-source verification is asserted.

Each member must personally check the work attributed to them, confirm the applicable GenAI level and sign and date their declaration after reviewing this revision. The fields below are intentionally uncompleted; an earlier confirmation cannot certify later work.

Member	Signature	Date
Syed Ashher Majid (23I-0007)		
Ahmed Laiq (23I-0657)		
Muhammad Harris Azhar (23I-2535)		

Abstract

Sonder investigates adaptive diagnosis of Grade-8 Mathematics misconceptions when one answer supports several explanations. Its deterministic engine updates a Bayesian posterior, selects questions by expected information gain and returns a diagnosis, no-misconception result or explicit refusal. The revised scope is Mathematics only, following the supervisor's 16 October 2026 action summary.

The documented implementation is the 12 October evaluation build, not a newly corrected product. Its candidate bank contains 117 authored items: 114 are assigned to diagnostic units and 111 load under the default exclusions. The build includes a pure Python engine, a local student interface, an in-memory service and a separately tested PostgreSQL schema. Automated checks and deterministic synthetic cases establish implementation behavior; they do not establish real-student accuracy. Always-A answers and a constructed partial-misconception pattern expose adverse diagnoses. The near-row stopping heuristic does not prove an absolute confidence ceiling.

External expert acceptance of the bank, empirical calibration, durable storage, authenticated teacher review and independent teammate reproduction remain outstanding. This revision corrects scope, evidence statements and traceability, adds sequence diagrams, and replaces unsupported percentages with versioned local reproduction outputs. The next milestone is a bounded, expert-reviewed Mathematics diagnostic with persistent evidence and a real teacher result view.

Acknowledgments

The project team thanks the project supervisor, Dr. Ali Zeeshan Ijaz, for guidance on scope and technical depth; the proposal panel, whose comments on coordination and technical challenge shaped this report; the teacher who gave the stakeholder interview; and the faculty of the FAST School of Computing. The misconception catalogue used to seed the Mathematics content is the one published by Eedi through the Kaggle competition “Mining Misconceptions in Mathematics”.

Executive Summary

Problem. Different misconceptions can predict the same option. Sonder studies how to select useful follow-up Mathematics questions under a bounded question budget and report insufficient evidence rather than force a label. Bayesian updating and adaptive selection are established methods; this project does not claim their invention or the absence of comparable systems.

Scope and version. The supervisor's *Sonder Mid Evaluation Action Summary*, dated 16 October 2026, directs Mathematics-only scope. The report describes evaluation commit `2c266d4` from 12 October and report corrections dated 21 October. Physics/Chemistry content and evaluation are removed from the active plan. Optional AI and communication features follow correction of the core.

Implemented baseline. A pure engine performs prior/likelihood/posterior calculations, expected-information-gain selection, stopping, immutable session transitions and evidence traces. The local student flow uses a service with an in-memory store, owner cookies and retry checks. PostgreSQL content/run tables and integrity rules are a separate schema artifact; the live baseline does not persist runs. Of 117 authored candidate items, 114 are assigned and 111 load after three P1 exclusions. The bank has student-author approval records but no demonstrated bank-wide external Maths-expert acceptance.

Evidence. Chapter 7 distinguishes local exact-build reproduction, historical CI reports and future acceptance tests. A deterministic 43-case synthetic suite gives 26 correct diagnoses, seven none-detected outcomes, eight no-separator refusals and two exhausted runs. These cases follow the assumed model and do not measure educational accuracy. Adverse always-A and partial-holder policies remain visible. New outcome and library-timing figures are generated from retained raw records; they are not an adaptive-versus-baseline evaluation or an end-to-end performance claim.

Corrections and remaining work. R5 now has explicit traceability, proposed ownership, a conditional milestone and an acceptance test. Content counts, panel dates, scope and ownership are reconciled. SSDs and SDs distinguish current memory behavior from proposed database and teacher workflows. The Chen-notation ERD is supplied with a link to its clearer online copy. The report qualifies the near-row refusal rule: only exact equality preserves likelihood ratios. Remaining work requires expert item decisions,

exact/near-row regressions, seeded equal-budget experiments, durable replay/retry checks and a real teacher evidence view. No signature, expert decision or teammate verification is fabricated by this revision.

Contents

Students' Submission i

Anti-Plagiarism Statement iii

Authors' Declaration v

Abstract vii

Acknowledgments ix

Executive Summary xi

List of Abbreviations xxiii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 3
1.6 Project Scope . . . 3
1.7 Project Modules . . . 4
1.8 Stakeholders and User Classes . . . 5
1.9 Success Criteria . . . 5
1.10 Constraints and Assumptions . . . 6
1.11 Ethical, Legal and DEI Considerations . . . 6

2 Research and Existing Solutions 7

2.1 Research Method / Approach . . . 7
2.2 Existing Solutions / Products . . . 7
2.3 Comparison of Existing Solutions . . . 8
2.4 Limitations and Gaps . . . 8
2.5 Proposed Solution . . . 8
2.6 Complex Computing Problem . . . 9

2.7 Technical Challenges . . . 9
2.8 Research Findings and Design Decisions . . . 10

3 Proposal Feedback and Iteration Plan 11
3.1 Proposal Feedback Summary . . . 11
3.2 Proposal Action Register . . . 11
3.3 Iteration Strategy . . . 12
3.4 Iteration 1 Plan . . . 12
3.5 Iteration 2 Plan . . . 12
3.6 Work Breakdown and Responsibilities . . . 13
3.7 Progress Against Plan . . . 14
3.8 Delays, Deviations and Recovery Plan . . . 14

4 Software Requirements 15
4.1 User Interactions / Use Cases . . . 16
4.2 Use Case Diagram . . . 17
4.3 Detailed Use Cases . . . 18
4.4 Functional Requirements . . . 19
4.4.1 Module 1: Diagnostic engine . . . 19
4.4.2 Module 2: Content store . . . 20
4.4.3 Deferred product requirements . . . 20
4.5 Non-Functional Requirements . . . 20
4.5.1 Reliability . . . 20
4.5.2 Usability . . . 21
4.5.3 Performance . . . 21
4.5.4 Security . . . 21
4.6 Acceptance Criteria . . . 22
4.7 Requirements Traceability . . . 23

5 System Design 25
5.1 System Architecture . . . 25
5.2 Architectural Components . . . 26
5.3 Interfaces and Dependencies . . . 27
5.4 Architecture Decisions and Rationale . . . 28
5.5 Design Evolution . . . 29
5.6 Domain Model . . . 30
5.7 Data Design . . . 30
5.8 Process / Behavior Design . . . 35
5.8.1 Interaction design: scope and notation . . . 35
5.8.2 System sequence: take a diagnostic . . . 36
5.8.3 Internal sequence: answer, retry and publication . . . 36

5.8.4 Internal sequence: diagnostic calculation and refusal 41
5.8.5 Target content acceptance and freezing workflow 44
5.8.6 Data flow: one diagnostic step 45
5.8.7 State diagram: session outcomes 46
5.8.8 State diagram: content lifecycle 46

6 Iteration 1: Design and Implementation 47
6.1 Iteration Objectives 47
6.2 Selected Use Cases / Features 47
6.3 Design 47
6.4 Implementation 48
6.5 Completed Features / Module 48
6.6 Technical Implementation Details 49
6.6.1 Setup and execution (to reproduce) 49
6.6.2 Calculation 49
6.6.3 Selection 50
6.6.4 Stopping and refusal 50
6.6.5 Pack loading 51
6.7 Algorithms and Technical Decisions 52
6.8 Tools, Frameworks and Technologies 52
6.9 Demonstration Evidence 53
6.9.1 Normal case: worked example D1 (unit U1, MATH-1379 student) 53
6.9.2 Edge case: worked example D2 (unit U4, inseparable pair) 53
6.9.3 The same examples in the browser 54
6.9.4 Failure cases 54
6.10 Iteration Outcome 54

7 Testing and Early Validation 55
7.1 Testing Strategy 55
7.2 Test Levels and Techniques 55
7.3 Test Cases 56
7.4 Test Results 56
7.4.1 Performance 57
7.5 Acceptance Testing / Validation 58
7.6 Edge and Failure Cases 59
7.7 Defects and Corrective Actions 60
7.8 Current Validation Status 60

8 Project Management and Development Evidence 61
8.1 Work Completed 61
8.2 Work Remaining 62

8.3 Individual Responsibilities . . . 62
8.4 Contribution of Each Team Member . . . 62
8.5 Repository and Version Control Evidence . . . 63
8.6 Milestones and Commit Evidence . . . 64
8.7 Tools and Tool-Assisted Development . . . 64
8.8 External Components and Dependencies . . . 67
8.9 Risks and Mitigation . . . 68
8.10 Security and Privacy Considerations . . . 68
8.11 Current Limitations . . . 69
8.12 Peer Review and Confirmation Evidence Still Required . . . 69

9 Remaining FYP Plan **71**
9.1 Remaining Scope . . . 71
9.2 Remaining Modules and Features . . . 71
9.3 Remaining Iterations . . . 72
9.4 Implementation Plan . . . 72
9.5 Testing and Validation Plan . . . 72
9.6 Deployment Plan . . . 73
9.7 Tentative Timeline . . . 73
9.8 Expected Final Deliverables . . . 73

10 Conclusion **75**

A Proposal Action Register **77**

B Detailed Requirements **79**

C Detailed Test Cases **83**
C.1 Deterministic students through the run loop . . . 83
C.2 Injected faults (mutation check) . . . 83
C.3 Selection test classes (`tests/test_selection.py`) . . . 84
C.4 Commands to reproduce . . . 84

D Repository and Contribution Evidence **85**
D.1 Version and Complete Source Checkout . . . 85
D.2 Both Services and Expected Output . . . 86
D.3 Content and Parameter Identity . . . 87
D.4 SQL Checks and Their Execution Status . . . 87
D.5 Reproduction Record and Sign-off . . . 88

E Additional Design Diagrams **89**
E.1 Supplementary interaction diagrams . . . 89

E.2 Supplementary data diagrams 104

F Meeting and Progress Evidence **107**

List of Figures

4.1 Use case diagram of the Sonder system. Student, Teacher and Parent are primary actors; Content Reviewer is a secondary actor. Parents view only teacher-approved diagnosis summaries for their linked child. 17

5.1 Maths-only architecture: solid elements describe baseline 2c266d4; dashed elements are proposed persistence and teacher integration. The current service holds runs in memory and is lost on restart. The candidate content awaits external expert acceptance. 25

5.2 Domain model of the diagnostic core, in crow's foot notation. At each end of a line: a bar means one, a circle means zero, a crow's foot means many. Each name reads from the entity above to the entity below. A session ends in at most one of diagnosis or escalation (refusal). 30

5.3 ERD: Mathematics content pack (subject, topics, misconceptions, items, options, Q-matrix links, sources, reviews). 32

5.4 ERD: diagnostic units (ordered hypothesis slots and item positions) and the engine model with its frozen inputs (parameters, response model, probabilities, engine release). 33

5.5 ERD: diagnostic runs (session, hypotheses, questions, candidate ranking, steps, posteriors, diagnosis, escalation). 34

5.6 SSD-01: UC1, take a Mathematics diagnostic (R1, R2, R8, R9). Evaluation-build local student flow; D1 completes after four answers. . . 37

5.7 SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 1 of 3). . . . 38

5.8 SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 2 of 3). . . . 39

5.9 SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 3 of 3). . . . 40

5.10 SD-03: UC1 computation (R1, R2, R8): Bayesian update, information-gain selection and ordered stopping. The R4 alternative explains SSD-02. (part 1 of 2). 42

5.11 SD-03: UC1 computation (R1, R2, R8): Bayesian update, information-gain selection and ordered stopping. The R4 alternative explains SSD-02. (part 2 of 2). 43

5.12 Target content workflow, not a completed acceptance claim: the expert-review gate is outstanding. NONE, UNMODELLED and units precede freezing; student-owner approval and agent audits alone do not pass this gate. 44
5.13 Evaluation-baseline computational flow; ranking and trace are in memory. R4 is tolerance-based and has the exact/near-row limitation described here. 45
5.14 Baseline state machine; near-row R4 correctness remains open (target database column `diagnostic_session.status`). All outcomes are terminal; rules are evaluated in order R1 to R5 after each update. 46
5.15 Content state machines enforced by the database (`state_guard`). Approval and rejection happen only by a review by a non-author. 46

7.1 Counts from `deterministic-students.csv`; 43 deterministic sessions on the 111 loaded candidate items in seven units, default assumed parameters and budget 8. There is no randomness or real-student sample. Counts verify baseline behavior, not empirical accuracy. 57
7.2 Raw-backed first-step timing distributions from `step-latency.csv`, 100 observations per unit. Boxes show quartiles/median, whiskers the default 1.5-IQR range and points outliers. Units are milliseconds; settings are default Params, unchanged baseline, one local process. The proposed 300-ms complete-path requirement is not established by this library-only measurement. 58

E.1 SSD-05: UC1 interrupted-response recovery (R8, R9). Available memory runs can be retried; a restarted engine loses the run. 90
E.2 SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 1 of 3). 91
E.3 SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 2 of 3). 92
E.4 SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 3 of 3). 93
E.5 SSD-02: UC1 refusal alternative (R2, R9). Worked example D2 returns the same neutral completion message as a supported diagnosis. 95
E.6 SSD-03: UC5 content-item review (R6). Schema-supported contract; reviewer UI proposed. 96
E.7 SSD-04: UC6 content-pack freeze (R6). Schema-supported contract; administration UI proposed. 97

E.8 SD-01: UC1 session start (R1, R8, R9). The loaded pack feeds the immutable engine; a memory-store entry holds the student snapshot and private trace. (part 1 of 2). 98
E.9 SD-01: UC1 session start (R1, R8, R9). The loaded pack feeds the immutable engine; a memory-store entry holds the student snapshot and private trace. (part 2 of 2). 99
E.10 SD-04: UC5 content review (R6). PostgreSQL applies a valid non-author review and records the audit; invalid writes roll back. The controller is proposed. (part 1 of 2). 100
E.11 SD-04: UC5 content review (R6). PostgreSQL applies a valid non-author review and records the audit; invalid writes roll back. The controller is proposed. (part 2 of 2). 101
E.12 SD-05: UC6 content freeze (R6). The database checks the content freeze gates and locks the version; response-model freezing follows separately. The controller is proposed. (part 1 of 2). 102
E.13 SD-05: UC6 content freeze (R6). The database checks the content freeze gates and locks the version; response-model freezing follows separately. The controller is proposed. (part 2 of 2). 103
E.14 ERD: users and access, classes and enrolment, and audit. 104
E.15 Team-supplied Chen-notation ERD of the diagnostic core, from Full_ERD_Chen_Notation.png. This logical overview is not a one-to-one export of all 41 physical SQL tables; the area diagrams document those details. 105

List of Tables

1.1 Stakeholder interview (one Grade-8 teacher). 2
1.2 Project modules and Iteration 1 state. 4
1.3 User classes. 5
1.4 Measurable success criteria and current status. 5

2.1 Source-bounded comparisons; unverified features remain unassessed. . . . 8
2.2 Design trade-offs to evaluate rather than universal capability rankings. . . 9
2.3 Computing complexity grounded in this implementation. 9
2.4 Findings and current or proposed responses. 10

3.2 Revised Maths-only iteration plan; dates require agreement. 12
3.3 Proposed next-review packages from the supervisor summary; agreement pending. 13
3.4 Baseline packages and revised acceptance gaps. 14

4.1 Use cases by actor. "I1" marks use cases whose engine part is built in Iteration 1. 16
4.2 UC1 — Answer diagnostic questions (engine path). 18
4.3 UC5/UC6 — Review items and freeze a content pack. 18
4.4 Engine functional requirements. 19
4.5 Content functional requirements. 20
4.6 Product functional requirements outside Iteration 1. 20
4.7 Acceptance criteria and verification. 22
4.8 Requirement → design → implementation → test → owner. 23

5.1 Components and their Iteration 1 state. 26
5.2 Architecture decisions. 28
5.3 Design changes since the proposal. 29
5.4 Selected integrity rules enforced by the database. 35
5.5 Interaction diagrams and their requirements evidence. 44

6.1 Implemented modules (all under `sonder_engine/` unless stated). 48
6.2 Stopping and refusal rules (defaults: $\theta = 0.80$, $\theta_U = 0.50$, `min_support`=2, `min_support_none`=3, $\eta = 0.01$ bits, budget 8). 51
6.3 Technical decisions and open items (spec decision IDs). 52

6.4 Tools and technologies. 52
6.5 Worked example D1 on unit U1 (rounding). Top posterior entries after each answer. 53

7.1 Verification levels and their limits. 55
7.2 Locally reproduced deterministic synthetic outcomes, baseline 2c266d4; 43 sessions. 56
7.3 21 October local library timing; 100 first-step observations per unit. . . . 57
7.4 Retained adverse and failure behavior. 59
7.5 Report corrections versus uncompleted technical corrections. 60
7.6 Known-fault groups retained as open limitations. 60

8.1 Contribution by member (pull requests merged into `dev`, counted by branch owner; #3 is a `dev` to `main` synchronisation). 62
8.2 Milestones (merge dates from the repository history). 64
8.3 GenAI register for materially assisted artefacts. 65
8.4 External components, risks and fallbacks. 67
8.5 Risk register. 68

9.1 Remaining work in dependency order. 71
9.2 Maths-only planned studies and required controls. 72

D.1 Exact baseline content and assumed-parameter hashes. 87

List of Abbreviations

FYP	Final Year Project
FR / NFR	Functional / Non-Functional Requirement
CD-CAT	Cognitive Diagnostic Computerized Adaptive Testing
EIG	Expected Information Gain (mutual information between the hypothesis and the next answer, in bits)
Q-matrix	Table linking each answer option to the misconception(s) it indicates
NONE	Engine hypothesis: the student holds no catalogued misconception
UNMODELLED	Engine hypothesis: the answers are not explained by any catalogued misconception
TV	Total variation distance between two answer distributions
P1, V5′, R1–R5	Engine specification rules: response-model check, unit-membership rule, stopping and refusal rules
ERD	Entity-Relationship Diagram
CI	Continuous Integration
PR	Pull Request
GenAI / LLM	Generative AI / Large Language Model
RBAC	Role-Based Access Control
SDG	Sustainable Development Goal

Chapter 1

Introduction and Project Vision

1.1 Project Overview

Sonder (formerly ConceptBridge) is a teacher-supervised platform that diagnoses specific misconceptions in Grade-8 Mathematics and helps students correct them. A student answers multiple-choice diagnostic questions. The system maintains a probability for each candidate misconception, picks the follow-up question that best tells the candidates apart, and either reports a diagnosis or refuses and escalates to a teacher. A teacher reviews each outcome before it reaches a student or parent. The planned product then delivers study material and re-checks the correction with a fresh question, immediately and after a delay.

The project belongs to Stream B of the FAST final-year-project scheme: the product has a bounded teacher workflow, a primary stakeholder (the teacher) and committed acceptance criteria, with the CD-CAT method as the research anchor of its core module. This revision follows the supervisor's 16 October 2026 action summary: the active project is Mathematics only. The earlier STEM title and three-subject proposal are historical, not active dataset or evaluation commitments. The implementation evidence remains the 12 October evaluation baseline; later product fixes are not assumed verified.

1.2 Problem Statement

Given a student response that is consistent with more than one misconception, select the next diagnostic item that maximises the expected reduction in posterior uncertainty under a fixed question budget, while evaluating wrong diagnoses, coverage and refusal together.

1.3 Problem Context and Motivation

In a fixed one-label diagnostic design, a wrong answer can be assigned a label even when several explanations fit. Sonder investigates that specific ambiguity in its candidate Maths bank. This is a problem formulation, not a claim that all existing assessment products use fixed quizzes or lack adaptive diagnosis.

A stakeholder interview with one teacher (Table 1.1) confirms the premise. The teacher checks working or asks the student to explain; records of misconceptions are mostly kept in memory; class-wide patterns are hard to see without a tool; and, asked whether the same wrong answers always mean the same misunderstanding, the answer was "No. Different reasons can lead to the same wrong answer." The teacher would double-check a tool that shows uncertainty at first and trust it more once it proved reliable. This is one teacher, not the two to three teachers and ten to twelve students the interviewee recommended, so it is a first data point and not a completed study.

Table 1.1: Stakeholder interview (one Grade-8 teacher).

Question	Answer
How do you find out why a student got an answer wrong?	By checking their working or asking them to explain.
Do you keep a record of misconceptions?	Mostly it stays in the teacher's memory.
Do the same wrong answers always mean the same misunderstanding?	No. Different reasons can lead to the same wrong answer.
How much time is spent finding the reason behind mistakes?	Limited, as teachers have many students and tasks.
Would you trust a new tool that shows uncertainty?	Initially I would double-check it, but I would trust it more if it proved reliable.

The work supports Sustainable Development Goal 4 (quality education) as motivation; the contribution claimed is computational, not an efficacy claim.

1.4 Project Vision

Sonder should let a teacher see, for any diagnosed student, which misconception the evidence supports, how strongly, which alternatives remain, and the question-by-question trail behind it, and should say plainly when the evidence is not enough. One deterministic engine serves the declared Mathematics units; differences live in reviewed content rather than unit-specific code.

1.5 Project Objectives

O1. Maintain a Bayesian posterior over a unit's misconceptions, plus `NONE` and `UNMODELLED`, and update it after every answer.

O2. Select each next question by expected information gain, deterministically and with a stored explanation.

O3. End a session with a diagnosis, a "none detected" result or an explicit refusal (no separating question, budget exhausted, unmodelled), with explicit refusal when the implemented rules trigger.

O4. Store every question, candidate ranking, answer, posterior and decision as an evidence trail.

O5. Build versioned, reviewed content packs; Mid-1 target: an externally accepted Mathematics subset.

O6. Meet the proposal's targets at evaluation time: at most 5% confident misdiagnoses with at least 70% coverage on held-out data; posterior update plus selection under 300 ms; Cohen's $\kappa \geq 0.7$ between the engine and an independent teacher.

1.6 Project Scope

Included in the approved project. The diagnostic engine; the content store (Q-matrix, misconception catalogue, question bank) for Mathematics; student, teacher, parent and admin views; teacher approval, correction and escalation; student interventions (study notes, video links); a verification loop with a delayed re-check; classes, join codes, announcements and direct messaging; a self-improving question-bank pipeline; an AI consultant, learning hub and concept tutor chat (bounded and read-only with respect to the diagnosis); containerised deployment with a 2 Mbps target; and a classroom pilot with consent.

Built in Iteration 1 (Mid-1). Mathematics only: the candidate Maths_v2 bank (student-owner approved; external expert acceptance pending), the engine library with stopping rules, run loop and evidence trail, a local engine service with a live student Mathematics diagnostic in the user interface, the database schema with automated checks, design artefacts, tests and the pack validator. The other interface screens remain the scripted proof of concept.

Excluded. Automatic curriculum generation; diagnosing every Grade-8 misconception; efficacy claims from a small pilot; real-time learning from live data of minors; clinical

or psychological assessment; and any LLM or retrieval result determining the live diagnosis. Additional-subject content is outside the revised scope. Optional features follow acceptance of the corrected core.

The baseline contains 117 authored candidate items, 114 assigned to units and 111 loaded at default settings. External expert acceptance and empirical accuracy are separate outstanding requirements.

1.7 Project Modules

Table 1.2: Project modules and Iteration 1 state.

Module	Responsibility	State at mid-report
Diagnostic engine	Posterior update, EIG selection, stopping and refusal, run loop, evidence trail	Library built and tested; connected to the student screen through a local service
Content store	Misconception catalogue, items, options, Q-matrix, reviews, frozen packs	Maths_v2 pack and schema built; production import into the hosted database not yet done
Database	Versioned content, runs, audit, users and classes (41 tables)	Written and tested automatically in CI; live runs not yet saved to it
Product layer	Student, teacher, parent, admin screens; classes	Student Mathematics diagnostic live; other screens scripted
Teacher gate and interventions	Approve, correct, reject; study notes; verification loop	After G4/G5
AI consultant, tutor, question-bank pipeline	Bounded assistance and content generation	Conditional after core acceptance
Deployment and evaluation	Containers, low-bandwidth, security, pilot, baselines	Iterations 3 to 4

1.8 Stakeholders and User Classes

Table 1.3: User classes.

Class	Characteristics and needs
Student (primary)	Grade-8; answers diagnostic questions, receives teacher-approved remediation; sees an explanation, not raw probabilities.
Teacher (primary)	Time-poor; needs the full evidence trail, binding authority to approve, correct or reject, and a queue of refusals to resolve.
Parent (secondary)	Wants a plain-language summary, only after teacher approval.
Admin / content reviewer	Maintains the catalogue and coverage, checks content provenance and views aggregate performance.

1.9 Success Criteria

Table 1.4: Measurable success criteria and current status.

Criterion	Target	Status
Confident misdiagnosis and coverage, evaluated together	$\leq 5\%$ confident misdiagnosis with $\geq 70\%$ coverage on held-out data	Not testable: no real-student data
Latency	Posterior update plus selection $<$ 300 ms	See raw-backed local library measurements in Chapter 7; full-path target unverified
Teacher agreement	Cohen's $\kappa \geq 0.7$ engine vs. independent teacher	Not yet measured
Mid-1 (this report)	Worked examples reproduce; tests pass; invalid input rejected	Baseline behavior reproduced; reliability and acceptance gaps remain (Chapter 7)

1.10 Constraints and Assumptions

- Time: eight months for the project; Iteration 1 was compressed (the repository was created on 9 October 2026 from existing documents).
- The item bank is synthetic and remains a candidate: items were drafted with an LLM, approved by Harris as the student content owner, and topic T1 was checked by Ashher. These checks do not replace independent qualified Maths-expert acceptance. No real student has answered it, so every probability (slip 0.10, guess 0.30, priors, thresholds) is assumed.
- The active scope is Mathematics only; there is no remaining additional-subject content assignment.
- The diagnostic path must be deterministic and auditable. Generated content cannot enter the live bank without human review.
- Deployment target: low-bandwidth (about 2 Mbps), low-end Android devices, containerised.
- A pilot with minors requires institutional approval, guardian consent and student assent, which have not yet been obtained.

1.11 Ethical, Legal and DEI Considerations

Equity. The 2 Mbps target and low-end device support keep the tool usable beyond well-connected schools; content is kept curriculum-neutral. **Transparency.** Refusals are disclosed to the teacher as outcomes with evidence, not hidden. **Privacy.** No student data has been collected; the proposal commits to pseudonymous identifiers, data minimisation, written guardian consent, student assent, withdrawal rights and formal ethics approval before any pilot. **Legal.** No copyrighted instrument text is redistributed; the Eedi catalogue is used to seed misconception names, and its licence terms are still to be recorded. **Responsible AI.** The generated items require independent external expert acceptance; the engine never calls a model; GenAI use is disclosed in Section 8.7.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

The report combines one teacher interview, inspection of the candidate Maths bank and evaluation-build code, and a claim-level review of primary sources. The references below were checked at the publisher/author or product-owner page for the stated claim; this is not a claim that every historical source was read in full. The unsupported EDGE comparison is removed. Earlier FAST-project names are not used as evidence of missing capabilities without an accessible artifact and reproducible comparison protocol.

2.2 Existing Solutions / Products

Wang et al. describe multiple-choice diagnostic distractors representing misconceptions and a NeurIPS challenge that includes personalized question selection [1]. Consequently, describing Eedi-related research as only a fixed, non-adaptive map would be inaccurate. Cheng explicitly studies computerized adaptive testing for cognitive diagnosis [2]; adaptive cognitive diagnosis is prior work.

ALEKS' own overview describes selecting assessment questions using previous responses to estimate mastered topics and readiness [3]. ASSISTments' own description identifies curriculum-aligned assignments, immediate student feedback and teacher performance data [4]. These positive descriptions do not establish whether either platform implements a particular misconception-level refusal or approval mechanism. MATHia and DIAGNOSER were named in the proposal, but no current claim-level capability audit is supplied here; they are not assigned negative capability ratings.

2.3 Comparison of Existing Solutions

Table 2.1: Source-bounded comparisons; unverified features remain unassessed.

Source	Supported observation	Limit of inference
Wang et al. [1]	Diagnostic distractors and personalized question-selection research	Not a complete audit of the current Eedi product
Cheng [2]	Adaptive selection for cognitive diagnosis	Does not validate Sonder's content or stopping rules
ALEKS [3]	Response-adaptive knowledge assessment	Named-misconception refusal and teacher-release workflow not assessed
ASSISTments [4]	Student feedback and teacher assignment data	Exact internal diagnostic model not established by this source
Sonder baseline	Replayable posterior, item ranking and coded refusals in local tests	Expert acceptance, calibration, persistence and teacher release incomplete

2.4 Limitations and Gaps

The specific project problem is ambiguity in the candidate Grade-8 Maths option-to-misconception mapping, with limited independent separating questions and a budget of eight. The proposed contribution is an evaluated, auditable implementation with appropriate refusal and teacher oversight for this bounded bank. No claim is made that no existing or deployed system combines similar capabilities. The present evidence does not prove the content is valid or that the local model is more effective than alternatives.

2.5 Proposed Solution

The engine combines explicit likelihood assumptions with Bayesian updating and Shannon information gain [5]. These choices make decisions inspectable and replayable without a model service; they do not make posterior confidence calibrated. A versioned content interface and an evidence-preserving product layer surround the pure calculation. An LLM may assist offline drafting but cannot authoritatively change a live diagnostic result.

Table 2.2: Design trade-offs to evaluate rather than universal capability rankings.

Approach	Useful property	Required evidence
Explicit Bayesian model	Transparent likelihoods and repeatable updates	Content validity, calibration and stopping regressions
Fixed/random item policies	Simple comparison policies	Same content, budget, truth labels and evaluation cases
Rule or LLM comparator	Alternative classification or decision policy	Fully specified implementation, prompts/settings and fair held-out comparison before claims

2.6 Complex Computing Problem

Given ambiguous Maths responses, select questions that reduce uncertainty while controlling incorrect diagnoses, refusal and question cost. The difficulty is the interaction between limited evidence, noisy/partial application of a misconception, weak mappings and honest reporting. Established methods can still require substantial engineering and validation; novelty is not inferred from a missing reference.

Table 2.3: Computing complexity grounded in this implementation.

Characteristic	Evidence or design consequence
Conflicting requirements	Fewer questions, lower wrong-diagnosis rate and adequate coverage must be evaluated together
Data limitations	Shared options, sparse trigger groups, disputed links and no observed student frequencies
Depth of analysis	Bayesian assumptions, information gain, exact versus near separability and policy baselines
Stakeholder consequences	A wrong confident result can direct inappropriate intervention; teachers need inspectable evidence
System integrity	Retry, replay, ownership and durable trace consistency must agree across boundaries

2.7 Technical Challenges

Two exact indistinguishable misconception pairs, sparse companion coverage, three excluded P1 items and three unplaced items limit the current bank. Eleven unchanged-number rounding links and approximately 90 audit-rated stretch links require qualified

external review. An always-A policy and a constructed partial-holder response are adverse cases, not evidence of valid diagnoses. Near-equal likelihood rows may accumulate evidence, so the R4 threshold cannot establish a mathematical ceiling in the general case.

2.8 Research Findings and Design Decisions

Table 2.4: Findings and current or proposed responses.

Finding	Response
Some authored items cross topic labels	Use declared diagnostic units and check linked-hypothesis closure
Repeated stems are dependent evidence	Exclude twins after one is presented
Three predicted options violate P1 at guess 0.30	Keep exclusions visible; content fix and expert decision remain open
Disputed mappings lack external acceptance	Retain candidate status; record item-level expert decisions before release
R4 uses near-row tolerance	Qualify its interpretation; exact/near-row regression and revised specification remain required
Adverse policies produce diagnoses	Retain counterexamples and compare policies with equal budgets; no unsupported percentage improvement claim

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The 16 October action summary identifies the proposal panel as 11 September 2026. Its technical-challenge, usefulness and coordination concerns remain open until dated corrective evidence is reviewed. A meeting on 9 September precedes those comments and cannot be cited as their resolution. This report records the later supervisor direction to restrict the active scope to Mathematics; it does not invent a meeting minute or a completed technical fix.

3.2 Proposal Action Register

Feedback	Report response	Closure evidence still needed
Weak coordination and uneven effort	Baseline contribution record retained; one bounded package per member proposed	Actual challenged PR issue, response and verification; agreed owners/dates
Weak technical challenge	Define Maths ambiguity, adverse policies, exact/near-row issue and equal-budget evaluation	Reproducible results and corrected specification/tests
Usefulness to students/teachers	Prioritise persistent evidence and real teacher result view before optional AI/communication	Restart/retry/replay checks and teacher view consuming real records

Feedback	Report response	Closure evidence still needed
Discuss scope with supervisor	16 October summary directs Maths only; this report's active plan is revised	Team confirmation and corresponding repository roadmap update after review

3.3 Iteration Strategy

Table 3.2: Revised Maths-only iteration plan; dates require agreement.

Iteration	Scope	Gate
1 baseline	Candidate Maths content, pure engine, local memory service, schema and evidence	FYP-I mid baseline, reviewed 16 October
2 priority	Expert-accepted Maths subset, reliability corrections, equal-budget experiments, persistent U1 path and teacher view	FYP-I final; acceptance-gated
3 conditional	Authenticated approval/release, interventions and verification; optional communication only after core acceptance	FYP-II mid
4	Deployment, access/security audit and consented Maths pilot if approvals allow	FYP-II final

3.4 Iteration 1 Plan

The baseline objective was an interface-connected Maths calculation and trace. The local numerical/session flow exists, but dataset acceptance, diagnostic reliability and the teacher service are incomplete. Working software and model-consistent tests do not close those acceptance gaps.

3.5 Iteration 2 Plan

The revised priority is a bounded corrected core, not additional subject content or optional features. Correct the Maths bank with an independent expert; investigate adverse policies and stopping semantics; implement persistent U1 records; and connect a teacher evidence view after that contract is ready. Subsequent authenticated correction, approval and release precede trusted student use.

3.6 Work Breakdown and Responsibilities

Table 3.3: Proposed next-review packages from the supervisor summary; agreement pending.

Member	Main package	Acceptance support
Harris	Corrected Maths release, external expert process, research/report and content diagrams	Ashher checks consistency/coverage; Ahmed reproduces import hashes
Ashher	Diagnostic correctness, exact/near-row tests, baseline evaluation and plots; teacher evidence view after persistence contract	Harris reproduces results; Ahmed checks session integration
Ahmed	Persistent U1 path, API contract and reproducible setup	Ashher independently checks restart, resume, retry and replay

These are proposed assignments, not signed transfers or completed work. Available capacity and calendar dates must be agreed before commitment; implementation must not default back to Ahmed.

3.7 Progress Against Plan

Table 3.4: Baseline packages and revised acceptance gaps.

Package	State	Evidence / remaining gap
I1-01 process	Partial	Branch/PR/CI artifacts exist; reviewer assignments alone are not review evidence
I1-02 design	Report revised	New SSDs/SDs, clarified statuses and Chen ERD; team design verification pending
I1-03 content	Candidate	117 authored, 114 assigned, 111 loaded; external expert acceptance missing
I1-04 engine	Implemented baseline	Calculations replay; adverse diagnoses and near-row R4 interpretation remain unresolved
I1-05 live flow	Local checkpoint	Memory only; restart loses runs; durable U1 path outstanding
I1-06 classes/views	Planned	No real login, enrolment authorization or teacher result service in the baseline
I1-07 validation	Partial	Exact-build local evidence in Chapter 7; empirical accuracy and peer reproduction pending
I1-08 submission	Revision candidate	Recompiled report; member declarations and supervisor/team verification remain pending

3.8 Delays, Deviations and Recovery Plan

Earlier documents promised persistence on 18 October and a teacher view on 25 October under Ahmed. Neither is established as complete in the evidence reviewed here. Those dates are superseded as unverified historical promises, not replaced with invented dates. The revised dependency is: Ahmed's persistent U1 contract, then Ashher's teacher view, then authenticated review/release. Calendar dates and the handoff must be agreed in the next recorded review. No Physics/Chemistry work remains assigned.

Chapter 4

Software Requirements

The active scope is Mathematics only. Current behavior is the 12 October memory-based baseline; enrolment, real login, durable storage and authorized teacher release are target functionality. Requirements are numbered R1 to R9 (functional R1 to R6 and R9, non-functional R7 and R8) and refer to the proposal requirements FR-01 to FR-05 and its non-functional targets. Each carries a measurable acceptance criterion; statuses are those of the integrated `dev` branch at commit `2c266d4` on 12 October 2026.

4.1 User Interactions / Use Cases

Table 4.1: Use cases by actor. "I1" marks use cases whose engine part is built in Iteration 1.

ID	Actor	Interaction	Iteration
UC1	Student	Answers diagnostic questions on a unit; each answer updates the posterior and triggers the next question or a result	I1 (engine and student screen)
UC2	Teacher	Inspects the evidence trail of a run: questions, ranking, reasons, answers, posteriors	I1 (trace; terminal view)
UC3	Teacher	Resolves a refused (escalated) run using the tied candidate set	Target G5
UC4	Teacher	Approves, corrects or rejects an engine diagnosis before release	Target G5
UC5	Content reviewer	Reviews items (not the author), approves or rejects them	I1 (schema, data)
UC6	Content reviewer / admin	Freezes a content pack and its response model	I1 (schema)
UC7	Student, teacher	Creates or joins a class by code	Schema in I1; screens later
UC8	Parent	Views teacher-approved diagnosis summaries for their linked child	Gated I3
UC9	Teacher	Sends announcements and direct messages	Gated I3

4.2 Use Case Diagram

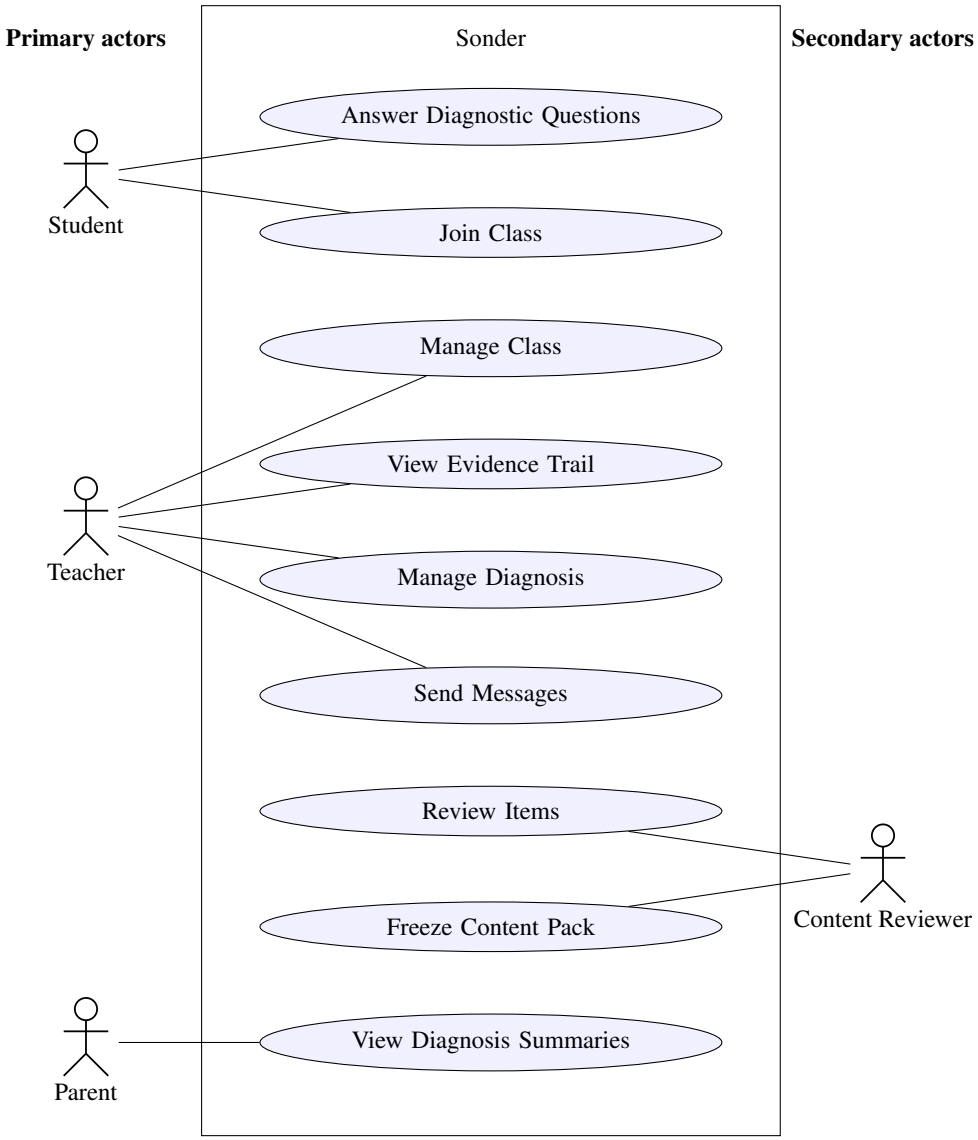


Figure 4.1: Use case diagram of the Sonder system. Student, Teacher and Parent are primary actors; Content Reviewer is a secondary actor. Parents view only teacher-approved diagnosis summaries for their linked child.

4.3 Detailed Use Cases

Table 4.2: UC1 — Answer diagnostic questions (engine path).

Actor	Student (through the interface); the engine is called by the service layer
Precondition	Current local path: a validated candidate Maths pack and parameters are loaded; an owner cookie identifies the run; a unit is selected. Target trusted path additionally requires externally accepted content, authentication and enrolment authorization
Main flow	1. A session starts with the prior over the unit's hypotheses. 2. The engine chooses the eligible item with the highest expected information gain and stores the ranking and reason. 3. The student picks an option. 4. The engine applies the Bayes update and evaluates the stopping rules R1–R5. 5. If the outcome is `in_progress`, return to 2. 6. Otherwise the session ends with its outcome and trace.
Alternative flows	Invalid option or zero-probability evidence: the request is rejected and the session state is unchanged. Budget (eight questions), no remaining items, or best EIG below 0.01 bits: `escalated_exhausted`. The baseline tolerance-based R4 heuristic triggers: `escalated_no_separator`.
Postcondition	The trace (questions, candidates, answers, posteriors, outcome) is complete and replayable from the option sequence alone

Table 4.3: UC5/UC6 — Review items and freeze a content pack.

Actor	Content reviewer (active teacher or admin, never the item's author)
Main flow	1. Items are drafted, then submitted for review. 2. A reviewer records a review for the next round, approving or rejecting; approved items lock. 3. When every item is approved and every unit satisfies the pack rules, the pack is frozen with a SHA-256 hash. 4. The response model (probabilities from slip and guess) is frozen against it.
Alternative flows	A freeze with a draft or in-review item, a missing source, a missing NONE/UNMODELLED hypothesis or a rule violation is rejected by the database.
Postcondition	A frozen pack and response model are immutable and are the only inputs a run may use

4.4 Functional Requirements

4.4.1 Module 1: Diagnostic engine

Table 4.4: Engine functional requirements.

ID	Requirement	Source	Status
R1	The engine selects the next item by maximising expected information gain over the live posterior; ties go to the earliest item in pack order; it returns the full ranking, entropy before and a generated reason	FR-01	Met (library)
R2	Insufficient evidence is a first-class outcome: a run ends as `diagnosed`, `none_detected`, `escalated_unmodelled`, `escalated_no_separator` or `escalated_exhausted`, with an evidence trail and, for refusals, the tied candidate set	FR-02	Implemented baseline; adverse policies and near-row R4 remain unresolved; no persisted or teacher-facing result
R3	One deterministic engine serves all declared Mathematics units with versioned hypotheses and item mappings	FR-03	Implemented for Maths units; external content acceptance remains open
R9	A student takes a Mathematics diagnostic in the user interface that is computed by the real engine, can resume after a refresh, and never sees the result (replies carry no diagnosis, posterior, rule or correct answer; a lost reply is retried without counting the answer twice)	Task T07.02	Met (local checkpoint)

4.4.2 Module 2: Content store

Table 4.5: Content functional requirements.

ID	Requirement	Source	Status
R6	Content packs are versioned, reviewed and frozen. An item is approved only by a recorded review from someone other than its author; a frozen pack is hashed and immutable; the trusted release must use frozen, externally accepted packs	Proposal; trust requirement	Partial: schema rules, pack loader and a CI import check built; production import into the hosted database not done

4.4.3 Deferred product requirements

Table 4.6: Product functional requirements outside Iteration 1.

ID	Requirement	Source	Status
R4	No diagnosis reaches a student or parent before a teacher approves, corrects or rejects it	FR-04	Deferred until G5; the student view already hides every result
R5	Class announcements, direct messaging and parent–teacher communication respect role and class access rules	FR-05	Conditional Iteration 3 after core acceptance; proposed owner Harris, pending agreement

4.5 Non-Functional Requirements

4.5.1 Reliability

R8: the calculation is deterministic and rejects invalid input. The same pack, parameters and answers give identical posteriors, selections, reasons and decisions; invalid options raise `invalid_option`, impossible evidence raises `zero_evidence`, and a rejected request leaves the session unchanged. A negative information gain beyond -10^{-12} is treated as a defect.

4.5.2 Usability

Teachers must read a decision without the probabilities being a black box: every selection carries a generated sentence stating the pair being separated and what each predicts. Student and parent views show an explanation, not raw posteriors. A usability study and the low-bandwidth target (a usable flow around 2 Mbps with session recovery) are deferred to Iterations 3 and 4.

4.5.3 Performance

R7: one posterior update plus question selection completes in under 300 ms. The new local reproduction and its raw timings are described in Section 7.4.1; they exclude database, network and browser time and do not establish the deployed target.

4.5.4 Security

The engine has no I/O, network, clock or random source (checked by tests). Pseudonymous identifiers, role-scoped access and an append-only audit are required of the product; the schema defines roles, hashed credentials, append-only run records and an audit table. The local engine service requires a shared token, owns each run through a signed HttpOnly cookie, checks the request origin and listens on the loopback address only; there is no real login and no deployed service yet, and an RBAC audit is planned for Iteration 4. Repository policy forbids secrets; a complete secret scan is not claimed.

4.6 Acceptance Criteria

Table 4.7: Acceptance criteria and verification.

ID	Acceptance criterion	Verification
R1	EIG agrees with exact arithmetic to 10^{-11}; choice and ranking agree with a brute-force reference on 300 random posteriors; the specification worked examples reproduce to four decimals (D1: T1-005 at 0.7887, then T1-006, T1-007, T1-009; D2: T4-001 0.7517, T5-011 0.3668); an empty candidate list returns `None`; asked items and their twins are excluded	`tests/test_selection.py` (40 tests)
R2	Each of the five stopping outcomes is produced by a constructed case; refusals name the tied set; a student with no misconception ends `none_detected`; MATH-1514/1773 end `escalated_no_separator` naming both	`tests/test_engine_stopping.py` (22), `test_engine_students.py`
R3	Declared Maths units load without unit-specific inference branches; exclusions and content versions are explicit	Source inspection, Maths pack tests and pure-engine boundary tests
R9	Browser answers produce the engine's next item; a refresh resumes the same question; replies carry no diagnosis, posterior, rule or correct answer; a repeated request is not counted twice	`tests/test_engine_api.py` (16), 123 interface tests, `scripts/check_engine_ui.py` (D1, D2)
R6	Invalid items and units are rejected by rules V1–V11; Maths_v2 loads into seven units with every linked misconception inside its unit (V5′); schema rejects invalid edits	`tests/test_engine_pack.py` (40); `behaviour_test.sql` (75 rejected writes and 15 of 15 checks, run in CI)
R7	Median and 95th percentile of one step $<$ 300 ms	Raw timing CSV and reproduction tool, Section 7.4.1
R8	Repeat runs identical; invalid input rejected with a named error	`tests/test_engine_session.py` (12), `test_bayes.py` (19), `test_engine_trace.py` (5)
R4	Only an authorized teacher decision releases a result; pending or rejected results remain hidden	Future authenticated integration test after persistence/teacher-view gates
R5	Authorized class announcements and direct threads succeed; unauthorized class/guardian access fails	Conditional Iteration 3 test; proposed owner Harris, pending team agreement

4.7 Requirements Traceability

Table 4.8: Requirement → design → implementation → test → owner.

ID	Design	Implementation	Test	Owner
R1	Spec §3.4; Section 6.6.3	`selection.py`	`test_selection.py`	Harris
R2	Spec §3.5; Figure 5.14; tables `diagnosis`, `escalation`	`stopping.py`, `session.py`, `trace.py`	`test_engine_stopping.py`, `test_engine_students.py`	Ahmed
R3	Spec §2.1	whole `sonder_engine` package (no subject branches)	`test_engine_no_ai.py`	Ashher
R4	Teacher review tables, planned	approval not implemented; the student view hides every result (`sonder_api/student_view.py`)	`test_engine_api.py`	Ashher: proposed teacher view; approval owner to agree
R5	Class-scoped communication and role policy; conditional Iteration 3 milestone	Fixtures only; implementation deferred until core acceptance	Planned send/read and cross-class/guardian denial tests	Harris (proposed; confirm)
R6	Schema plan §4; Section 5.8	`pack.py`, `sonder_pack/`, schema SQL	`test_engine_pack.py`, `behaviour_test.sql`	Ahmed, Harris (data)
R7	Proposal NFR	`session.py`	raw-backed timing reproduction; end-to-end test pending	Ashher (proposed evaluation)
R8	Spec §2.3	`bayes.py`, `session.py`, `selection.py`	`test_bayes.py`, `test_selection.py`, `test_engine_session.py`	Harris, Ashher
R9	`sonder_api/README.md`; architecture diagram	`sonder_api/`; interface routes and screens	`test_engine_api.py`; interface tests; `check_engine_ui.py`	Ahmed

Chapter 5

System Design

5.1 System Architecture

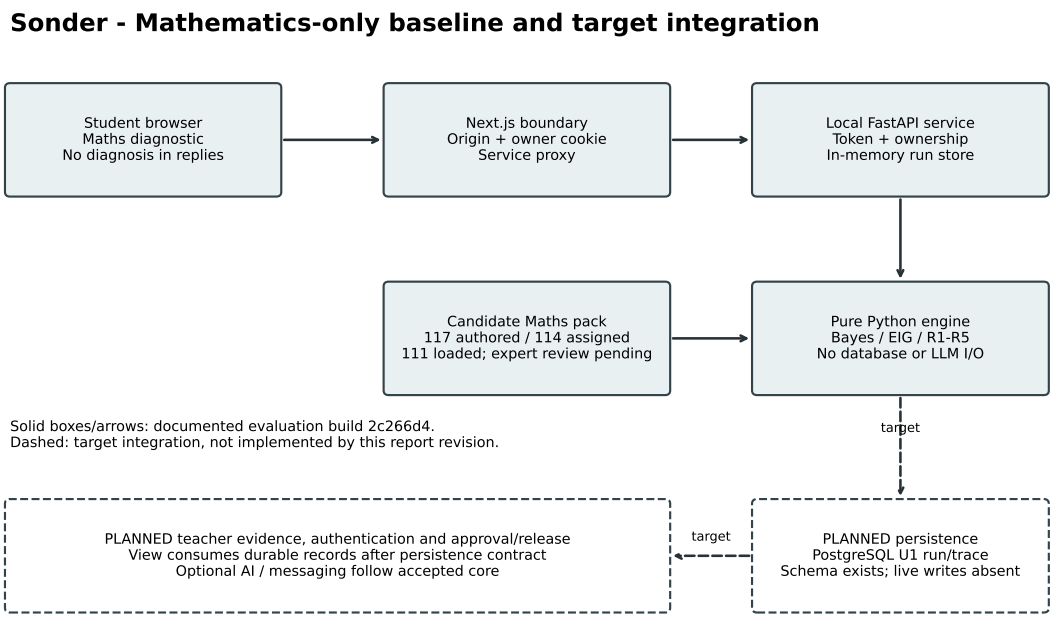


Figure 5.1: Maths-only architecture: solid elements describe baseline 2c266d4; dashed elements are proposed persistence and teacher integration. The current service holds runs in memory and is lost on restart. The candidate content awaits external expert acceptance.

The system has four parts joined by one-directional interfaces. The browser talks to a Next.js server, which will authenticate and scope access (not built yet; there is no real login) and calls an engine service over HTTP/JSON. The engine service runs a pure Python library, `sonder_engine`, in process; the library performs the calculation and has no I/O, no network, no model call and no database access. The target PostgreSQL 16 database

is designed to hold content, run records and audit; the local baseline does not write its live runs to it. Offline tools (the pack validator, CI and tests) are not on the request path. The AI consultant and other model-backed services of later iterations are read-only with respect to the engine: they can explain a result but cannot write a posterior.

5.2 Architectural Components

Table 5.1: Components and their Iteration 1 state.

Component	Responsibility	State
`sonder_engine` (library)	Prior, likelihood, posterior (`bayes.py`, `model.py`); eligibility and EIG selection (`selection.py`); stopping and refusal (`stopping.py`); run loop (`session.py`); evidence trail (`trace.py`); unit loader and rules (`pack.py`)	Built, 186 Python tests in total
`sonder_pack`	Reads Maths_v2 files, builds units U1–U7, prints warnings, replays a session on the command line	Built
Validator (`validation/`)	Checks the CSV bank and its source batches	Built, in CI
Database schema	41 tables, 110 triggers, 1 view in seven domains: users, classes, content pack, diagnostic units, engine model, runs, audit	Written; 15 of 15 checks run in CI on PostgreSQL 16; live runs not yet saved to it
Engine service `sonder_api` (FastAPI)	Start, answer and trace endpoints; pack loader; signed run cookie, shared token and origin check; runs held in memory	Built (local); storage in the database to do
Next.js server and interface	Same-origin routes to the engine service; student diagnostic screens; role-based screens, authentication, class flows	Student Mathematics diagnostic live; other screens scripted
CI (GitHub Actions)	Five jobs: interface build and tests; dataset validator; engine tests; database schema checks in a throwaway PostgreSQL container; engine API tests	Built

5.3 Interfaces and Dependencies

Content to engine. The baseline reads a versioned candidate pack through its loader; the trusted target requires an externally accepted, frozen pack: units with ordered hypotheses and items, options with graded misconception links, and a versioned parameter set. **Engine to storage.** The target persistent service will write each presented question with its full candidate ranking, each answer with its posterior, and the outcome in one transaction; database triggers check the writes but never calculate. **Engine public API** (`sonder_engine`): `prior`, `likelihood_matrix`, `update_posterior`, `eligible_items`, `expected_information_gain`, `select_next`, `check_stop`, `start`, `next_question`, `answer`, `replay`, `trace_of`. A session is an immutable value; every function returns a new state, and a rejected request leaves the state unchanged. **Live student path (built).** The browser calls same-origin Next.js routes, which call the local engine service; the service runs the unchanged engine, owns each run through a signed HttpOnly cookie, checks the request origin and a shared token, and returns only the next question or a neutral completion message. A retried answer is not counted twice. **Dependencies.** NumPy ≥ 2.0 (the only third-party package of the engine), Python ≥ 3.11, FastAPI, uvicorn and httpx for the optional service, PostgreSQL 16+ (hosted on Neon), Docker for the schema checks, Node for the interface.

5.4 Architecture Decisions and Rationale

Table 5.2: Architecture decisions.

Decision	Alternatives	Chosen	Trade-off
Diagnosis method	Fixed quiz; rule-based; one-shot LLM; retrieve-and-rerank	Bayesian posterior with EIG selection	Deterministic, auditable, needs no training; depends on assumed slip and guess until calibrated
Scope of one session	Per topic; whole subject; per diagnostic unit	Diagnostic unit	Crossing items stay usable; units are defined by hand and limited to eight hypotheses
Engine structure	Logic in SQL; logic in the web app; pure library	Pure Python library, no I/O	Easy to test and reproduce; needs an integration layer
Where integrity rules live	Application code; database constraints and triggers	Database enforces freeze, review, append-only and trace rules	Strong guarantees; a heavy schema that is hard to change (slimming is an open team decision)
Handling ambiguity	Always return the top hypothesis; threshold only; explicit refusal	Explicit refusal outcomes with a separability rule	Lower coverage in exchange for no false-confidence diagnosis
LLM role	In the diagnostic path; outside it	Outside: drafting content only, with human review	Slower content growth; the diagnosis stays reproducible

5.5 Design Evolution

Table 5.3: Design changes since the proposal.

Area	Change	Why
Session scope	Topic → diagnostic unit (hypothesis list plus item list)	26 of 117 items cross topics; a topic-scoped session cannot use them
Hypothesis limit	$K_{\max}$ 6 → 8	One unit needs seven misconceptions
Twin items	Added twin groups: asking one makes its twins ineligible	Repeating a stem gives no independent evidence
Response model	Rule P1 generalised: guess $< (4 - s)/4$	A three-option predicted set made a misconception holder prefer the correct answer
Link strength	Added link grades S/T with weights 1.0 and 0.5	About 90 links are plausible but weak
Schema	Planning model → 41 tables (three revisions): diagnostic units, candidate ranking, engine parameters, response model	Spec v2 findings; stored ranking for the audit trail
Engine shape	Added stopping rules R1–R5, run loop, evidence trail and loader	Needed for a complete session and the refusal requirement
Product access to the engine	Added a local engine service behind same-origin interface routes; the student sees only questions and a neutral completion message	One engine implementation, no secrets in the browser, and every result stays behind the teacher gate (R4)

5.6 Domain Model

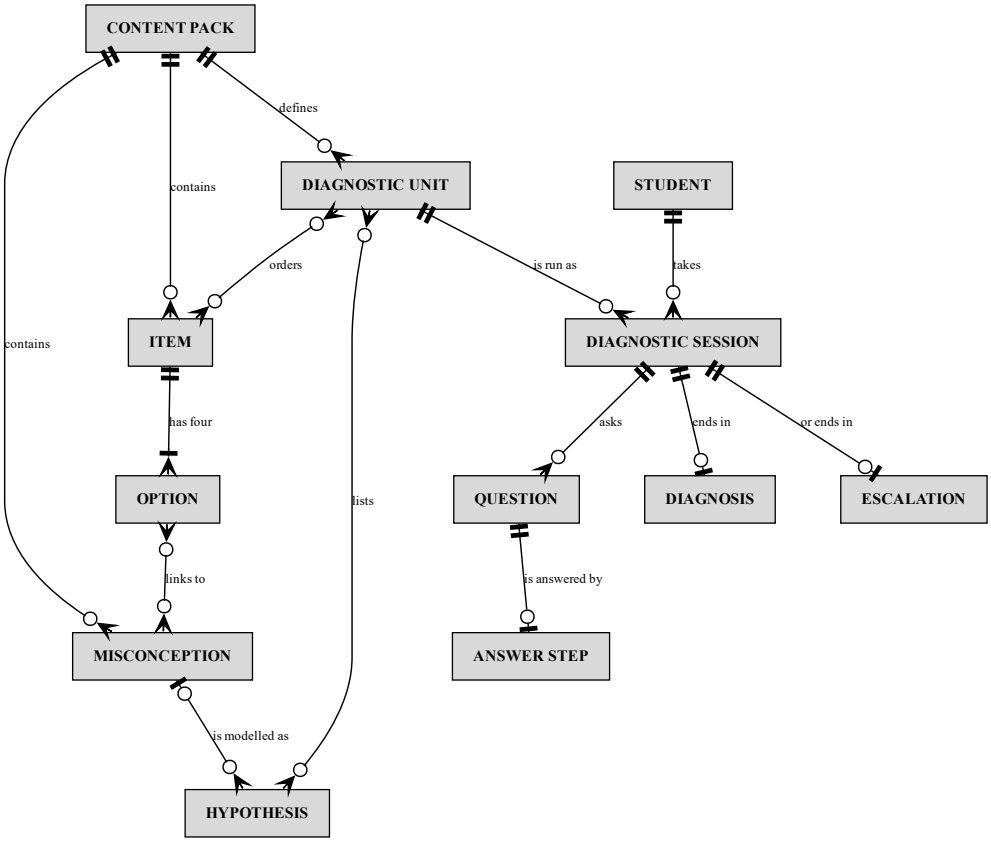


Figure 5.2: Domain model of the diagnostic core, in crow's foot notation. At each end of a line: a bar means one, a circle means zero, a crow's foot means many. Each name reads from the entity above to the entity below. A session ends in at most one of diagnosis or escalation (refusal).

5.7 Data Design

The Iteration 1 schema (revision 3) has 41 tables, 245 columns, 66 foreign keys and 110 triggers. The design rule is that the database *checks* what the engine writes and never calculates. The area ERDs below use crow's foot notation, and were generated from the DBML so they match the SQL. An entity is a rectangle; key attributes are underlined; foreign keys are not listed as attributes because each one is a named relationship line. Entities shown only as a name (for example `item` in Figure 5.5) belong to another area and are defined in the diagram for that area. The user-supplied Chen-notation overview

and its clearer online link are in Appendix E. It is a diagnostic-core conceptual view, not a mechanically complete physical-schema export.

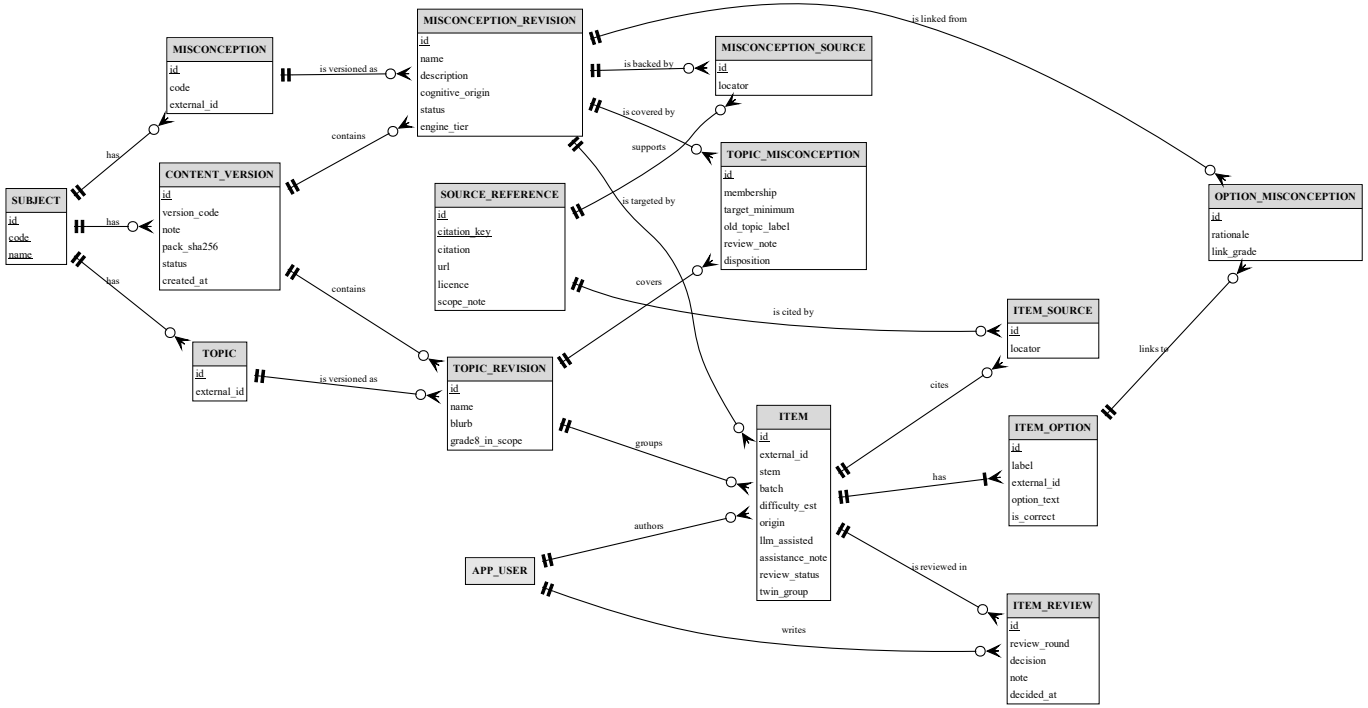


Figure 5.3: ERD: Mathematics content pack (subject, topics, misconceptions, items, options, Q-matrix links, sources, reviews).

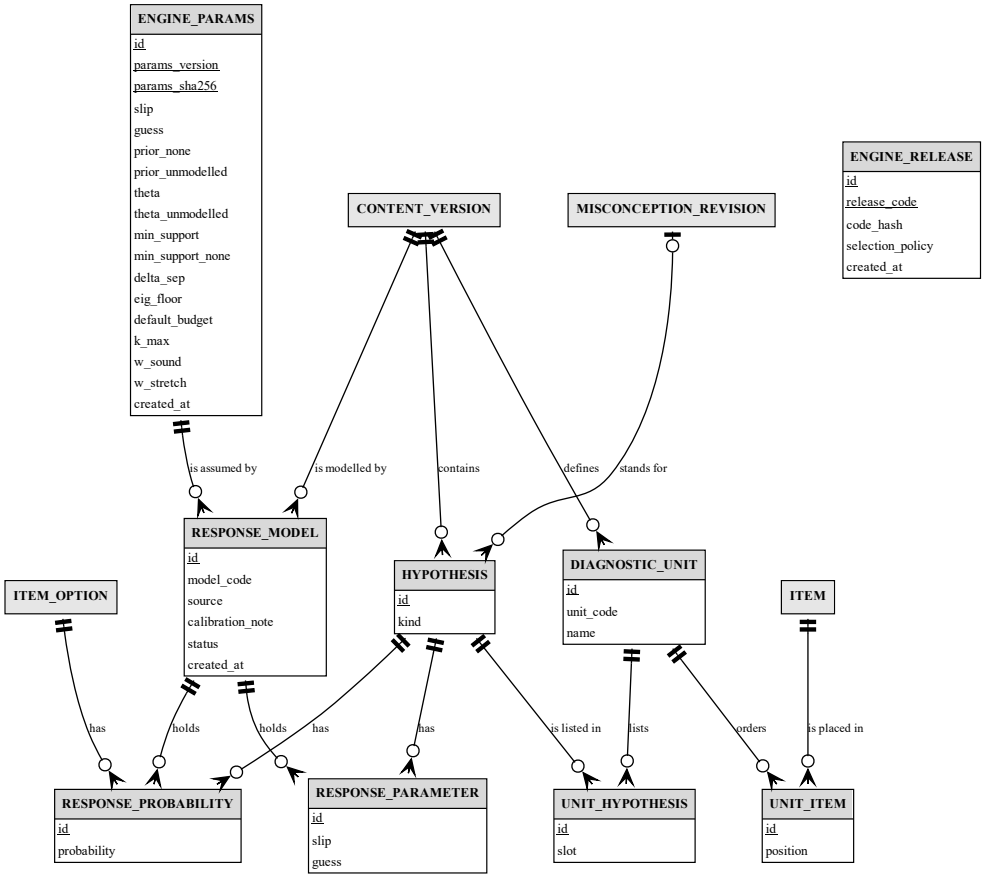


Figure 5.4: ERD: diagnostic units (ordered hypothesis slots and item positions) and the engine model with its frozen inputs (parameters, response model, probabilities, engine release).

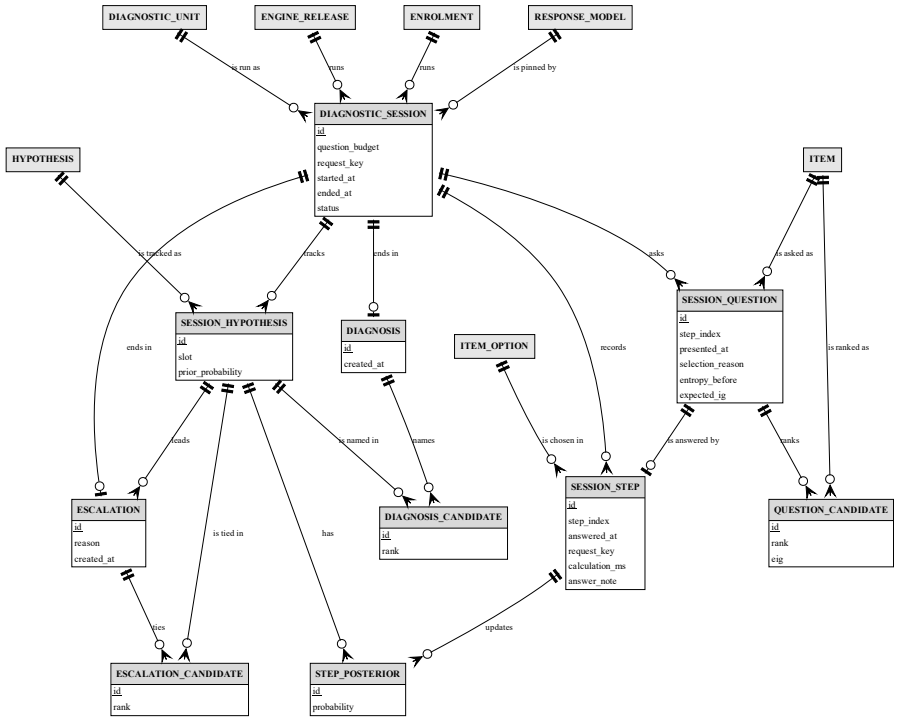


Figure 5.5: ERD: diagnostic runs (session, hypotheses, questions, candidate ranking, steps, posteriors, diagnosis, escalation).

Key integrity rules (Table 5.4) include: exactly four options and one correct option per item; the correct option has no link; approval only by a recorded review from a non-author; pack and response-model freezing with hashes and completeness checks; append-only run records; unique request keys that make retried answers harmless; and a commit-time check that every finished run has a complete, normalised trace.

Table 5.4: Selected integrity rules enforced by the database.

Area	Rule
Items	Four options A–D, one correct (checked at commit); only draft items editable; a wrong option has at most three links
Review	Approve or reject only by inserting an `item_review` by an active teacher or admin who is not the author, for the next round; reviews are immutable
Pack freeze	Hash present; no draft or in-review item; source on each curated item; NONE and UNMODELLED exist; every unit complete; rule V5′; afterwards immutable
Response model freeze	Pack frozen; every approved item $\times$ hypothesis has four probabilities summing to 1; unit size $\leq k_{\max}$; rule P1 holds
Runs	Needs a frozen model, an enrolled student, one unit; priors written first; questions in order within the budget, never repeating an item or showing a twin; each presentation stores its ranking (ties in pack order)
Outcomes	A `diagnosed` run has a rank-1 misconception; an `escalated_*` run has a leader; a no-separator refusal has a tied set of at least two

5.8 Process / Behavior Design

Ashher reviewed the implemented data flow against the current engine and schema and corrected the document, and added the implemented diagnostic state-machine diagram (PR #39); this review covers alignment with the engine and schema and is not a sign-off of the Mathematics content.

5.8.1 Interaction design: scope and notation

The prescribed mid-report template places sequence diagrams among the appropriate process and behavior artefacts in Section 5.8; it does not prescribe a number of diagrams. The following system sequence diagram (SSD) and internal sequence diagrams (SDs) refine UC1 and requirements R1, R2, R8 and R9. The content-review and pack-freeze interactions (UC5/UC6, R6), the refusal alternative and the session-start interaction are in Appendix E.

The notation follows the supplied course slides, *SDA Lect06: System Sequence Diagrams*, slides 14–22, 26–31 and 38–41, and *SDA Lect09: Sequence Diagrams*, slides 4–8, 12–22 and 26–34. An SSD treats Sonder as one black box and names external events by domain intent. An SD exposes the collaborating boundaries, store and pure modules. Solid arrows denote calls or system events; dashed arrows denote returned information; activation bars show execution; guarded `loop`, `alt` and `opt` frames show repetition and alternatives. A `ref` frame refers to the detailed computation diagram. Long SDs continue across labelled panels with repeated participant headers.

The diagnostic interactions describe the report's evaluation build `2c266d4`, checked against its locally available Git objects, rather than the older code on the diagram branch. They contain no database write, real login, teacher result screen or LLM call. Proposed content controllers are labelled explicitly in the supplementary diagrams; their underlying SQL constraints exist, but the reviewer and administration interfaces are not claimed as live features. Conceptual SSD operations are refined by the SDs and are not literal endpoint names.

5.8.2 System sequence: take a diagnostic

Figure 5.6 models the normal external interaction of UC1. The student starts a unit diagnostic and submits one answer at a time. The system returns another question or the neutral completion message. The same public message is used for a refusal, as illustrated by SSD-02 in Appendix E; a diagnosis and its confidence never cross this boundary before teacher review.

5.8.3 Internal sequence: answer, retry and publication

Figure 5.7 refines the answer event. The Next.js boundary validates the origin, signed owner cookie and request body; the FastAPI boundary checks the service credentials before the memory store processes the request. Under its lock, the store checks ownership and cached replies. An identical retry returns the original snapshot; a conflicting retry or stale step is rejected. A new answer is calculated and projected before the state, trace and cached reply are published. Thus an invalid option or impossible evidence does not partially update the session. The diagram abstracts the web request boundary; it does not claim new classes in the pure engine.

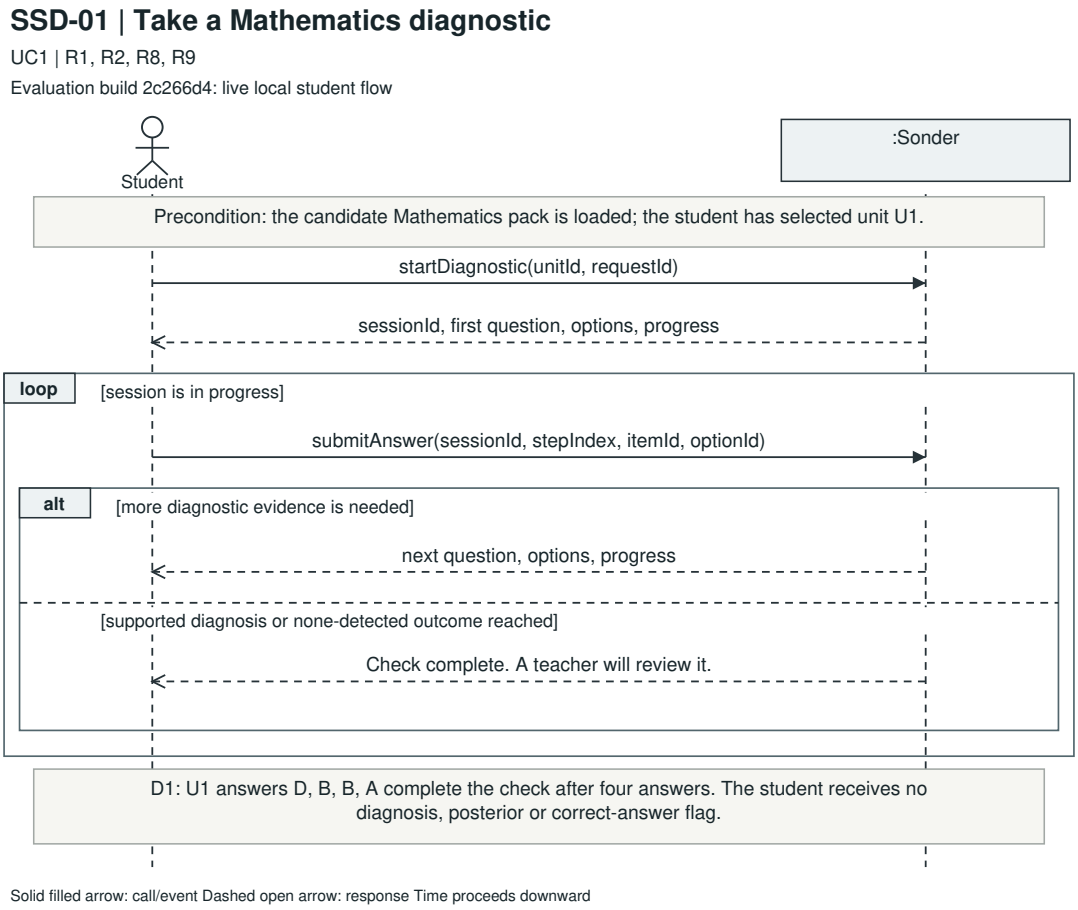


Figure 5.6: SSD-01: UC1, take a Mathematics diagnostic (R1, R2, R8, R9). Evaluation-build local student flow; D1 completes after four answers.

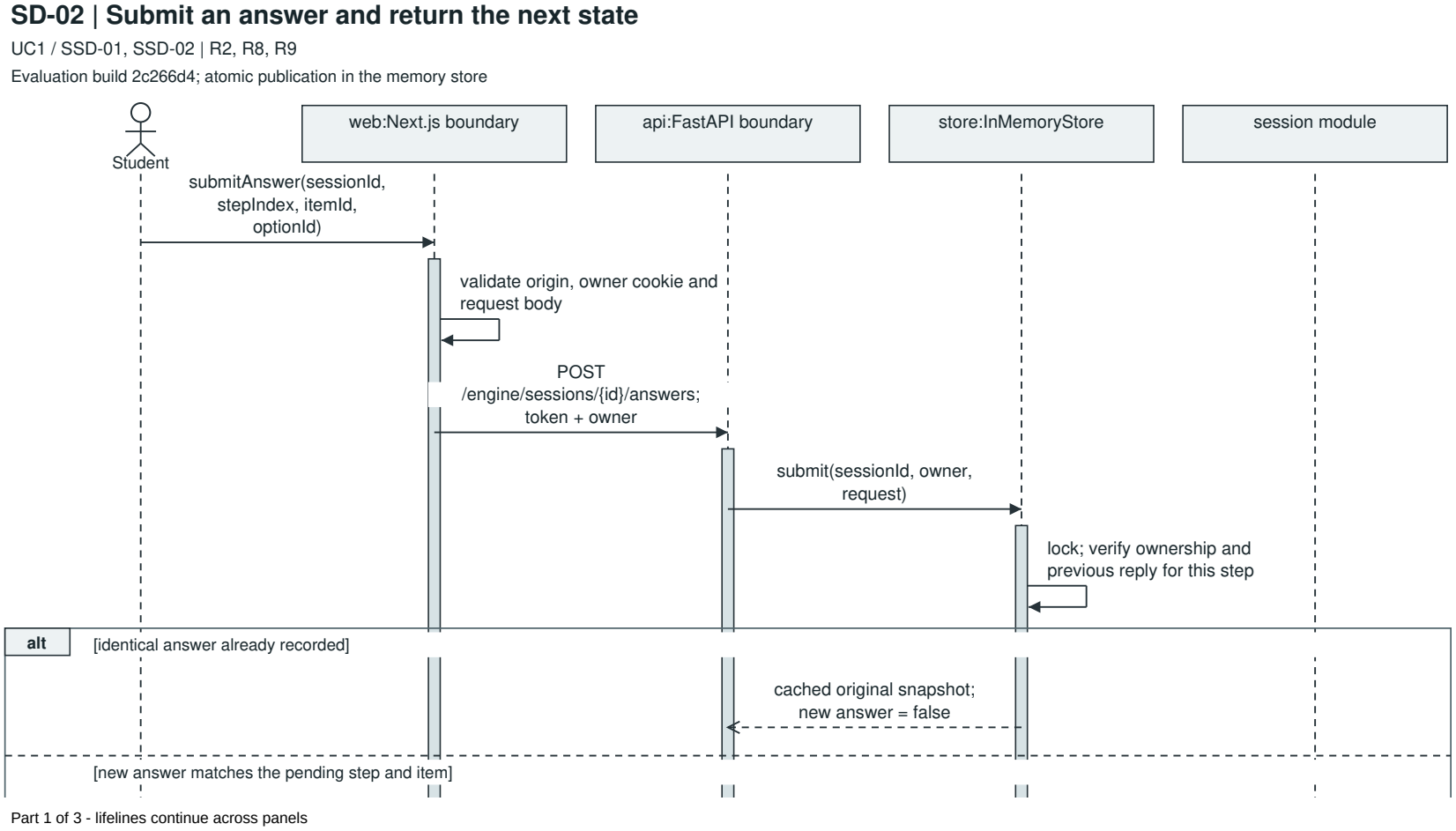


Figure 5.7: SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 1 of 3).

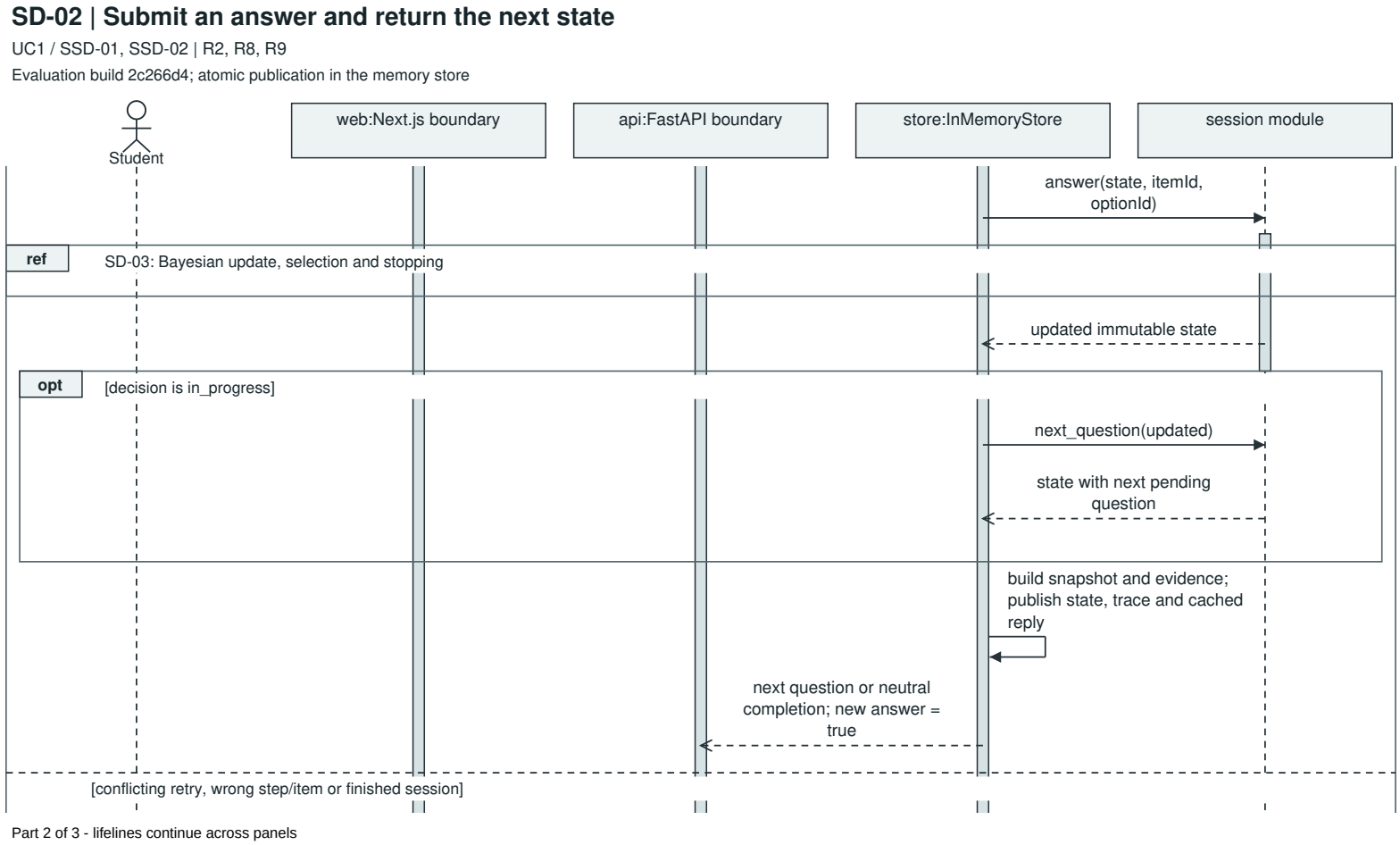


Figure 5.8: SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 2 of 3).

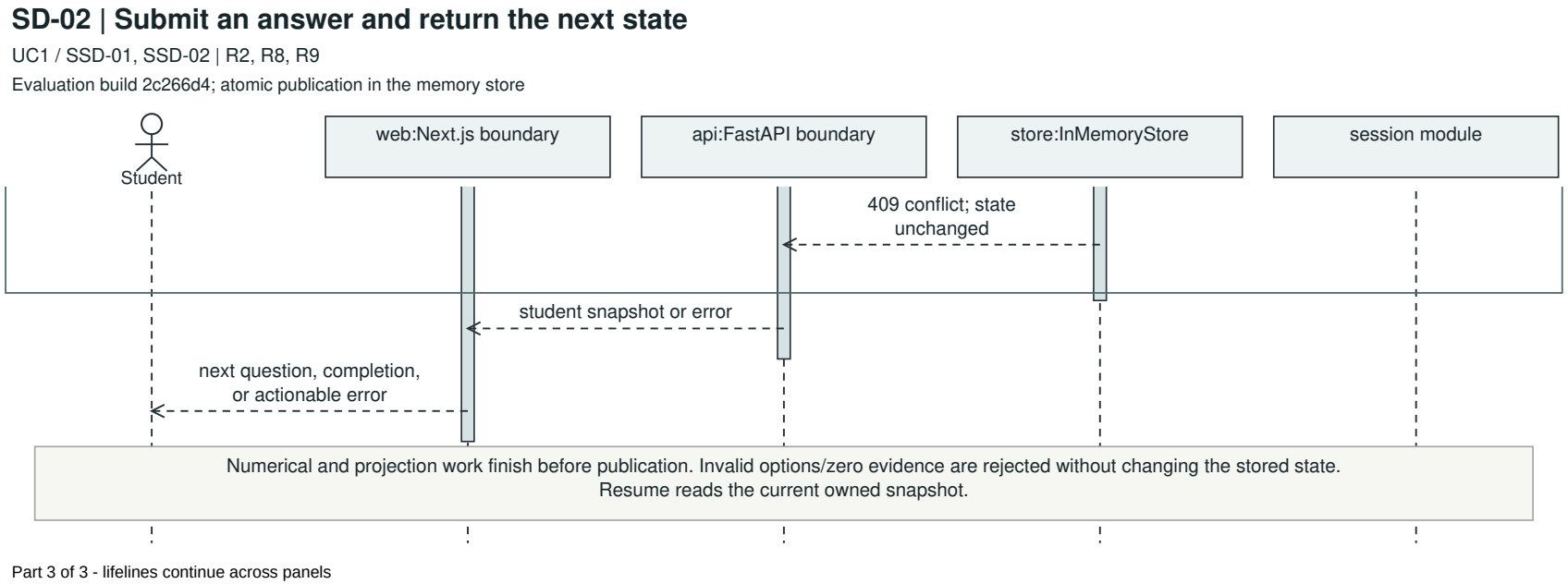


Figure 5.9: SD-02: UC1 answer processing (R2, R8, R9), including cached retries, conflicting requests and atomic in-memory publication. (part 3 of 3).

5.8.4 Internal sequence: diagnostic calculation and refusal

Figure 5.10 expands the computation referenced by SD-02. The session module calls the Bayesian functions, excludes asked items and twins, calculates the best remaining information gain when the budget permits, then calls the stopping module. Rules R1–R5 are evaluated in order. A new immutable state records the answer, posterior, support and decision; if still in progress, the caller asks `next_question` to record the next presentation. The pure modules are shown as modules, not invented object classes.

In worked example D2, the second answer leaves MATH-1514 and MATH-1773 at about 0.4813 each. R4 returns `escalated_no_separator` with both candidates because the remaining eligible items cannot distinguish them and the exact-pair conditional share is below the diagnosis threshold (the near-row limitation is stated below). The full decision remains in the evidence trace; the student sees only completion.

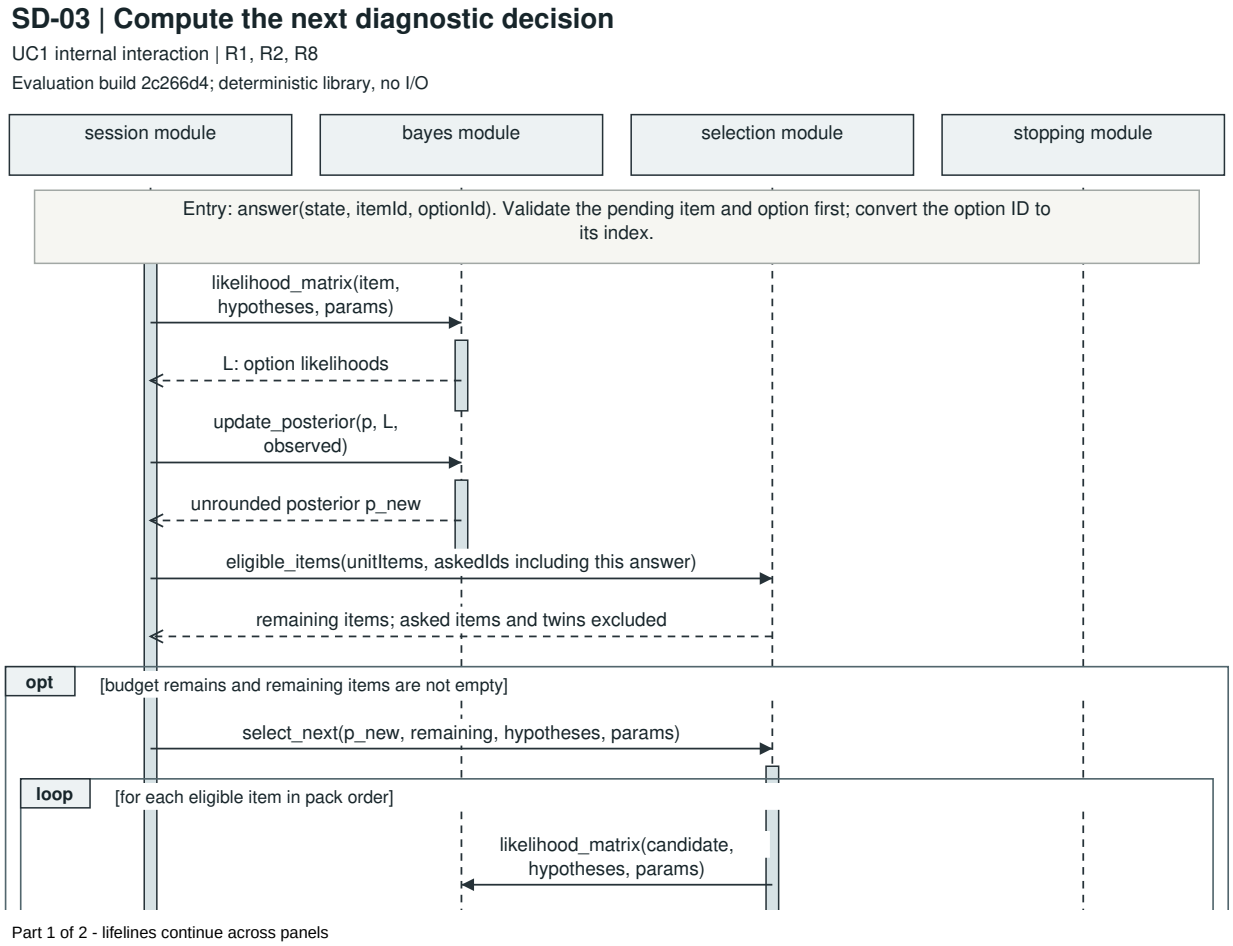


Figure 5.10: SD-03: UC1 computation (R1, R2, R8): Bayesian update, information-gain selection and ordered stopping. The R4 alternative explains SSD-02. (part 1 of 2).

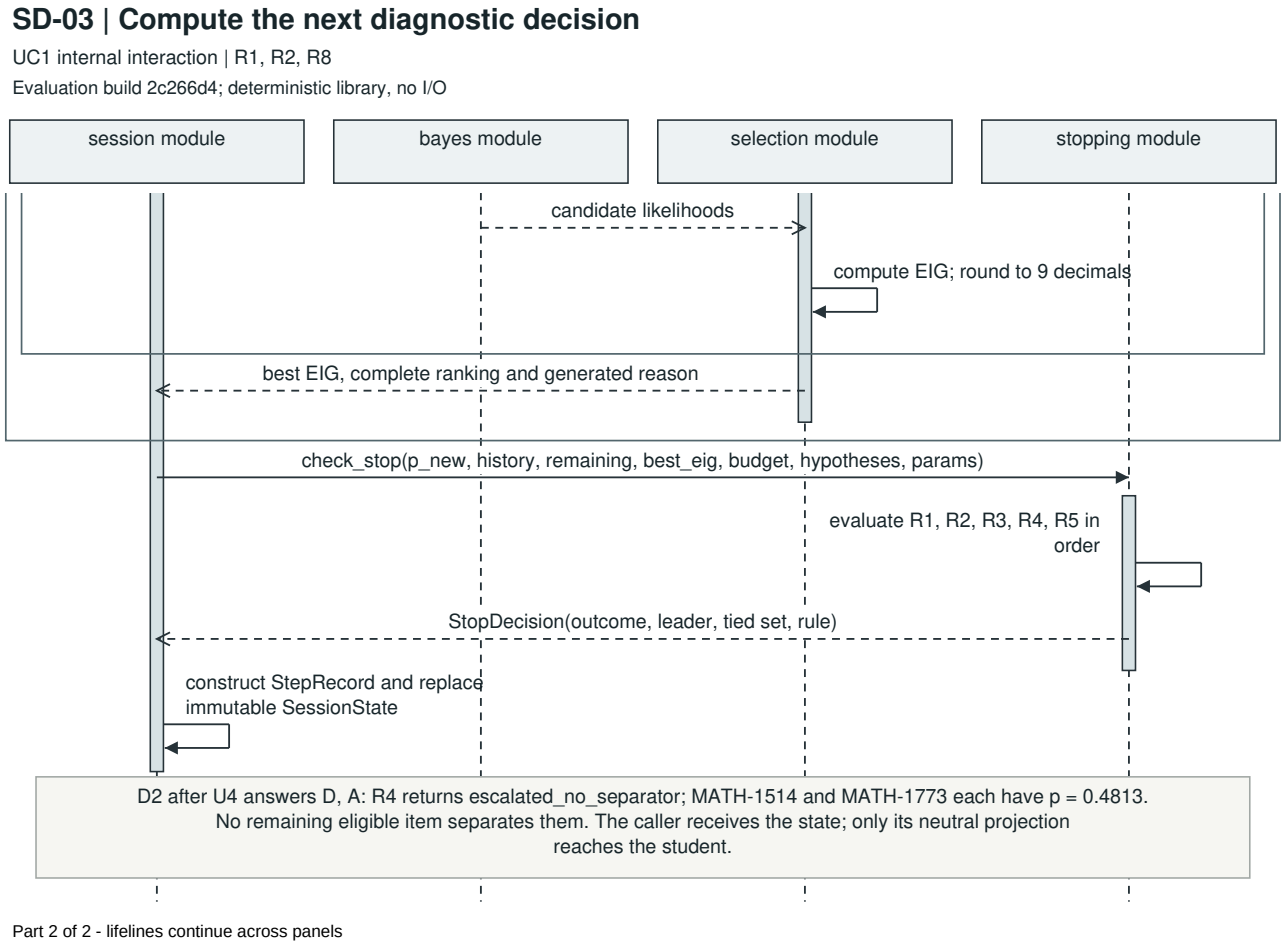


Figure 5.11: SD-03: UC1 computation (R1, R2, R8): Bayesian update, information-gain selection and ordered stopping. The R4 alternative explains SSD-02. (part 2 of 2).

Table 5.5: Interaction diagrams and their requirements evidence.

Diagrams	Use case / requirements	Implementation or design boundary
SSD-01/02; SD-01	UC1; R1, R2, R8, R9	Evaluation-build student boundary, service and memory store
SD-02/03	UC1; R1, R2, R8, R9	`store.py`, `session.py`, `bayes.py`, `selection.py`, `stopping.py`; session/API tests cited in Chapter 7
SSD-03; SD-04	UC5; R6	`item_review_guard`, `item_review_apply`, audit; proposed reviewer controller
SSD-04; SD-05	UC6; R6	`state_guard`, `freeze_guard`, audit; proposed pack controller

Editable sources, the lecture/template mapping and reproducible figure generator are in `HarrisDOcs/design/sequences/`. These additions were prepared with Codex assistance and are subject to the team's design review; they do not change the evaluation build or claim a new independent review.

5.8.5 Target content acceptance and freezing workflow

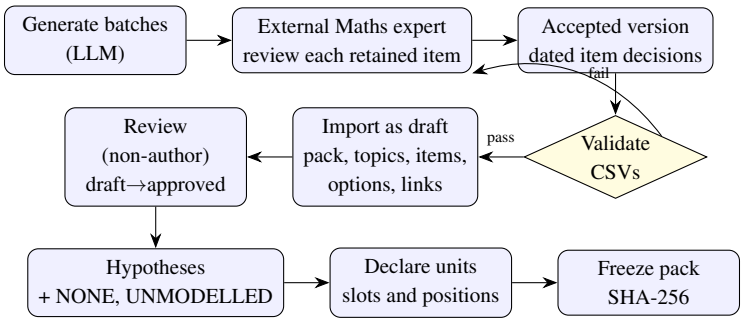


Figure 5.12: Target content workflow, not a completed acceptance claim: the expert-review gate is outstanding. NONE, UNMODELLED and units precede freezing; student-owner approval and agent audits alone do not pass this gate.

5.8.6 Data flow: one diagnostic step

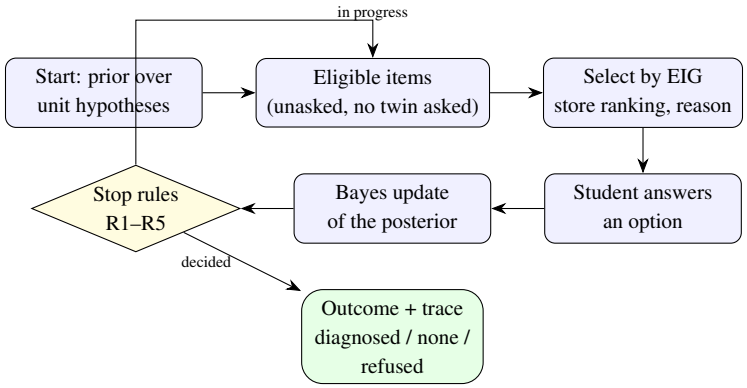


Figure 5.13: Evaluation-baseline computational flow; ranking and trace are in memory. R4 is tolerance-based and has the exact/near-row limitation described here.

Exact versus near-row limitation (supervisor action 3). The baseline R4 uses total-variation tolerance $\delta = 0.05$. If two hypotheses have *exactly equal* likelihood rows for every possible remaining observation, their posterior odds cannot change and $p(\ell)/p(C)$ is an exact conditional share bound. Near-equal rows do not have this property: odds are multiplied by their likelihood ratio at each answer, and small differences can accumulate. Therefore the baseline ratio is a heuristic when $0 < \mathrm{TV} \leq \delta$, not an absolute confidence ceiling. The exact-pair D2 example remains reproducible, but general R4 correctness requires exact-row and near-row regression cases, revised behavior or a justified finite-budget bound, and matching specification/tests. No engine correction is claimed by this wording change.

5.8.7 State diagram: session outcomes

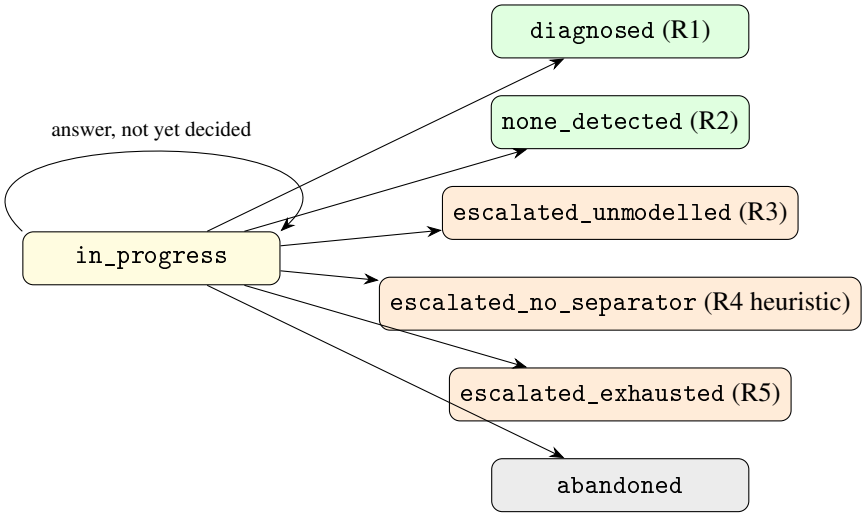


Figure 5.14: Baseline state machine; near-row R4 correctness remains open (target database column `diagnostic_session.status`). All outcomes are terminal; rules are evaluated in order R1 to R5 after each update.

5.8.8 State diagram: content lifecycle

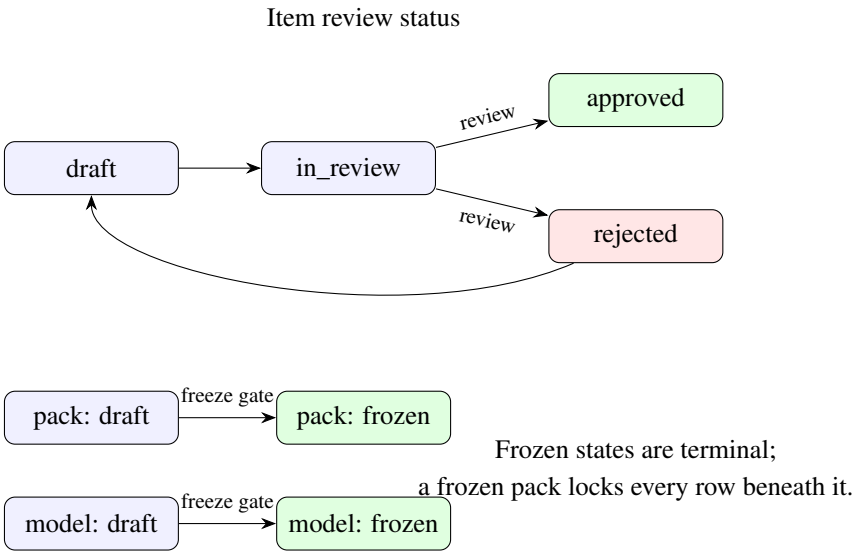


Figure 5.15: Content state machines enforced by the database (`state_guard`). Approval and rejection happen only by a review by a non-author.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Build a real, tested diagnostic core for Mathematics: take an answer from the candidate pack, update competing misconception probabilities by a stated rule, choose the next item by computed information gain, handle tie, no-separator and budget states explicitly, and leave an evidence trail. The evaluation version is the integrated `dev` branch at commit `2c266d47472e130eea2fb68ae1c19972d0994cee` (12 October 2026), the code freeze used for the live demonstration; no release tag has been created. The objective also includes a live student Mathematics diagnostic in the interface that calls the same engine.

6.2 Selected Use Cases / Features

UC1 (engine path and the student screen), UC2 (evidence trail, as a data structure and a terminal view), UC5 and UC6 (rules in the schema and loader), and requirements R1, R2, R3, R6, R7, R8 and R9 of Chapter 4. Teacher approval, a teacher result screen and messaging are not part of this iteration's built scope.

6.3 Design

The design follows the engine specification v2 (`docs/ENGINE_SPEC_DRAFT.md`), which was measured on the real bank, and the schema plan (Chapter 5). Three principles shape it: the engine is a pure function of its inputs (so every result is reproducible); the database checks writes but never calculates; and a refusal is a designed outcome with the same standing as a diagnosis.

6.4 Implementation

Table 6.1: Implemented modules (all under `sonder_engine/` unless stated).

File	Content	Author
`model.py`	`Item`, `Params` (all assumed parameters, validated), hypothesis-list checks	Ashher
`bayes.py`	`prior`, `likelihood_matrix` (with rule P1 and link grades), `update_posterior`	Ashher
`selection.py`	`eligible_items`, `entropy_bits`, `expected_information_gain`, `select_next`	Harris (v2 alignment: Ashher)
`pack.py`	Builds units from rows; rules V1–V11; warnings W1–W6; exclusions	Ahmed
`stopping.py`	Rules R1–R5, support, separability, `check_stop`	Ahmed
`session.py`	`start`, `next_question`, `answer`, `replay`; immutable session state	Ahmed
`trace.py`	Evidence trail with field names of the schema; JSON round trip	Ahmed
`sonder_pack/`	The only code that reads files; unit table (JSON); `run_session` demonstration command	Ahmed
`sonder_api/`	Local FastAPI engine service: start, answer and trace routes; student view that hides every result; signed run cookie, shared token, origin check; in-memory run store	Ahmed
UI routes (`app/api`, `lib/diagnostic`, `app/student`)	Same-origin routes and the student Mathematics diagnostic screens (topic list, questions, completion, resume, error handling)	Ahmed

6.5 Completed Features / Module

- Prior over a unit's hypotheses, Bayesian update, graded links (sound 1.0, stretch 0.5).
- Expected-information-gain selection with deterministic ties, full ranking and a generated reason.
- Five stopping and refusal rules and six outcomes, including the tied set of a refusal.
- A run loop that is an immutable value, replayable from the option sequence.

- An evidence trail whose fields follow the database tables.
- A loader that builds seven diagnostic units from the Maths_v2 files, applies hard rules and reports warnings and every exclusion.
- A local engine service and a live student diagnostic in the interface: the student answers real questions, can resume after a refresh, and sees only a neutral message at the end ("Your check is finished. A teacher will review it.").
- The candidate Maths_v2 bank and its validator; the 41-table schema with automated checks in CI.

6.6 Technical Implementation Details

6.6.1 Setup and execution (to reproduce)

```
git clone https://github.com/sonder-fyp/SONDER.git
cd SONDER && git checkout dev          # or the evaluation tag
python -m pip install -e .  # Python 3.11+, NumPy 2.x
python -m unittest discover -s tests    # 186 tests
python -m validation.math_dataset       # bank structure check
python -m sonder_pack.maths_v2          # units, hashes, warnings
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

The live demonstration runs the same commit as two loopback processes: the engine service (`python -m sonder_api --host 127.0.0.1 --port 8000 --trace`, which needs the optional API dependencies) and the interface (`npm run dev` in `Sonder-UI-master`), opened at `http://127.0.0.1:3000`; the setup is in `sonder_api/README.md`.

6.6.2 Calculation

Let $\mathcal{H} = (m_1, \ldots, m_k, \text{NONE}, \text{UNMODELLED})$ with $k \leq 8$. For an item i with four options and a hypothesis h, let $P_h(i)$ be the set of options h predicts: the options linked to h if any; otherwise the correct option; for NONE the correct option; for UNMODELLED all four. With $e = \text{slip}$ if $P_h(i)$ is the single correct option and $e = \text{guess}$ otherwise,

$$P(o \mid h, i) = \begin{cases} (1-e)/|P_h(i)| & o \in P_h(i) \\ e/(4-|P_h(i)|) & \text{otherwise} \end{cases}, \qquad P(o \mid \text{UNMODELLED}, i) = \tfrac{1}{4}.$$

Rows sum to one and every entry is positive. A stretch link with weight w gives the row $w \cdot \text{row}_{\text{linked}} + (1-w) \cdot \text{row}_{\text{NONE}}$. The prior is $p_0(\text{NONE}) = 0.25$, $p_0(\text{UNMODELLED}) = 0.05$ and the remainder is split equally. After observing option o on item i,

$$p'(h) = \frac{p(h)\,P(o \mid h,i)}{\sum_g p(g)\,P(o \mid g,i)},$$

and impossible evidence raises `zero_evidence`.

6.6.3 Selection

The information gain of an eligible item i, in bits, is the mutual information between the hypothesis and the answer:

$$\text{EIG}(i) = \sum_h \sum_o p(h)\,P(o \mid h,i) \log_2 \frac{P(o \mid h,i)}{P(o)}, \qquad P(o) = \sum_h p(h) P(o \mid h,i).$$

Eligible items are the unit's items minus those already asked and minus the twins of asked items. Each EIG is rounded to nine decimals; the highest wins, and an exact tie goes to the earliest item in pack order. Many real items tie exactly (T1-005, T1-006 and T1-007 all give 0.7887 bits at the start of unit U1), so the tie rule decides the demonstrated session and is part of the specification. A value in $[-10^{-12}, 0)$ is clamped to zero; anything more negative raises `negative_eig` as a defect. An empty list returns `None`.

The *reason* stored with each choice is generated from the computation, never written freely: it names the first pair among the three most probable hypotheses (excluding UNMODELLED) whose answer distributions differ by total variation greater than $\delta = 0.05$, states what each predicts, the EIG, the number of candidates, the next-best item and whether a tie was broken by item order. An unlinked item still has positive EIG while UNMODELLED has mass, because UNMODELLED answers uniformly and every other hypothesis predicts the correct answer.

6.6.4 Stopping and refusal

Support of a hypothesis set is the number of answers that its members trigger: a misconception's linked options, NONE's correct option, UNMODELLED's unlinked wrong options. Two hypotheses are separable by an item if their answer distributions differ by total variation $> \delta$. The rules are evaluated in order after each update (Table 6.2).

Table 6.2: Stopping and refusal rules (defaults: $\theta = 0.80$, $\theta_U = 0.50$, `min_support`=2, `min_support_none`=3, $\eta = 0.01$ bits, budget 8).

Rule	Outcome	Fires when
R1	`diagnosed`	The leading misconception has $p \geq \theta$ and support $\geq$ `min_support`
R2	`none_detected`	$p(\text{NONE}) \geq \theta$ and support $\geq$ `min_support_none`
R3	`escalated_unmodelled`	$p(\text{UNMODELLED}) \geq \theta_U$ with enough support
R4	`escalated_no_separator`	Unused eligible items exist, but the set C of hypotheses that no remaining item separates from the leader ℓ has at least two members, $p(C) \geq \theta$, $p(\ell)/p(C) < \theta$ and enough support: the baseline heuristic's ratio is below θ, so the baseline returns this tied set; near-row correctness remains open
R5	`escalated_exhausted`	Budget used, no items left, or best EIG $< \eta$

Exact versus near-row limitation (supervisor action 3). The baseline R4 uses total-variation tolerance $\delta = 0.05$. If two hypotheses have *exactly equal* likelihood rows for every possible remaining observation, their posterior odds cannot change and $p(\ell)/p(C)$ is an exact conditional share bound. Near-equal rows do not have this property: odds are multiplied by their likelihood ratio at each answer, and small differences can accumulate. Therefore the baseline ratio is a heuristic when $0 < \text{TV} \leq \delta$, not an absolute confidence ceiling. The exact-pair D2 example remains reproducible, but general R4 correctness requires exact-row and near-row regression cases, revised behavior or a justified finite-budget bound, and matching specification/tests. No engine correction is claimed by this wording change.

6.6.5 Pack loading

The loader builds unit U1–U7 from `items.csv`, `topics.csv` and a hand-written unit table. Hard rules V1–V11 refuse to load: four options, one correct, unique IDs, no link on the correct option, every linked misconception inside the unit (V5′), at most eight hypotheses, at least a budget's worth of item groups, a supported pack format, and rule P1. Warnings W1–W6 are reported on every load (inseparable pairs, weak separability, hypotheses with fewer than three trigger groups, items with two unlinked wrong options, skipped items). Every exclusion is configuration, never a change to the data. The three source files are hashed (SHA-256) and the hashes, engine version and all parameters are recorded in each trace.

6.7 Algorithms and Technical Decisions

Table 6.3: Technical decisions and open items (spec decision IDs).

ID	Decision	Status
D-1	Merge the inseparable misconception pairs (1514/1773, 1074/1651) in the catalogue instead of keeping them for a refusal demonstration	Recommended; not decided
D-2	Three items that link one misconception to three options are skipped for unit U6 at load; `guess` stays 0.30	Applied (11 Oct.); content fix open
D-3	Raise `min_support` to 3 (requires reproducible policy evaluation before an improvement claim)	Recommended; default kept at 2
D-4	Three items belong to no unit	Left unplaced
D-5	Keep companion hypotheses with fewer than three trigger groups	Kept; documented
EIG	Round EIG to nine decimals before comparing; ties by pack order	Implemented
Links	Optional T links would weigh 0.5; absent grades default to S=1.0, an unresolved candidate-bank weakness	Assumed

6.8 Tools, Frameworks and Technologies

Table 6.4: Tools and technologies.

Tool	Use
Python 3.11+ (CI 3.12), NumPy 2.x, `unittest`	Engine, tests
PostgreSQL 16 (Neon), SQL, DBML, dbdiagram	Schema and its tests
Mermaid, draw.io	ERD and data-flow diagrams
Next.js, React, Node	Interface: student Mathematics diagnostic live, other screens scripted
FastAPI, uvicorn, httpx	Local engine service and its tests (optional extras)
Docker, PostgreSQL 16 image	Throwaway database for the schema checks in CI
GitHub, GitHub Actions	Repository, pull requests, CI (interface, dataset, engine)
Claude (Anthropic), Claude Code, OpenAI Codex	Item drafting; code, tests, diagrams, service and interface (Section 8.7)

6.9 Demonstration Evidence

6.9.1 Normal case: worked example D1 (unit U1, MATH-1379 student)

This command replays four answers; the engine chose every question:

```
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

Table 6.5: Worked example D1 on unit U1 (rounding). Top posterior entries after each answer.

Step	Item	EIG (bits)	Answer	Posterior (top three) after the answer
0	T1-005	0.7887	D, correct	NONE 0.503, MATH-1988 0.352, MATH-1248 0.039
1	T1-006	0.6472	B, correct	NONE 0.842, MATH-1379 0.066, MATH-1988 0.066
2	T1-007	0.2232	B, linked to MATH-1379	MATH-1379 0.567, NONE 0.347, UNMODELLED 0.040
3	T1-009	0.5546	A, linked to MATH-1379	MATH-1379 0.942, NONE 0.028, UNMODELLED 0.024

After step 3 rule R1 fires: `diagnosed`, MATH-1379, $p = 0.942$ with two supporting answers. Each step prints its generated reason, for example at step 2: "Chosen to separate NONE (p=0.842) from MATH-1379 (p=0.065): NONE predicts C = correct; MATH-1379 predicts B. Expected information gain 0.223 bits, best of 18 candidate items; next best T1-008 at 0.223 bits (tie broken by item order)." The first two answers were correct, so the posterior first moved towards NONE; the two answers linked to MATH-1379 then reversed it. This is why `none_detected` requires three supporting answers: with two, the session would have ended there as "no misconception".

6.9.2 Edge case: worked example D2 (unit U4, inseparable pair)

A student who always chooses the option linked to MATH-1514 reaches a posterior of about 0.481 for each of MATH-1514 and MATH-1773 after two answers. Both misconceptions predict the same options on every item of unit U4, so no remaining item separates them and the baseline heuristic's computed ratio is below θ. Rule R4 ends the session as `escalated_no_separator`, naming both. The behaviour is checked by `tests/test_`

engine_stopping.py (test_d2_1514_refused_naming_both) and tests/test_engine_session.py. The refusal is the designed behaviour; the underlying cause is a defect of the data (KF-06) and is not hidden by it.

6.9.3 The same examples in the browser

In the interface a student chooses Student, Start a diagnostic, Mathematics and unit U1, then answers D, B, B, A: the session ends after four answers and the student sees only the neutral completion message. The diagnosis (MATH-1379 at 0.9423, rule R1) is printed by the engine service's trace and by python -m sonder_api.inspect_session. The same flow on unit U4 with answers D, A ends after two answers with the refusal naming MATH-1514 and MATH-1773 (0.4813 each, rule R4), again hidden from the student. Both runs are checked by scripts/check_engine_ui.py.

6.9.4 Failure cases

An option outside the item raises invalid_option; evidence with zero probability raises zero_evidence; an item set that violates P1 at the chosen guess is refused when the unit is loaded or the session started. In each case the session state is unchanged (tests/test_engine_session.py). If the engine service stops during a session, the interface says it could not reach the check and offers Retry, shows no invented result, and a retry resends the identical answer only if it was not recorded, so no answer is counted twice; after a restart the in-memory run is gone and a fresh check works (interface tests).

6.10 Iteration Outcome

The numerical/session checkpoint is implemented, but corrected-core acceptance is incomplete: the content, adverse-policy behavior and near-row stopping limitation remain open. The calculation, stopping and refusal rules, run loop and evidence trail are implemented and tested (merged 11 October 2026), and a student can take a real Mathematics diagnostic in the interface (merged 12 October 2026). Not achieved: saving live runs to the database (runs are held in memory and lost on restart), the production import of the pack into the hosted database, a teacher result screen, and a real login. The evidence trail is seen in the engine terminal and with python -m sonder_api.inspect_session, not yet on a teacher screen.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

The distinction is between implementation verification and diagnostic validity. Exact arithmetic, independent ranking references, deterministic replay and invalid-input checks verify software behavior. They do not validate the candidate item mappings or establish accuracy on real students. The 21 October reproduction exports the unchanged 12 October evaluation commit into a separate report verification folder; it does not run the older diagram-branch engine and call it the evaluation build.

7.2 Test Levels and Techniques

Table 7.1: Verification levels and their limits.

Level	Check	Limit
Numerical	Hand, fraction/decimal and brute-force references	Correct arithmetic does not establish a valid student response model
Contract	Candidate CSV, unit membership, P1 and exclusions	Structural acceptance does not replace Maths-expert review
Session/API	Immutable transitions, retry/conflict, ownership and output projection	Memory tests do not prove restart durability or authenticated class authorization
Synthetic	Deterministic and adverse response policies	Model-constructed truth is not an observed educational diagnosis
Schema	Historical SQL negative writes and import probe	Current report revision does not claim a fresh database execution

7.3 Test Cases

The exact-build suite contains 186 Python tests: Bayesian calculation, selection, pack loading, stopping, session/trace, synthetic policies, no-AI boundary, CSV validation and API behavior. The raw verbose log is retained in `verification/results/python-tests.txt`. The 123 interface tests and schema CI results are historical reported evidence from the baseline; neither the complete UI suite nor PostgreSQL was rerun for this report revision. No current-source passing result is inferred from a CI run on another commit.

7.4 Test Results

The 21 October local run passed all 186 tests. The dataset validator, pack loader and D1/D2 command-line replays also returned exit code zero; full stdout/stderr is retained. The candidate bank contains 117 authored items and 468 options; 114 are assigned to units and 111 are loaded under default exclusions. The three unplaced items and the three excluded P1 items are different sets. Unit U6 has 14 loaded items after its exclusions and drops MATH-1416 from its hypothesis list.

Table 7.2: Locally reproduced deterministic synthetic outcomes, baseline 2c266d4; 43 sessions.

Outcome	Count	Interpretation
Correct diagnosis	26	Model-consistent misconception policies
None detected	7	One deterministic NONE policy per unit
No-separator refusal	8	Exact indistinguishable pairs in the candidate content
Exhausted	2	Insufficient useful trigger evidence
Different misconception label	0	Only for this deterministic suite; adverse policies below still expose wrong labels

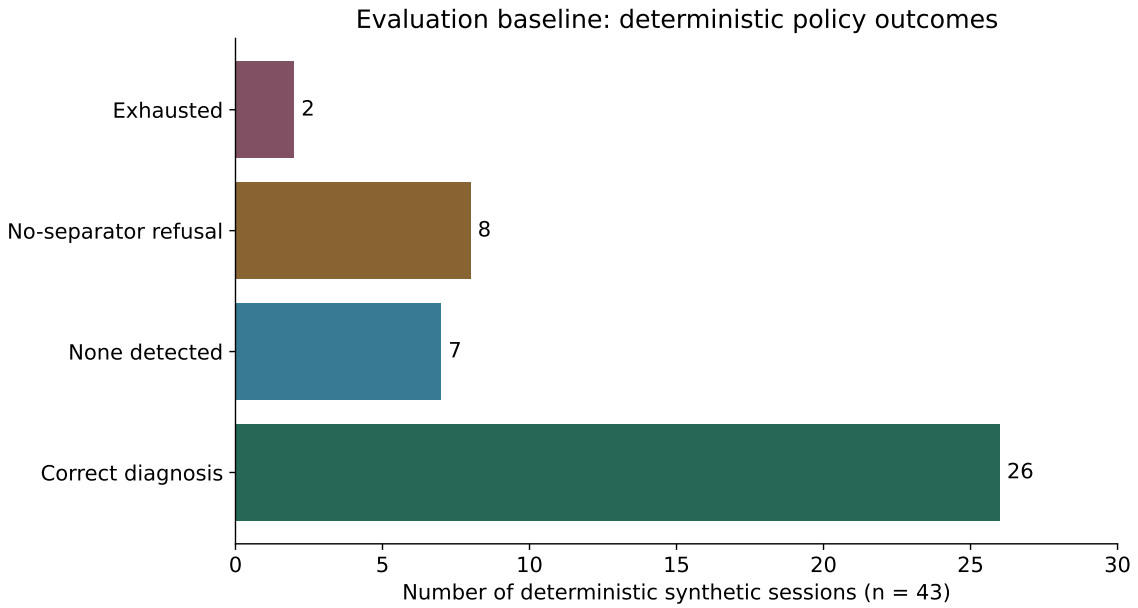


Figure 7.1: Counts from `deterministic-students.csv`; 43 deterministic sessions on the 111 loaded candidate items in seven units, default assumed parameters and budget 8. There is no randomness or real-student sample. Counts verify baseline behavior, not empirical accuracy.

7.4.1 Performance

The raw timing experiment uses 100 fresh runs each on U1, U4 and U7. Each timed observation includes `next_question` and `answer` for the correct answer to the first selected question; `start` is outside the timed interval. Parameters are defaults, the question budget is eight, and timing uses `perf_counter`. This is one first-step library experiment on Windows/Python 3.14, not full-session, browser, network, database, stress or classroom timing. Results depend on machine load.

Table 7.3: 21 October local library timing; 100 first-step observations per unit.

Unit	n	Median (ms)	95th percentile (ms)
U1	100	35.36	84.80
U4	100	40.96	62.70
U7	100	40.80	58.97

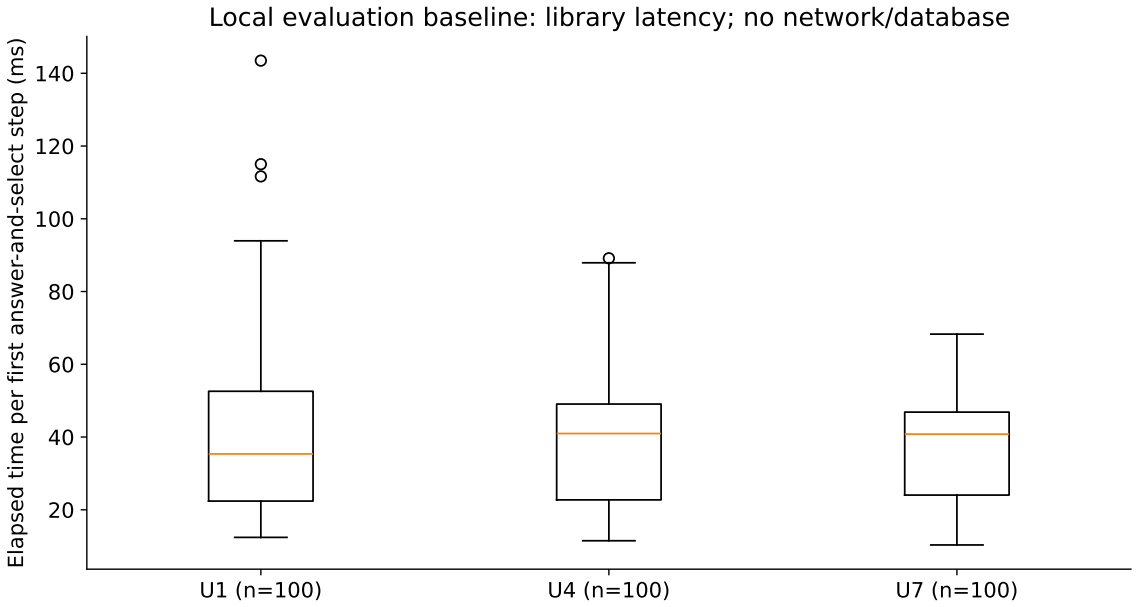


Figure 7.2: Raw-backed first-step timing distributions from `step-latency.csv`, 100 observations per unit. Boxes show quartiles/median, whiskers the default 1.5-IQR range and points outliers. Units are milliseconds; settings are default Params, unchanged baseline, one local process. The proposed 300-ms complete-path requirement is not established by this library-only measurement.

Earlier timing numbers without retained raw samples are removed from acceptance claims. A new end-to-end measurement with persistence and access control is required before claiming the deployed target.

7.5 Acceptance Testing / Validation

R1 and R8 have numerical and replay evidence. R2's outcome branches execute, but general near-row correctness and the adverse-policy cases remain unresolved. R3 covers the declared Maths units. R6 has structural/schema evidence but no demonstrated bank-wide external expert acceptance. R9 is a local memory checkpoint, not restart-resilient persistence. R4's authenticated review/release and R5's communication are planned. The R7 library measurement is bounded as above. This mixed status replaces a blanket statement that the core has passed educational acceptance.

7.6 Edge and Failure Cases

Table 7.4: Retained adverse and failure behavior.

Case	Observed baseline behavior or outstanding check
Always-A policy	Produces diagnoses in six units and exhausts U3; truth is not a particular misconception, so the labels must not be called correct
Partial MATH-2039 holder	Constructed policy receives MATH-1965 in U2 after three answers; retained as a wrong-label counterexample
Exact pair D2	U4 D,A returns R4 naming MATH-1514 and MATH-1773; neutral public completion
Near-equal rows	Absolute-ceiling claim is invalid; dedicated exact/near regressions and specification correction remain required
Identical/conflicting retries	Cached original reply versus conflict; covered by API tests and SD-02
Invalid option/zero evidence	Reject without publishing a partially changed state
Service/restart failure	Retry/unavailable behavior shown in SSD-05/SD-06; memory state is lost on Python restart; no durable-recovery claim

Raw adverse outputs are in `adverse-policies.csv`. They are different from random-answer simulation results. No random-clicker percentage, threshold improvement or noisy-student percentage is reported without a retained reproducible experiment.

7.7 Defects and Corrective Actions

Table 7.5: Report corrections versus uncompleted technical corrections.

Issue	Current handling
Nominal counts presented as live coverage	Corrected to 117 authored / 114 assigned / 111 loaded
Unsupported numerical and comparison claims	Removed or source-qualified; new figures derive from retained raw data
Near-row R4 ceiling wording	Qualified in design and implementation chapters; code/specification fix still open
Weak content and unverified expert approval	Candidate status retained; external expert review required
Missing answer/failure sequence evidence	SSDs/SDs distinguish memory behavior from proposed persistence/review
Repeated indexes and stale front headers	Class/main front-matter generation repaired; signatures remain unsigned

7.8 Current Validation Status

External expert review, empirical accuracy, calibration, independent teacher labels, equal-budget seeded adaptive/fixed/random comparisons and teammate reproduction remain outstanding. The bank is not accepted for trusted student use. The new local reproduction is Codex-assisted, not a substitute for a named independent teammate's check.

Table 7.6: Known-fault groups retained as open limitations.

Baseline IDs	Remaining limitation
KF-01/02/05	Three P1 exclusions, U6 hypothesis drop and three unplaced items
KF-03/07/12	Adverse-response diagnoses, subset confounding and partial holders resembling NONE
KF-04/06/08	Sparse trigger groups and exact/weak separating coverage
KF-09/10/11	Ungraded disputed links, assumed parameters and uncalibrated thresholds
KF-13/14	Unlinked options influence UNMODELLED; reasons show option letters
Supervisor action 3	Near-row R4 is heuristic; exact ratio invariance does not generalize to tolerance-based equivalence

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

- Repository, branch and pull-request workflow, contribution rules and five CI jobs (interface build and tests; dataset validator; engine tests; database schema checks in a throwaway PostgreSQL container; engine API tests).
- Maths_v2 content pack: 117 student-approved candidate items (114 assigned; 111 loaded), 468 options, 28 misconceptions in seven topics; dataset validator and an independent review of the first topic (T1) by Ashher.
- Engine specification v2, re-measured on the real bank, with decisions D-1 to D-6 and tasks T-1 to T-3, and its amendments (D-2 to D-5).
- The engine library: prior, likelihood, posterior, link grades and twin filtering; EIG selection; stopping and refusal rules; run loop; evidence trail; pack loader; command-line replay; 186 Python tests in total; 14 documented known faults.
- A local memory-based engine service and student Mathematics diagnostic in the interface, with 16 service tests, 123 interface tests and an end-to-end check script.
- PostgreSQL schema revision 3 (41 tables, 110 triggers) with a behaviour test suite and a data-load probe, both run automatically in CI.
- Design artefacts: seven ERDs, the data-flow document, the architecture diagram, state diagrams, and a proposed class-flow plan.

8.2 Work Remaining

Saving live runs and their evidence trail to the hosted database and the production import of the pack (T05.07); a teacher result view and a real login; resolution of the open specification decisions (D-1, D-3, T-1, T-2); the noisy-student and random-clicker simulations as committed, reproducible tests; a reviewed link-grade column; qualified independent Maths-expert review of every retained item (Ashher's T1 check is not external acceptance); the tagged evaluation commit. Later iterations are in Chapter 9.

8.3 Individual Responsibilities

See Table 3.3. Each work item has a lead and a reviewer; the work division is recorded in `ITERATION_1_TEAM_WORK_DIVISION_MATH_ONLY.md`: Harris reviews Ahmed's pull requests, Ahmed reviews Ashher's, and Ashher reviews Harris's.

8.4 Contribution of Each Team Member

Table 8.1: Contribution by member (pull requests merged into `dev`, counted by branch owner; #3 is a `dev` to `main` synchronisation).

Member	Merged PRs	Pull requests	Substantive contribution
Muhammad Harris Azhar	5	#5, #11, #14, #20, #26	Maths_v2 reviewed pack and its review notes; engine specification v2; EIG selection and its tests; ERDs and data-flow design
Syed Ashher Majid	8	#6–#9, #22–#25; #39 (documents, merged after the code freeze)	Dataset validator and the independent T1 item review; prior, likelihood and posterior; spec v2 contract, per-item P1 check, link grades, twin filtering and validator alignment; the implemented diagnostic state-machine diagram and the technical data-flow review and corrections (PR #39)
Ahmed Laiq	21	#1, #2, #4, #15–#19, #21, #27–#38	Repository, CI and rules; specification draft and amendments; schema revisions 1–3, tests and CI checks; architecture diagram; pack loader, stopping rules, run loop, trace, synthetic-student harness, known-fault register; engine service and live student screen; class-flow plan

The panel's first comment applies: the commit counts on `dev` (Ahmed 52, Ashher 31, Harris 11 at commit 2c266d4, merges included) and the merged pull-request counts are uneven. Commit counts are not used here as evidence of contribution; the table lists what each member designed, implemented, tested or reviewed. The individual evidence sections of the supervisor's worksheet carry each member's own account.

8.5 Repository and Version Control Evidence

Repository: https://github.com/sonder-fyp/SONDER. Branches: `main` (demo-ready, evaluation tags), `dev` (integration), feature branches named `<name>/<feature>`. Rules (CONTRIBUTING.md): no direct pushes; a pull request into `dev` with CI green; `dev` to `main` with one approval and a regular merge commit. Branch protection is not enabled on GitHub, so these rules are conventions. Historical process statement: since pull request #15, merges into `dev` need no GitHub approval; the work division names a reviewer, but a review cannot be inferred from that assignment or a merge. The repository was created on 9 October 2026 from existing proposal documents and the proof-of-concept interface. Evaluation commit: `2c266d47472e130eea2fb68ae1c19972d0994cee` on `dev`, 12 October 2026 (code freeze, used for the live demonstration); no release tag exists yet. No task board or GitHub issues are used: work items follow the task IDs T00 to T14 of the work-division document, and the pull requests are the record of completed work. Historical CI link (different commit, not proof of this revised report's complete build): https://github.com/sonder-fyp/SONDER/actions/runs/38177574282 (success).

8.6 Milestones and Commit Evidence

Table 8.2: Milestones (merge dates from the repository history).

Date	Pull request	Milestone
10 Oct 2026	#1–#5	CI fixes; engine specification draft; Maths_v2 pack merged into `dev`
11 Oct 2026	#6–#9	Dataset validator; Bayes prior, likelihood, posterior
11 Oct 2026	#11, #14	First ERD and data flow; engine specification v2
11 Oct 2026	#15–#19	Contribution rules; schema revisions 2 and 3; SQL and end-to-end schema design
11 Oct 2026	#20	EIG selection with tests
11 Oct 2026	#21–#26	Architecture diagram; engine v2 contract, link grades, eligibility, validator; revision-3 ERDs and data flow
11 Oct 2026	#27–#34	Spec amendments; pack loader; stopping rules; run loop and trace; synthetic-student harness; database checks in CI
12 Oct 2026	#35–#38	Engine service and live student screen; architecture diagram update; Word lock file removed; class-flow plan
12 Oct 2026	#39 (after the code freeze)	Ashher's implemented diagnostic state-machine diagram and technical data-flow review and corrections

Twenty-six of the 35 pull requests merged into `dev` were merged on 11 October and four more on 12 October. This reflects how integration was done late, not how the work was spread; it is recorded as a deviation in Chapter 3. Ahmed's engine work was delivered as a planned sequence of small pull requests, each building on the last and each listing its tests. The pull-request list and the CI runs are available at the repository's `pulls` and `actions` pages (Appendix D).

8.7 Tools and Tool-Assisted Development

Generative AI levels follow the course definition: L1 supportive, L2 assisted, L3 core assist, L4 substituted. No artefact is presented as L4.

Table 8.3: GenAI register for materially assisted artefacts.

Artefact	Tool and level	What entered the project	Verification	Owner
Maths_v2 item bank	Claude; L2	Items, options and misconception links generated in batches; only approved batches kept; one batch left as a draft	Student-owner approval, agent audits, validator and T1 review; external expert acceptance pending	Harris
`selection.py`, `test_selection.py`	Claude Code; L3	Module written test-first from the specification; all kept	Exact-arithmetic and brute-force checks; spec examples; seven injected bugs; first draft found diverging and rewritten	Harris
ERDs and data-flow document	Claude Code; L3	Mermaid code and document; exported and edited in draw.io	Checked against the SQL and DBML; three defective images re-exported; wrong freeze order corrected	Harris
Engine specification v2	Claude Code; L3	Text generated from Harris's plan, reviewed and re-prompted	Worked examples reproduced by tests; open decisions listed	Harris
Bayesian prior, likelihood, posterior	OpenAI Codex; L3	Code, input checks, tests and notes from the team spec	19 core and 6 candidate-T1 tests; hand and exact-fraction checks	Ashher
Validator and independent T1 review	OpenAI Codex; L3	Validator code and tests; a draft option-level review	16 validator tests; current bank: 0 structural errors, 3 P1 conflicts, 6 warnings; all 20 T1 marked answers checked; 11 weak links flagged	Ashher
Spec v2 contract, P1 check, link grades, twin eligibility	OpenAI Codex; L3	Model validation, link-grade blending, twin exclusion and tests	Weight-1 and twin-order tests; three P1 conflicts reported, not edited away	Ashher
Specification draft and amendments	Claude Code; L3	Draft and decisions D-2 to D-5	Rechecked against the bank and the engine	Ahmed
Database schema revisions, tests, tools	OpenAI Codex and Claude Code; L3	Schema, DBML, behaviour tests, import probe, Docker test runner	75 refused writes; 15 of 15 checks in CI; all 117 items load	Ahmed
Architecture diagram	Claude Code; L3	Specification and SVG generator	Layout checks; labels checked against `dev`	Ahmed
Engine run loop, loader, stopping rules, trace, harness	Claude Code; L3	Implemented from Ahmed's reviewed plan, test-first	89 run-loop tests; D1, D2; 43 synthetic students; injected faults caught	Ahmed

Artefact	Tool and level	What entered the project	Verification	Owner
Engine service and live student UI	OpenAI Codex (GPT Sol 6.1); L3, reviewed by Claude Code	Service and screens; problems found in an independent review were fixed	Four defects fixed; 16 service and 123 interface tests; live checks	Ahmed
Class-flow plan	OpenAI Codex; L2	Four-page proposal	Checked against the class tables; labelled a proposal	Ahmed
Original report / current revision	Claude Code / Codex; level for members to confirm	Baseline draft plus scope, research and traceability corrections, sequence/failure diagrams and raw-backed evidence	Automated checks described here; personal review and final GenAI confirmation pending	All

The 21 October report revision and reproduction tools are Codex-assisted. Attribution in this table is a draft inventory; no personal confirmation, signature or external review is supplied by the tool. Each member must confirm their own contribution and assistance level after checking the final revision.

The diagnostic calculation contains no generative model, and the engine never calls one at run time; the repository tests assert this. Mistakes in AI output that were found and corrected are listed in Table 7.5.

8.8 External Components and Dependencies

Table 8.4: External components, risks and fallbacks.

Component	Purpose	Risk or limitation	Fallback
Python 3.11+, NumPy 2.x	Engine	Version drift; float rounding (guarded by tolerances and rounding rules)	Pin the version range
FastAPI 0.120.0, uvicorn 0.38.0, httpx 0.28.1	Local engine service and its tests (optional extras)	Needed only for the live flow; one process holds runs in memory	The library and command-line session need none of them
PostgreSQL 16 (Neon); Docker image for tests	Storage; throwaway database for CI checks	Hosted-service availability and cost; schema size; Docker needed locally	Local PostgreSQL; the engine runs without it
Next.js, React	Interface; student diagnostic live	Other screens are scripted fixtures	Keep the labelled demo
Eedi catalogue (Kaggle)	Source of misconception names for Mathematics	Licence and redistribution terms not yet recorded; single-source bias	Self-authored misconceptions
Claude and OpenAI Codex (LLMs)	Draft items, code, diagrams	Wrong or forced links (about 90 stretch links remain); generation errors	Versioned candidate pack awaiting external acceptance; not needed at run time
GitHub and Actions	Repository, CI	Availability; branch protection off	Run the same commands locally

8.9 Risks and Mitigation

Table 8.5: Risk register.

Risk	Level	Mitigation
Parameters are assumed and the data are synthetic, so accuracy claims fail on real students	High	Do not claim accuracy; calibrate on a pilot; report negative results
Uneven contribution within the team	High	Per-member evidence table; named reviewers; weekly contribution check; individual ownership in the worksheet
Live runs are in memory and not saved; no login or teacher view	High	Save runs and traces to the session tables, add a teacher result view after G3/G4 and authenticated release at G5
Inseparable misconception pairs make refusals a data artefact	Medium	Merge pairs (D-1); write separating items (T-2)
Random clickers diagnosed too often	Medium	Evaluate content/model changes and threshold trade-offs using retained seeded data; do not assume support=3 fixes the adverse cases
Schema too heavy for the current scope	Medium	Open decision on slimming; the schema stays behind the engine interface
Panel scope concern: weak technical challenge	Medium	Challenge made explicit (Chapter 2); Maths-only scope direction recorded in the 16 October summary; dated panel closure evidence still pending
Ethics approval delays the pilot	Medium	Start the approval process in FYP-2 Month 1; simulated studies meanwhile
Licence of the Eedi catalogue; no repository licence file	Low–medium	Record the licence; keep only misconception names and IDs; add a repository licence

8.10 Security and Privacy Considerations

No student-response collection is documented in the reviewed artifacts; no participant collection is authorized by this report. The proposal commits to pseudonymous identifiers, data minimisation, written guardian consent, student assent, withdrawal rights, retention limits and formal ethics approval before any pilot with minors (approval not yet obtained). The local engine service requires a shared token, owns each run through a signed HttpOnly cookie (not marked secure, since it runs on local HTTP), checks the request origin and listens on the loopback address only; there is no real login and no deployed service. The schema defines roles, hashed credentials and tokens, and append-only and frozen records; the engine performs no I/O. CONTRIBUTING.md forbids secrets in the repository (connection strings go in GitHub secrets or local environment files); no

automated secret scan has been run yet. An RBAC audit and hardening are planned for Iteration 4.

8.11 Current Limitations

- Live runs are held in memory and lost on restart; they are not saved to the database. There is no real login and no teacher result screen; the evidence trail is read in the engine terminal.
- All probabilities and thresholds are assumed; the bank is synthetic; accuracy, calibration and teacher agreement are untested.
- The revised R3 is bounded to the declared Maths units; no additional-subject acceptance target remains. Only topic T1 has an independent item-by-item review; qualified external expert acceptance of every retained item remains outstanding.
- Fourteen known faults (Table 7.6); in particular inseparable pairs, skipped items and adverse diagnoses under the retained response policies.
- The browser walkthrough is checked by hand; no automated browser test exists.
- One teacher interview; no student or usability evidence.
- Specification measurements are reproduced in the repository for the deterministic students only; the noisy-student and random-clicker figures rest on reference scripts that are not in the repository.

8.12 Peer Review and Confirmation Evidence Still Required

Reviewer names and merge counts establish neither review depth nor correctness. For each accepted package the team must retain: the PR/commit and reviewer, the exact challenged issue, the author's response/change, the check rerun against that change and the dated decision. The present report does not have verified PR-thread excerpts for those fields and does not invent examples. A correction attributed to an AI review is not automatically an independent human review. The baseline contribution table is historical; the proposed 16 October allocation is in Chapter 3 and must be explicitly agreed.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

The active scope is Grade-8 Mathematics only. The priority is content validity, diagnostic reliability, persistent evidence and a real teacher workflow. Optional features are conditional on that core. No additional-subject dataset or evaluation target is scheduled.

9.2 Remaining Modules and Features

Table 9.1: Remaining work in dependency order.

Module	Deliverable
Candidate Maths release	Item-level construction/mapping decisions, reviewer expertise, dated external expert acceptance, rechecks and exact version
Diagnostic reliability	Always-A and partial-holder regressions, exact/near-row corrections, equal-budget seeded experiments and raw outputs
Persistent U1 service	Durable run/answer/trace storage, pinned content/parameters, restart/resume, retry/conflict and replay checks
Teacher evidence view	Real records and trace, explicitly owned after persistence contract is ready
Trusted release	Authentication, class scope, authorized correction/approval/rejection and student/parent release controls
Conditional product	Interventions and verification; communication, tutor and generation features only after the core is accepted

9.3 Remaining Iterations

Iteration 2 corrects and verifies the core before FYP-I final. Iteration 3 adds authenticated teacher release and the bounded intervention/verification cycle. Iteration 4 measures deployment, security and a consented Maths pilot where approvals permit. These are acceptance gates, not evidence of implementation.

9.4 Implementation Plan

Gate G1: Harris coordinates independent expert acceptance; Ashher checks coverage and Ahmed checks import consistency. G2: Ashher corrects stopping/reliability and supplies versioned baseline experiments; Harris independently reproduces them. G3: Ahmed implements persistent U1 and publishes its API contract; Ashher reproduces restart/retry/replay. G4: Ashher implements the real teacher evidence view after G3, subject to explicit ownership agreement. G5: agree and implement authenticated approval/release before trusted use. Dates, capacity and review appointments remain to be agreed; no completion is inferred from the old milestones.

9.5 Testing and Validation Plan

Table 9.2: Maths-only planned studies and required controls.

Study	Required protocol
V1	Seeded adaptive, fixed and random selection on the same bounded U1/U2 cases, equal budgets and explicit truth labels; keep adverse policies and raw outputs
V2	Additional classifiers only if fully specified and feasible; no claimed LLM superiority before a fair held-out comparison
V3	Held-out Maths labels only after source permissions, item mapping and evaluation/tuning split are defined
V4	Teacher-led pilot with consent, privacy, withdrawal process and separate immediate/delayed verification; conditional on access approval
V5	Qualified external expert review of every retained candidate item; identity, expertise, dated decisions and accepted version recorded
V6	Student-response check of the Q-matrix, distinct from expert content acceptance and code testing

Report incorrect diagnoses, diagnosed coverage, refusal category and question counts together. Wrong-diagnosis rate is wrong labels divided by diagnosed cases; coverage is diagnosed cases divided by eligible cases, with NONE cases reported separately. Do not treat posterior probability as accuracy. Separate parameter tuning from held-out evaluation and preserve seeds, policy definitions, versions and uncertainty. Proposal aspirations (at most 5% confident misdiagnoses with at least 70% coverage, and teacher agreement $\kappa \geq 0.7$) are unverified targets, not results.

9.6 Deployment Plan

Run the interface and engine on a CPU host with PostgreSQL, maintaining a source checkout containing content, API and interface dependencies. Measure the complete path separately from library latency. Low-bandwidth operation around 2 Mbps, interrupted-session recovery, role access and rollback must be tested. No optional model host is required for the core.

9.7 Tentative Timeline

G1–G3 precede the FYP-I final core review; G4 follows the published persistent contract. G5 precedes trusted student use. Pilot preparation follows accepted content, reliable diagnosis, access authorization and ethics/consent approval. Concrete dates and the proposed transfer of the teacher view must be recorded by the team; this revision does not manufacture commitments.

9.8 Expected Final Deliverables

A bounded Maths application with accepted content, versioned deterministic evidence, durable replay, authenticated teacher decisions, measured limitations and reproducible evaluation. Optional features and a pilot are included only when their acceptance and access prerequisites are met. Final artifacts include source/build instructions, raw results, review records and a report consistent with the accepted release.

Chapter 10

Conclusion

Sonder addresses ambiguity in a bounded Grade-8 Mathematics misconception bank through Bayesian updating, expected-information-gain selection and explicit refusal. These methods are established; the project must demonstrate a valid, reliable and useful implementation rather than infer novelty from unsupported comparisons.

The baseline engine and local student flow are implemented, but the bank remains a candidate pending qualified external expert acceptance. The counts are 117 authored, 114 assigned and 111 loaded items. Deterministic synthetic runs and arithmetic checks establish behavior under assumed inputs; adverse policies expose incorrect or unsupported labels. Near-equal likelihood rows do not justify an absolute ceiling. Local raw-backed figures are separated from historical CI reports and from unperformed empirical studies.

The 21 October revision applies the supervisor's Maths-only direction, corrects research/evidence statements and ownership chronology, restores R5 traceability, adds sequence and failure interactions, and presents the supplied Chen ERD with its clearer online link. It does not claim a corrected engine, durable service, authenticated teacher workflow, signed declarations or independent dataset acceptance. Those remain explicit acceptance gates in the revised plan.

Appendix A

Proposal Action Register

The proposal follow-ups are in Chapter 3. This register maps the 16 October supervisor action numbers to the current report corrections. Technical acceptance remains open where no independent evidence exists.

No.	Report action	Remaining acceptance evidence
1	Candidate status, weak mappings and expert requirement stated in Chapters 1, 2, 7	Qualified external review of retained items, dated corrections and exact accepted version
2	Unsupported percentages removed; raw-backed deterministic/adverse outputs retained in Chapter 7	Seeded equal-budget adaptive/fixed/random evaluation and peer reproduction
3	Exact/near-row limitation stated in Chapters 2, 5–7	Revised specification/behavior and exact/near-row regression cases
4	Memory behavior separated from target persistence/teacher view in Chapters 4–6, 9	Durable U1 path, real teacher view and authenticated release
5	Maths-only active scope and dated records in Chapters 1, 3, 9 and Appendix F	Agreed roadmap update and panel closure evidence
6	Primary-reference positive comparisons; unsupported EDGE role removed in Chapter 2	Any later capability audit requires claim-level evidence
7	117/114/111 counts, proposed allocation and unsigned personal confirmations	Each member's dated confirmation and actual peer-review examples

No.	Report action	Remaining acceptance evidence
8	Raw-backed readable plots; SSDs/SDs; labelled memory/proposed paths; supplied Chen ERD and clarity link	Team verifies figures; technical fixes remain separately open
9	R5 traceability restored; current versus target access separated; front matter/indexes repaired	Final member declarations and signatures when due
10	Exact baseline, content/parameter hashes and commands recorded in Appendix D	Complete corrected application reproduced by a named teammate; SQL/UI executions where not rerun

Appendix B

Detailed Requirements

Status is that of the evaluation branch `dev` (commit `2c266d4`, 12 October 2026).

R1 — Next-question selection (functional; owner Harris)

Statement. Given the current posterior over a unit's hypotheses and the eligible items, the engine chooses the item with the highest expected information gain in bits. Exact ties go to the earliest item in pack order. It returns the full ranking, the entropy before the answer and a generated reason.
Source. Proposal FR-01; engine specification §3.4.
Acceptance. EIG matches exact arithmetic to 10^{-11}; choice and ranking match a brute-force reference on random posteriors; the specification's worked examples reproduce to four decimals (D1: T1-005 0.7887, T1-006 0.6472, T1-007 0.2232, T1-009 0.5546; D2: T4-001 0.7517, T5-011 0.3668); an empty list returns `None`; a negative EIG beyond -10^{-12} raises an error; asked items and their twins are excluded.
Evidence. `selection.py`; 40 tests; seven injected bugs all caught.
Status. Met as a library. **Gaps.** Reasons show option letters, not text (KF-14); the budget and stopping decision belong to the run loop, which Ahmed built.

R2 — Refusal as a first-class outcome (functional; owner Ahmed)

Statement. When a supported diagnosis cannot be made, the run ends with an explicit refusal and the evidence behind it; a forced diagnosis is never presented.
Rules (spec §3.5). R1 diagnosed: leader $p \geq 0.80$ with at least two supporting answers. R2 none_detected: $p(\text{NONE}) \geq 0.80$ with at least three. R3 escalated_unmodelled:

$p(\text{UNMODELLED}) \geq 0.50$. R4 escalated_no_separator: a tied set that no remaining item can separate. R5 escalated_exhausted: budget (8) spent, no items left, or best EIG below 0.01.

Acceptance. Each outcome is produced by a constructed case; the D2 student (MATH-1514, MATH-1773) is refused naming both; a student without a misconception is `none_detected`.

Evidence. `stopping.py`, `session.py`; `test_engine_stopping.py` (22), `test_engine_students.py`.

Status. Implemented baseline behavior, with unresolved adverse-policy and near-row stopping limitations; results are not yet saved to the database or shown to a teacher. **Gaps.** Thresholds assumed (KF-10, KF-11); random clickers diagnosed often (KF-03).

R3 — One engine for the declared Maths units (functional)

Acceptance. All declared Maths units use the same inference API, with versioned hypotheses, explicit mappings and exclusions. No second-subject target remains.

Status. Implemented in the baseline; expert content acceptance remains outstanding.

R4 — Teacher approval gate (functional; target trusted workflow)

Statement. No diagnosis reaches a student or parent before a teacher approves, corrects or rejects it; until then the student sees only "complete, awaiting teacher".

Status. Authenticated approval/release is deferred until G5; reviews currently gate content only. The student view already never carries a diagnosis, posterior, rule or correct answer (`sonder_api/student_view.py`, tested).

R5 — Announcements and messaging (functional)

Acceptance. A teacher can send a message or announcement and a permitted recipient can read it; role and class rules hold.

Owner and milestone. Harris is the proposed owner, subject to explicit team agreement; conditional Iteration 3 after the core is accepted.

Acceptance test. A permitted teacher sends a class announcement/direct message, the permitted recipient reads it, and cross-class or unauthorized guardian access is denied.

Status. Deferred; fixtures only, not in the Iteration 1 schema.

R6 — Versioned, reviewed, frozen packs (functional; data owner Harris, rules Ahmed)

Statement. An item is approved only by a recorded review from someone other than its author; a frozen, hashed pack is immutable; the trusted target requires frozen externally accepted content; the local baseline uses the candidate loader.
Acceptance. Invalid edits are rejected (75 invalid writes rejected, run in CI); Maths_v2 loads (117 items, 468 options, 114 assigned and 111 loaded items in seven units); rule V5′ holds in every unit; all 28 misconceptions are used.
Status. Partial. The schema rules, pack loader and a CI import check exist (75 rejected writes, 15 of 15 checks, all 117 items load); the production import into the hosted database is not done; the three unplaced items, the link-grade column and a bank-wide human spot-check are open.

R7 — Latency (non-functional)

Acceptance. One posterior update plus selection takes under 300 ms.
Evidence. The 21 October local reproduction stores raw first-step timings (100 per U1/U4/U7), versions and content/parameter hashes. See Section 7.4.1.
Status. A bounded library measurement; deployed end-to-end latency remains unverified.

R8 — Determinism and input validation

Acceptance. Same pack, parameters and answers give identical posteriors, selections, reasons and decisions; an invalid option raises `invalid_option`; zero evidence raises `zero_evidence`; a rejected request leaves the state unchanged; parameters are recorded in every trace.
Status. Met for the engine and the service; pinning parameters per session in the database is not yet integrated.

R9 — Live student diagnostic (functional; owner Ahmed)

Statement. A student takes a Mathematics diagnostic in the interface that is computed by the real engine, can resume after a refresh, and never sees the result.
Acceptance. Answers in the browser produce the engine's next item; a refresh resumes the same question; replies carry no diagnosis, posterior, rule or correct answer; a lost reply

is retried without counting the answer twice; the D1 and D2 sessions match the engine.
Evidence. `sonder_api/`; 16 service tests; 123 interface tests; `scripts/check_engine_ui.py`.
Status. Met as a local checkpoint. **Gaps.** Runs are held in memory; no login; no teacher result view.

Proposal commitments not yet in R1–R9

Accuracy and calibration targets (at most 5% confident misdiagnoses with at least 70% coverage; $\kappa \geq 0.7$) are untestable without real data. The evidence trail (every question, ranking, answer, posterior and reason, append-only) is supported by the schema and by the trace module; storing it is not yet integrated. Privacy and security (pseudonymous identifiers, role-based access) apply to the product layer. Low-bandwidth usability (about 2 Mbps) and a usability score are deferred.

Appendix C

Detailed Test Cases

C.1 Deterministic students through the run loop

Each student always chooses the first option linked to its misconception (or the correct option for NONE) and never slips. Outcome: D diagnosed, N none_detected, S escalated_no_separator, X escalated_exhausted; the number is the count of answers used. These are the results of running the engine on the evaluation branch; `tests/test_engine_students.py` asserts the totals.

Unit	Student (MATH id): outcome and answers
U1	1248: D2; 1379: D3; 1988: D3; 2392: D2; NONE: N3
U2	61: D3; 658: D2; 1715: D2; 1965: D3; 2039: D2; NONE: N3
U3	2138: D3; 2206: D3; 1074: S3; 1651: S3; 2359: D3; NONE: N3
U4	126: D3; 228: D2; 1416: D3; 1514: S2; 1773: S2; 2093: X5; NONE: N3
U5	414: D2; 734: D2; 952: D3; 1072: D2; NONE: N3
U6	1074: S3; 161: D2; 767: D2; 1651: S3; 2359: X4; NONE: N3
U7	2359: D2; 1639: D4; 2093: D2; 2271: D3; 1514: S3; 1715: D2; 1773: S3; NONE: N4

Totals: 26 D, 7 N, 8 S, 2 X (43 students); no student needs more than five answers.

C.2 Injected faults (mutation check)

Wrong logarithm base; no rounding of EIG; reversed tie rule; worst item chosen; UNMODELLED allowed as a leader in the reason; >= instead of > in the separation test; wrong guess parameter. Each was caught by at least one test.

C.3 Selection test classes (`tests/test_selection.py`)

Class	Tests	Checks
EntropyTests	4	Uniform, deterministic and mixed distributions; zero terms
ExpectedInformationGainTests	7	Hand values; agreement with the update route; bounds; clamp and error for negative values
SelectNextTests	9	Highest EIG wins; ties by pack order; ranking order; empty list; rounding
SelectionReasonTests	9	Template fields; pair rule with total variation strictly above δ; tie sentence; fallback sentence
SpecWorkedExampleTests	2	D1 and D2 choices, EIGs and posteriors to four decimals
IndependentRealBankChecks	6	405 exact-fraction EIG values; 300 random posteriors against brute force; bounds; shuffled order; scripted sessions for U1–U5
Eligibility checks	3	Twin partners and asked items excluded; pack order preserved

C.4 Commands to reproduce

```
python -m unittest discover -s tests -v
python -m unittest discover -s tests -p "test_selection.py" -v
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

Appendix D

Repository and Contribution Evidence

D.1 Version and Complete Source Checkout

The implementation baseline is full commit `2c266d47472e130eea2fb68ae1c19972d0994cee`. The 21 October report revision is a review candidate in HarrisDOcs, not a newly tagged application release. The report's verification folder contains a Git-archive export of that commit and a local reproduction script. The current diagram branch is older and must not be substituted for the baseline.

```
git clone https://github.com/sonder-fyp/SONDER.git SONDER-review
cd SONDER-review
git checkout --detach 2c266d47472e130eea2fb68ae1c19972d0994cee
python -m pip install -e ".[api,api-test]"
python -m unittest discover -s tests -v
python -m validation.math_dataset
python -m sonder_pack.maths_v2
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
python -m sonder_pack.run_session --unit U4 --answers D,A --table
cd Sonder-UI-master
npm ci
npm test
npm run lint
npm run build
```

Use Python 3.11+ (CI baseline 3.12; local reproduction 3.14), NumPy 2.x and Node 22 per the baseline configuration. API extras pin FastAPI 0.120.0 and uvicorn 0.38.0; API-test extras pin httpx 0.28.1. Local installed versions are recorded separately in the verification environment file; differences are not hidden. If test helper imports are not resolved

by a workstation, set PYTHONPATH to that checkout's tests directory before unittest discovery.

D.2 Both Services and Expected Output

Generate local service-token and cookie-signing keys; keep them in the service environment and an ignored UI environment file. Never publish the values. The Next.js file needs SONDER_ENGINE_URL, SONDER_ENGINE_TOKEN, SONDER_COOKIE_KEY and SONDER_UI_ORIGIN. The token must match the engine's environment. Choose http://127.0.0.1:8000 for the engine URL and http://127.0.0.1:3000 for the UI origin; the configured origin and signing keys must remain stable during a rehearsal.

```
# Terminal 1, source checkout root; service token already set locally
python -m sonder_api --host 127.0.0.1 --port 8000 --trace
# Terminal 2, source checkout root
cd Sonder-UI-master
npm run dev -- --hostname 127.0.0.1 --port 3000
# Terminal 3, source checkout root, token set in this shell too
python scripts/check_engine_ui.py --all-policies
```

Expected: seven units with 111 loaded items and explicit exclusions/warnings. D1 selects T1-005, T1-006, T1-007, T1-009 and diagnoses MATH-1379 internally. D2 selects T4-001 then T5-011 and returns R4 internally. Both student pages show neutral completion. Restarting the Python service loses runs: that is a documented limitation, not successful persistent recovery. The pure-engine wheel alone is not the complete application; the content, pack loader, API, interface and configuration are required.

D.3 Content and Parameter Identity

Table D.1: Exact baseline content and assumed-parameter hashes.

Artifact	SHA-256
data/Maths_v2/bank/items.csv	`af0f75846c763bdc19ee1760ba22e1f4426b27c52ef5933a616f53cd0583b45a`
data/Maths_v2/topics.csv	`7d88223803beb462501eb9fa1cb882dd4a44c841324c616bb6de36494d765972`
sonder_pack/maths_v2_units.json	`61cfff3884738a2fac2621a3443e64455d1104870b2c462c999bf684e4af3e62`
parameters.json (canonical JSON, no trailing newline)	`972fbeff2455df00670ab1d4080425567b3c1544cafc233c643e60f833ff48c8`

The parameter digest identifies canonical, sorted compact JSON without the trailing newline. Store the complete parameter values as well as their hash. These hashes bind the inspected baseline, not an externally accepted corrected bank.

D.4 SQL Checks and Their Execution Status

The baseline includes `docs/db/iteration1/tests/run_schema_tests.py`, its SQL behavior checks, import probe and Docker configuration. Run only against a disposable local database, never the hosted production database:

```
docker compose up -d --wait
python docs/db/iteration1/tests/run_schema_tests.py
docker compose down
```

Inspect the baseline test runner/README for its exact expected checks and units. Historical CI reported schema checks and candidate import; this revision did not rerun PostgreSQL and therefore does not present a new SQL pass. Local Python reproduction covers engine/API behavior without starting a database. UI production-build and browser acceptance remain separately unrerun in this revision.

D.5 Reproduction Record and Sign-off

The report's `verification/results/` retains test logs, loader output, D1/D2 output, deterministic/adverse CSVs, first-step timing CSV, parameters and summary. Run `verification/reproduce_baseline.py` from the project root to regenerate local evidence from the export. `verification/environment.json` records package/runtime versions. A named teammate must reproduce the corrected full application, not merely this unchanged baseline, and record their date, versions, commands, results and limitations. That sign-off is outstanding. Peer-review examples and member declarations remain outstanding as specified in Chapter 8.

Appendix E

Additional Design Diagrams

E.1 Supplementary interaction diagrams

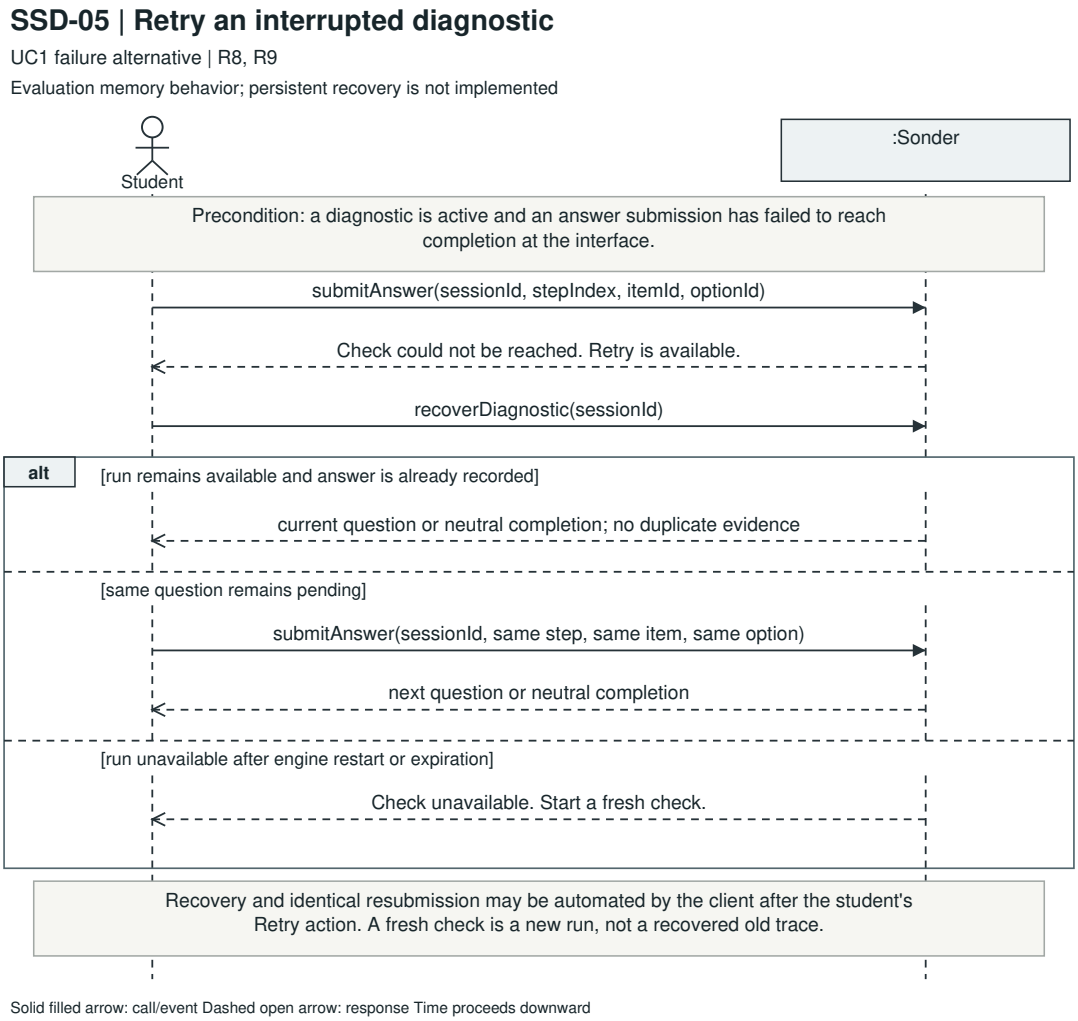


Figure E.1: SSD-05: UC1 interrupted-response recovery (R8, R9). Available memory runs can be retried; a restarted engine loses the run.

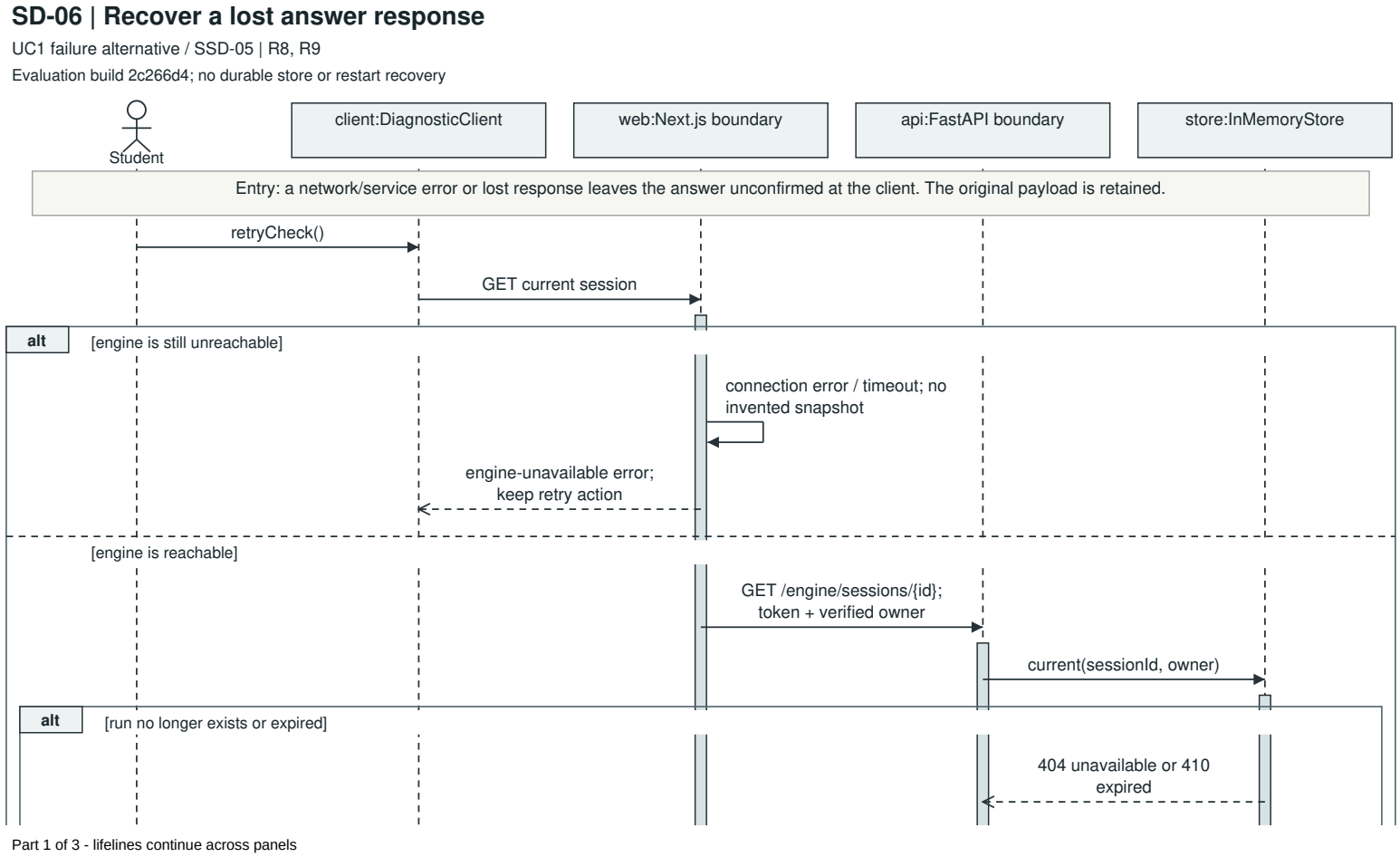


Figure E.2: SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 1 of 3).

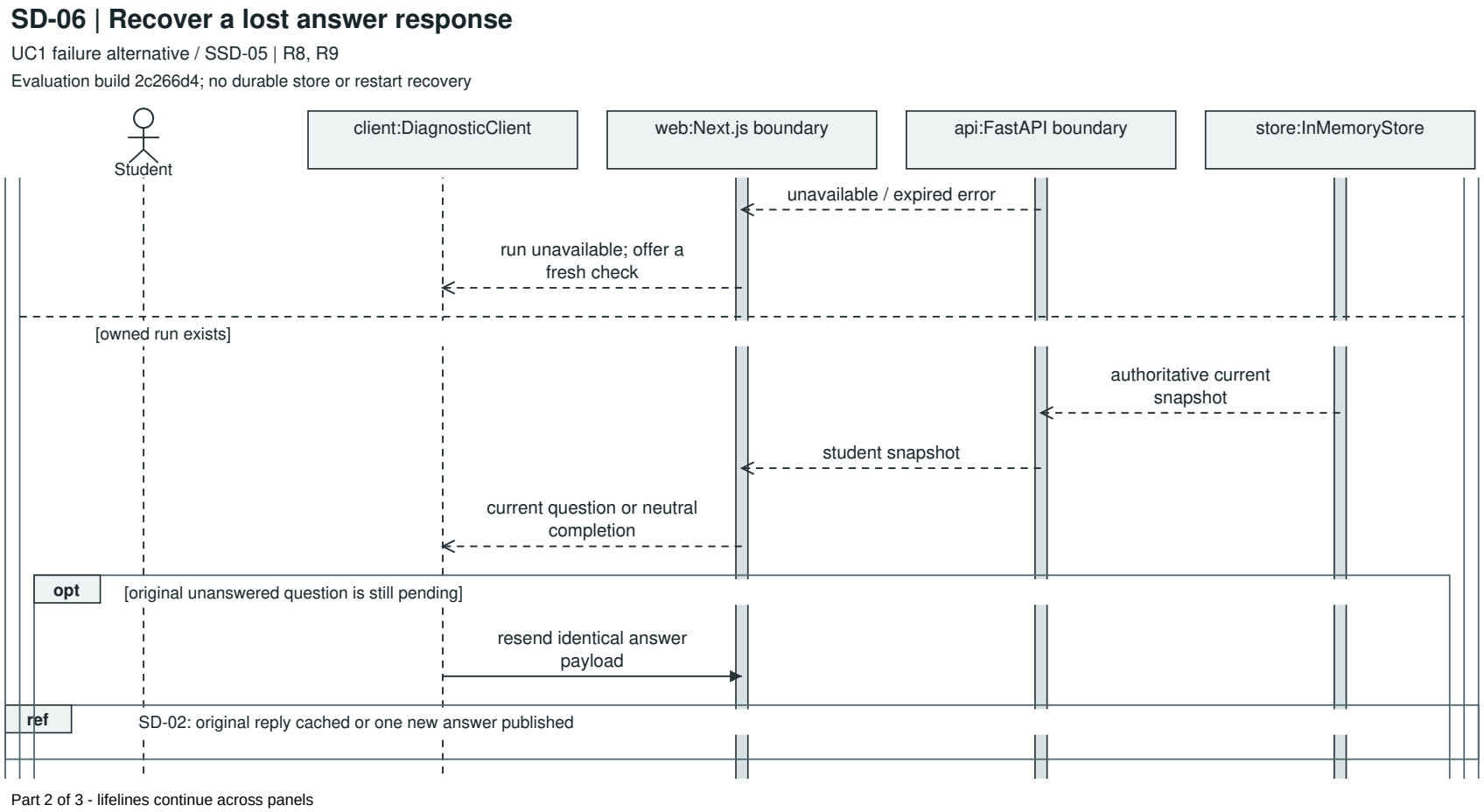


Figure E.3: SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 2 of 3).

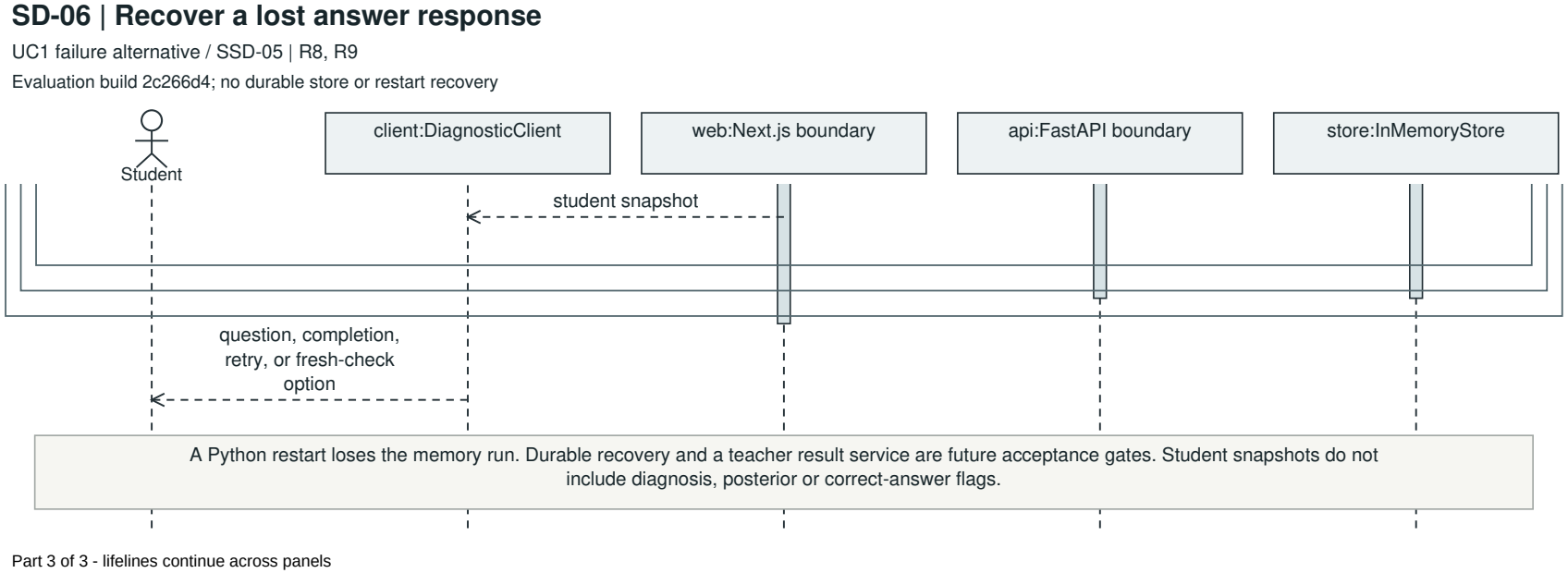


Figure E.4: SD-06: UC1 retry/failure handling (R8, R9). The client reads the authoritative snapshot before resending an unchanged answer; no persistent recovery is claimed. (part 3 of 3).

The following diagrams supplement Section 5.8.1. SSD-02 is the salient refusal scenario of UC1. SSD-03/04 show the external contracts for content review and freezing. SD-01 refines session start; SD-04/05 refine the content contracts. The database rules are implemented and exercised by the schema tests, but the proposed content controllers and their user interfaces are not part of the live evaluation flow. Item approval is distinct from the later teacher approval of a diagnosis. Freezing a response model is a separate transition from freezing content.

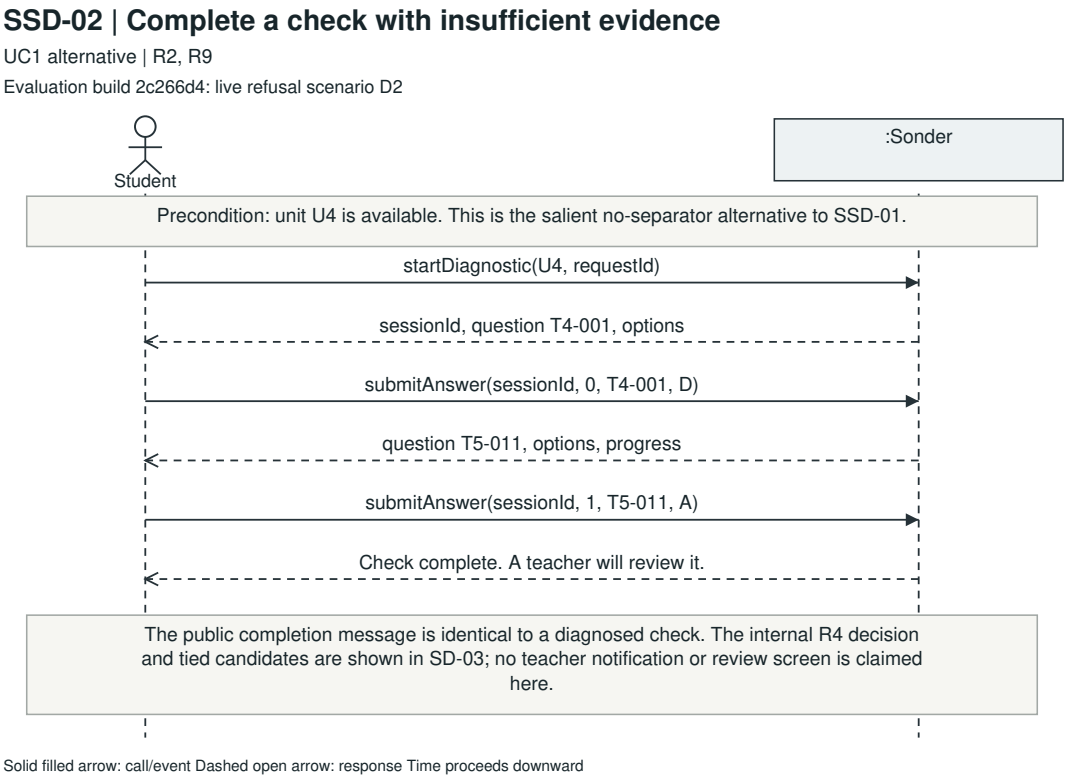


Figure E.5: SSD-02: UC1 refusal alternative (R2, R9). Worked example D2 returns the same neutral completion message as a supported diagnosis.

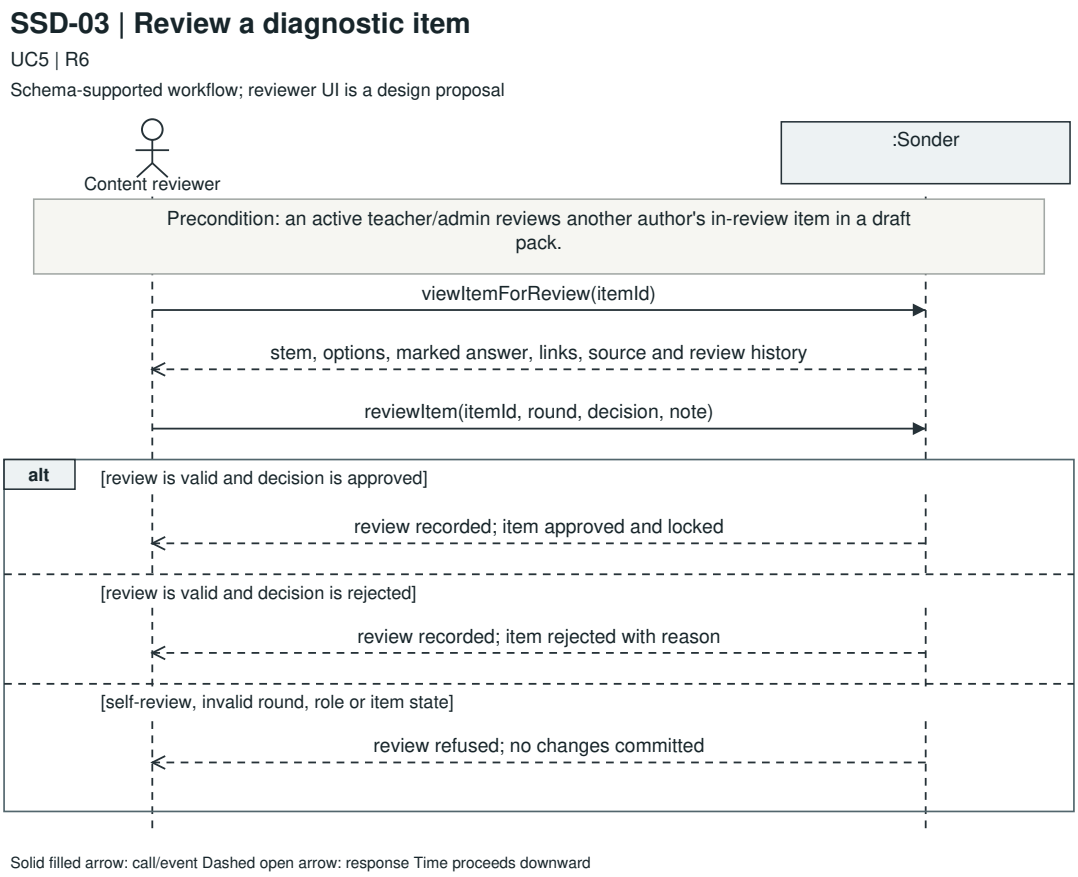


Figure E.6: SSD-03: UC5 content-item review (R6). Schema-supported contract; reviewer UI proposed.

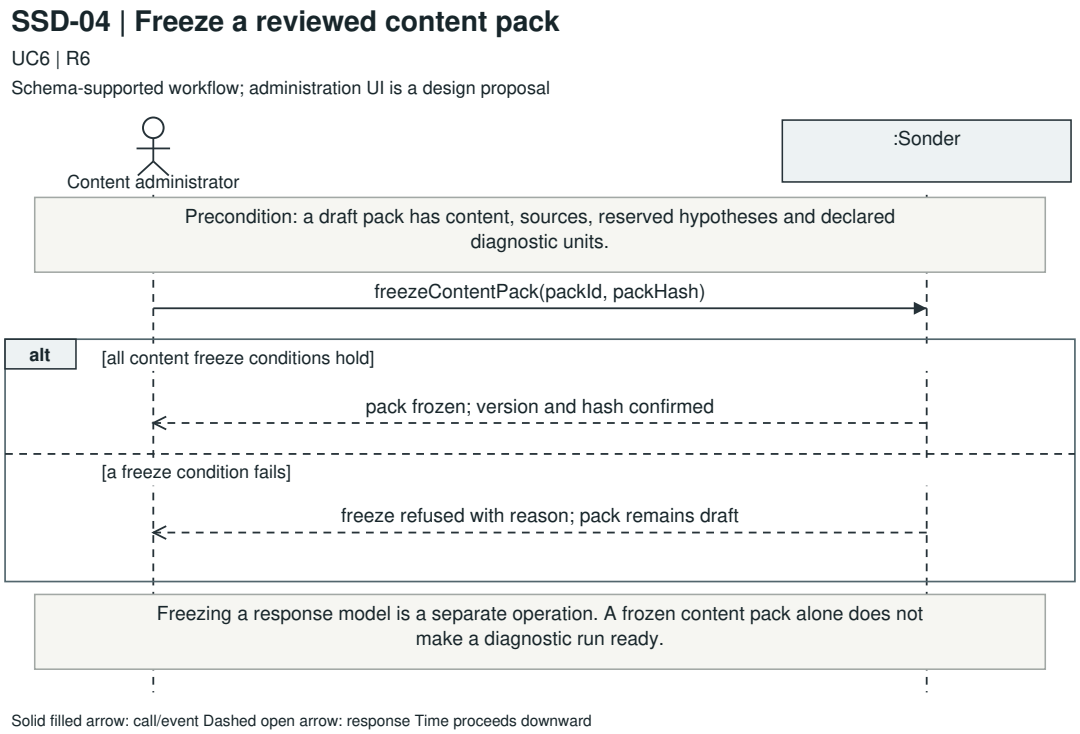


Figure E.7: SSD-04: UC6 content-pack freeze (R6). Schema-supported contract; administration UI proposed.

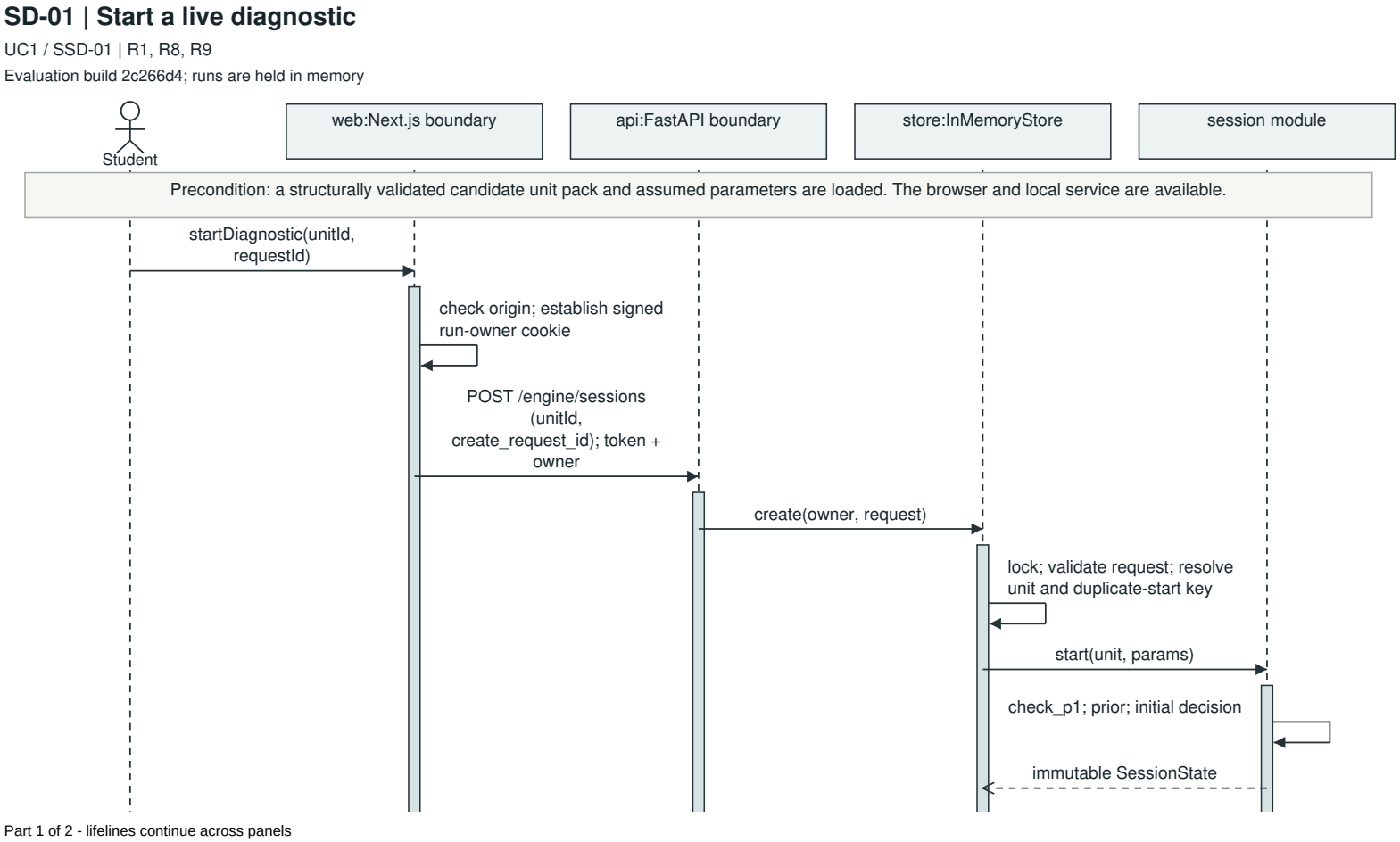


Figure E.8: SD-01: UC1 session start (R1, R8, R9). The loaded pack feeds the immutable engine; a memory-store entry holds the student snapshot and private trace. (part 1 of 2).

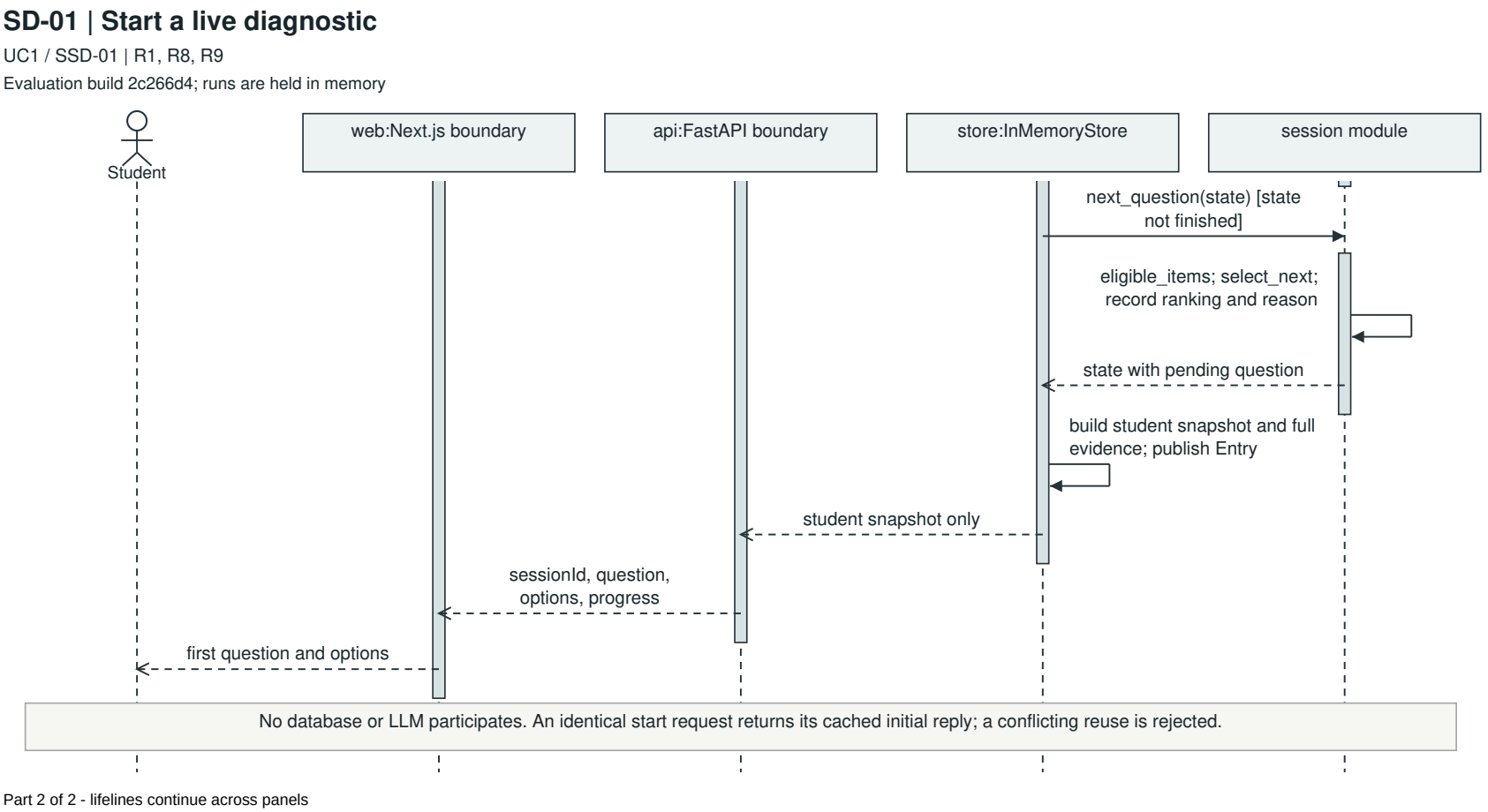


Figure E.9: SD-01: UC1 session start (R1, R8, R9). The loaded pack feeds the immutable engine; a memory-store entry holds the student snapshot and private trace. (part 2 of 2).

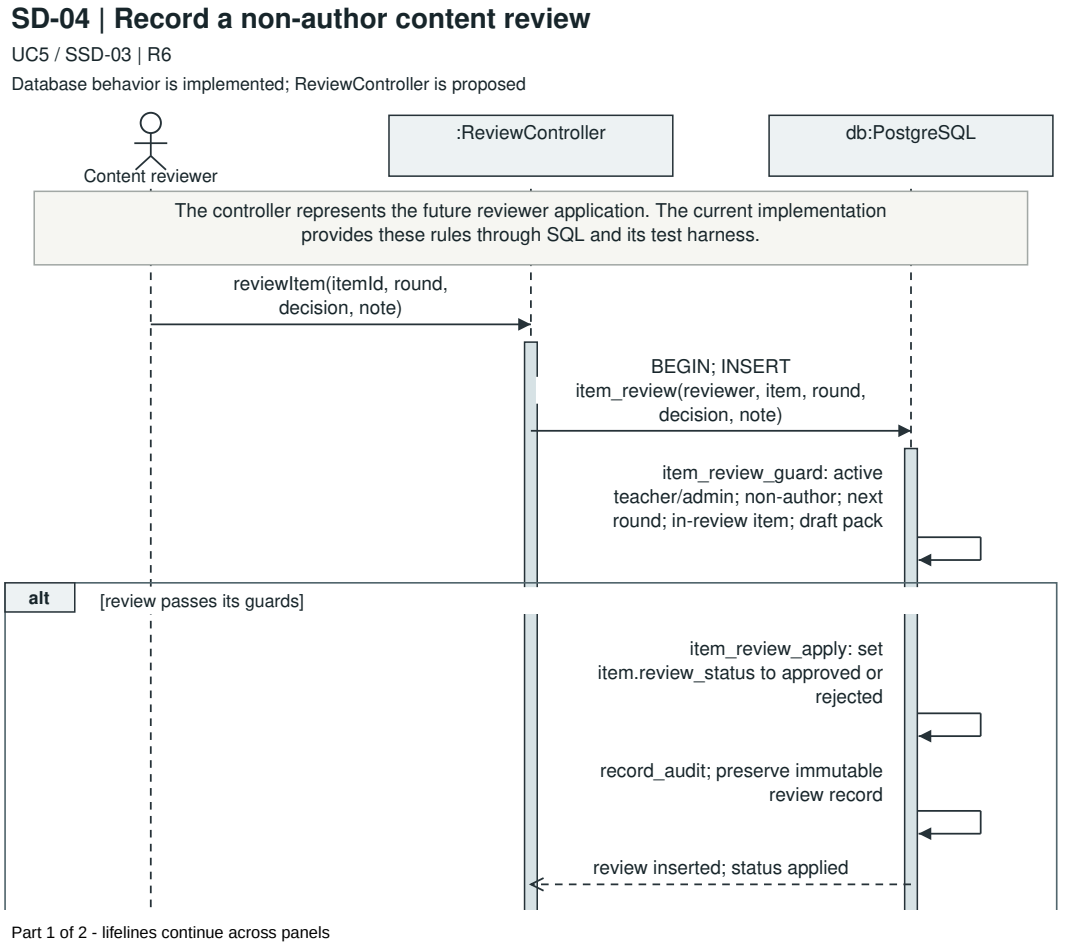


Figure E.10: SD-04: UC5 content review (R6). PostgreSQL applies a valid non-author review and records the audit; invalid writes roll back. The controller is proposed. (part 1 of 2).

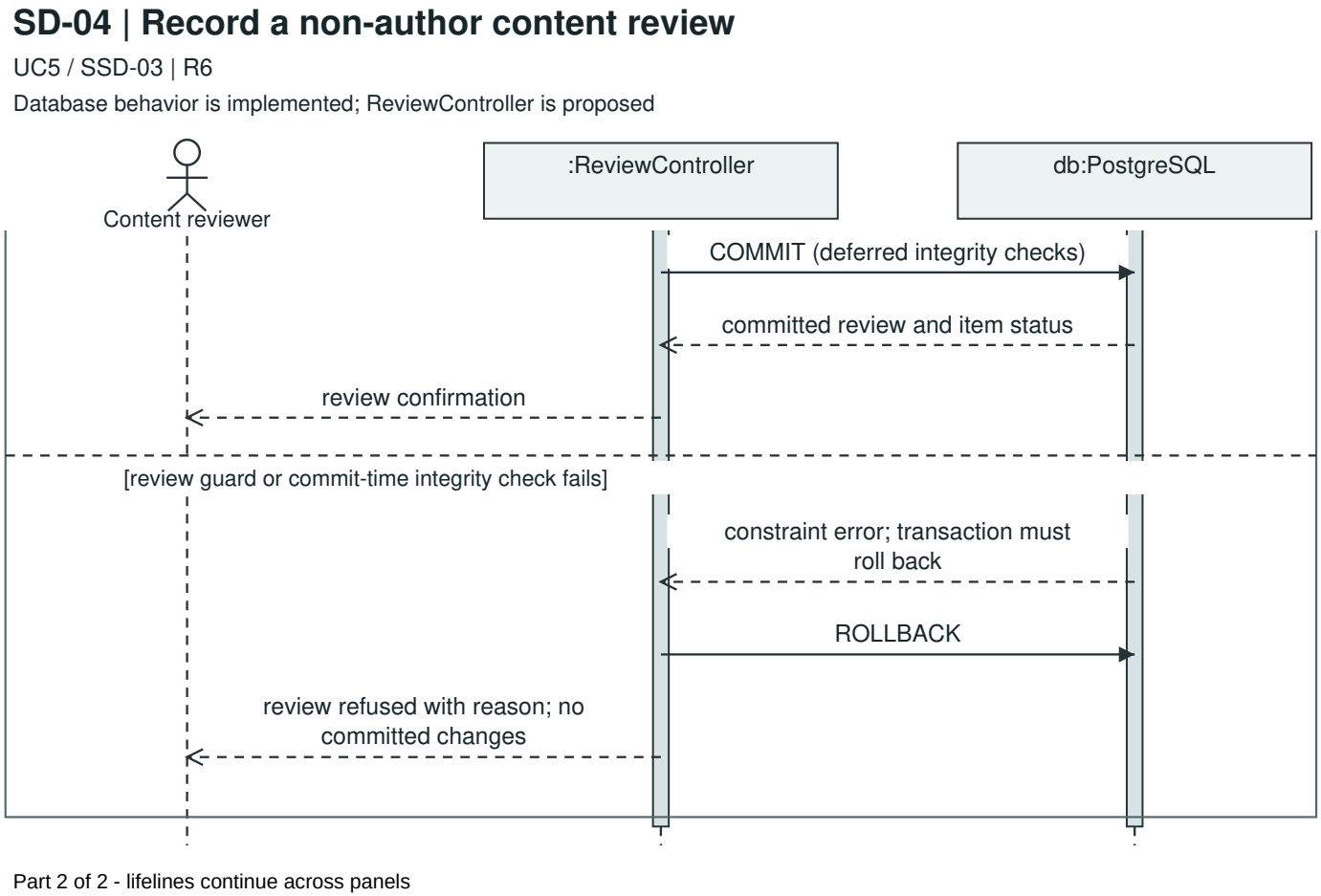


Figure E.11: SD-04: UC5 content review (R6). PostgreSQL applies a valid non-author review and records the audit; invalid writes roll back. The controller is proposed. (part 2 of 2).

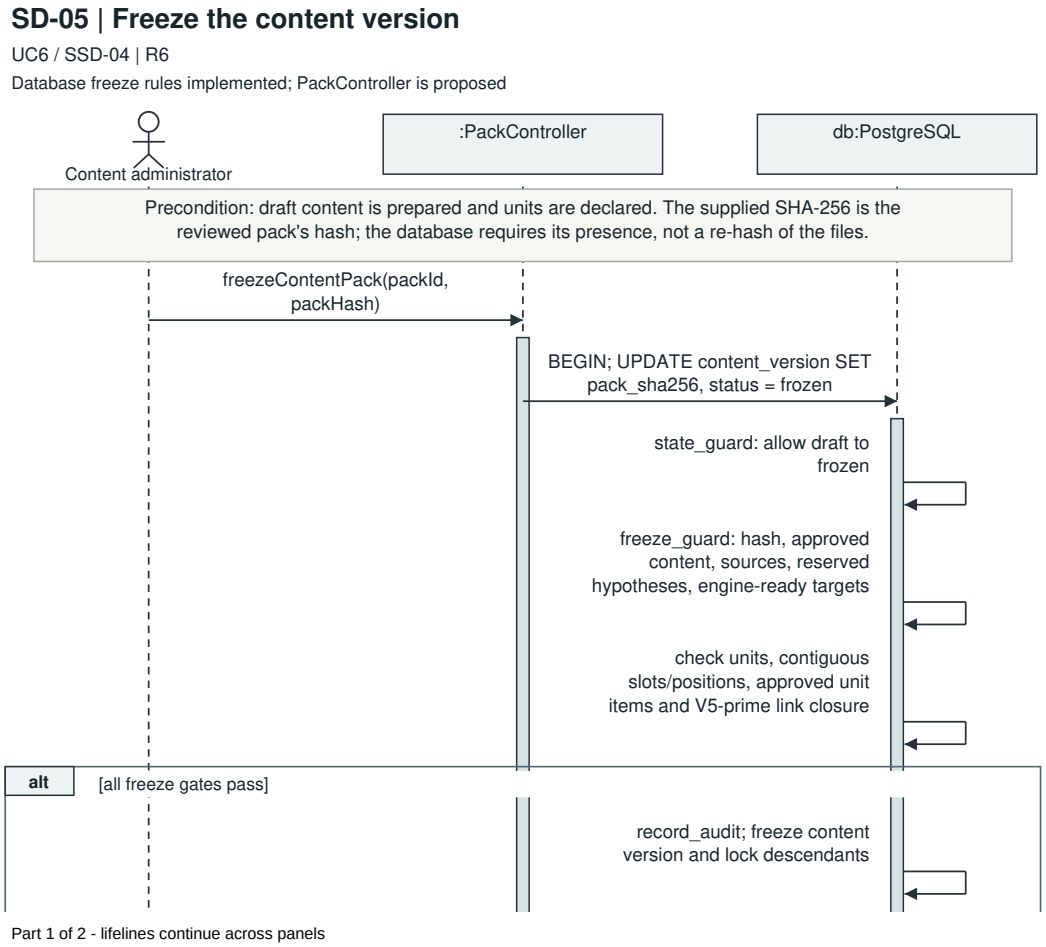


Figure E.12: SD-05: UC6 content freeze (R6). The database checks the content freeze gates and locks the version; response-model freezing follows separately. The controller is proposed. (part 1 of 2).

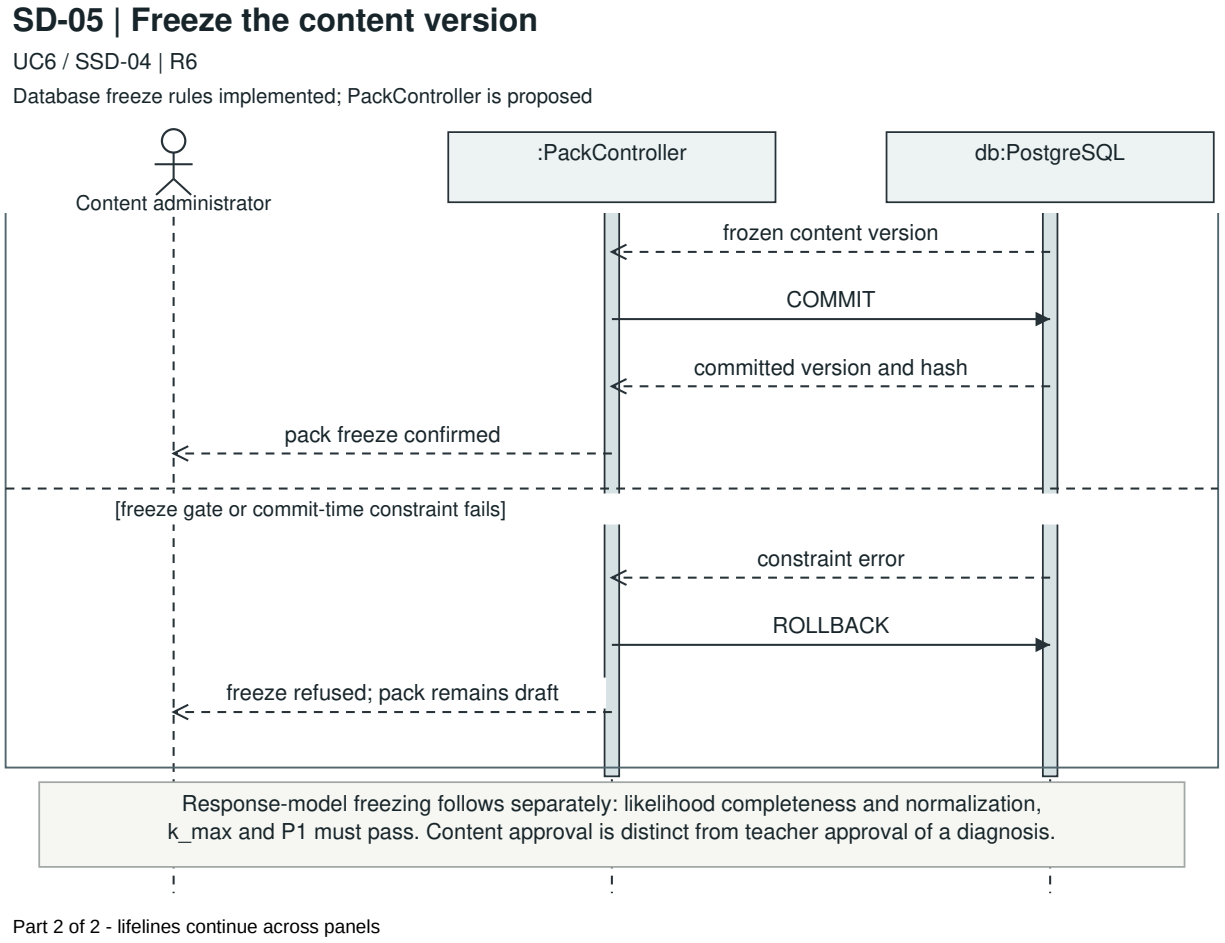


Figure E.13: SD-05: UC6 content freeze (R6). The database checks the content freeze gates and locks the version; response-model freezing follows separately. The controller is proposed. (part 2 of 2).

E.2 Supplementary data diagrams

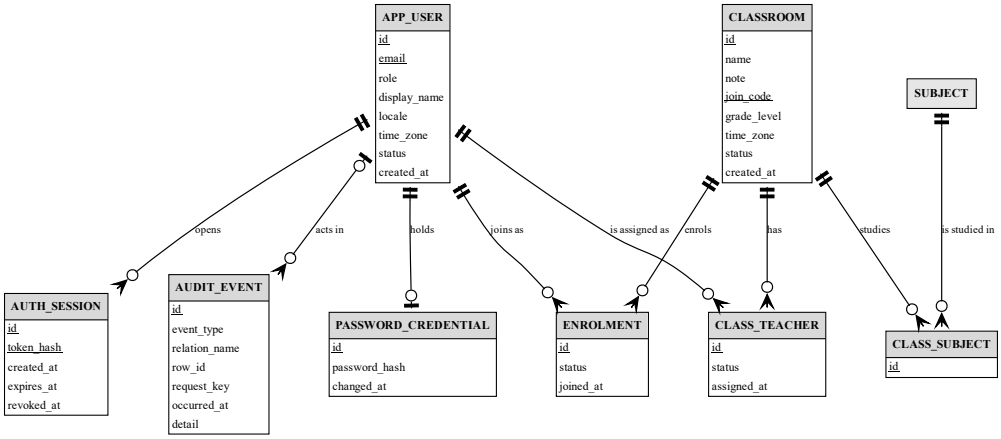


Figure E.14: ERD: users and access, classes and enrolment, and audit.

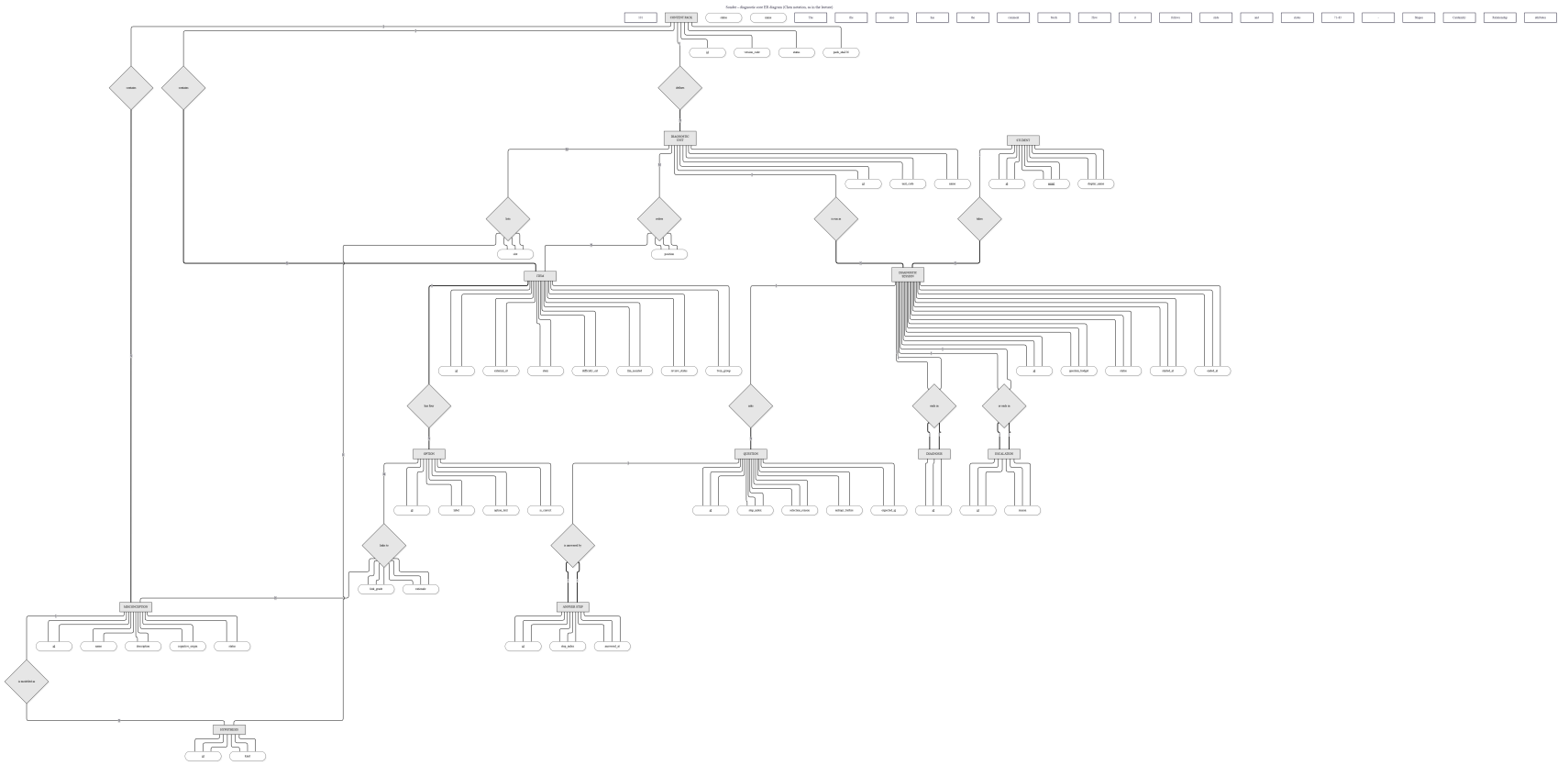


Figure E.15: Team-supplied Chen-notation ERD of the diagnostic core, from Full_ERD_Chen_Notation.png. This logical overview is not a one-to-one export of all 41 physical SQL tables; the area diagrams document those details.

For clear diagram click here.

Appendix F

Meeting and Progress Evidence

Date	Record	Interpretation
9 September 2026	Earlier report mentions a scope discussion	Predates the 11 September panel; cannot establish a response to later feedback; no signed minutes supplied
11 September 2026	Panel date identified in 16 October action summary	Technical challenge, usefulness and coordination follow-ups require dated corrective evidence
12 October 2026	Evaluation baseline `2c266d4`	Implementation date, distinct from later review and report revisions
16 October 2026	*Sonder Mid Evaluation Action Summary*	Maths-only direction, report corrections and proposed bounded work allocation; source file retained in HarrisDOcs
21 October 2026	Current report revision and local reproduction	Editorial corrections and automated evidence; not a supervisor acceptance, member confirmation or external expert decision

The next review must record agreed owners/dates, the persistence-to-teacher-view handoff, challenged peer-review issues and their verification. Those records are still outstanding.

Bibliography

[1] Z. Wang, A. Lamb, E. Saveliev *et al.*, "Results and insights from diagnostic questions: The NeurIPS 2020 education challenge," 2021.

[2] Y. Cheng, "When cognitive diagnosis meets computerized adaptive testing: CD-CAT," *Psychometrika*, vol. 74, no. 4, pp. 619–632, 2009.

[3] ALEKS, "About aleks," https://www.aleks.com/about_aleks/index, product-owner overview; accessed 21 October 2026. Used only for the stated adaptive knowledge-assessment description.

[4] ASSISTments, "Where every math assignment informs learning," https://www.assistments.org/, product-owner description; accessed 21 October 2026. Used only for assignments, feedback and teacher-data claims.

[5] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.